Linköping University | Department of Computer and Information Science
Master's thesis, 30 ECTS | Informationsteknologi
26 | LIU-IDA/LITH-EX-A--26/074--SE

A Method-Agnostic Tool for Threat Analysis and Risk Assessment

Metodagnostiskt Verktyg För Hotanalys och Riskbedömning

Elias Bruno & Erik Lidberg

Supervisor : Zeeshan Afzal
Examiner : Mikael Asplund

External supervisor : Pierre Kleberger

LINKÖPING UNIVERSITY

Linköpings universitet
SE–581 83 Linköping
+46 13 28 10 00 , www.liu.se

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida `http://www.ep.liu.se/`.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: `http://www.ep.liu.se/`.

© Elias Bruno & Erik Lidberg

Abstract

As cyber-physical systems become increasingly integrated and interconnected, risks and threats also become dependent on each component in a complex system. Previously, threat analysis and risk assessments have been done separately to identify and reduce vulnerabilities within each system. However, this is no longer a sustainable practice as cyber-physical systems and IT/OT systems are more dependent on each other than ever. Furthermore, the overlap in methods used to analyze and assess threats and risks for safety and security, respectively, is quite substantial, but there is a lack of tools to leverage this into a more comprehensive process. The need for tools with the capability to support threat analysis and risk assessment in both cybersecurity and functional safety is apparent. This thesis investigates how a method agnostic TARA/HARA framework can be designed and supported in software. The work explores design principles and technical solutions for supporting co-analysis of cybersecurity and functional safety across both OT and IT domains. Particular emphasis is placed on collaboration between multiple stakeholders, support for concurrent risk assessment, and mechanisms for maintaining and evolving risk assessments over time. As part of the study, an artifact is designed and implemented to demonstrate key concepts and functionality. This artifact is then evaluated with respect to threat model agnosticism, usability, flexibility and its support for collaborative and iterative risk assessment. Finally, the thesis investigates the potential role of Large Language Models (LLMs) as decision-support tools for threat identification and analysis.

Acknowledgments

We would like to extend our gratitude to our supervisor at Linköping University, Zeeshan Afzal, for his invaluable guidance and support throughout this thesis work. We also wish to thank our technical supervisors at RISE, Pierre Kleberger, Peter Folkesson, and Mazen Mohamad, for their expertise and constructive feedback.

We would also like to thank our examiner, Mikael Asplund, for his valuable insights and feedback, which helped strengthen the quality of this work. Furthermore, we are grateful to our opponents, Emil Hamrin and Junia Lundstedt Mannervik, for their thoughtful comments and constructive criticism during the review process.

Finally, we would like to express our appreciation to our classmate Jon Svensson Magnusson for his support, encouragement, and engaging discussions throughout the course of this project.

Contents

Abstract iii

Acknowledgments iv

Contents v

List of Figures ix

List of Tables xi

1 Introduction 1
- 1.1 Motivation . . . 2
- 1.2 Aim . . . 2
- 1.3 Research Questions . . . 3
- 1.4 Approach . . . 3
- 1.5 Delimitations . . . 4

2 Background 5
- 2.1 RISE . . . 5
- 2.2 Terminology . . . 5
  - 2.2.1 CIA Triad . . . 6
  - 2.2.2 Threat Analysis and Risk Assessment (TARA . . . 6
  - 2.2.3 HARA . . . 6
  - 2.2.4 STRIDE . . . 6
  - 2.2.5 Attack Trees . . . 7
- 2.3 Threat Models . . . 7
  - 2.3.1 Existing Threat and Risk Assessment (TARA/HARA) Tools . . . 8
- 2.4 Modeling Concepts and System Architecture . . . 9
  - 2.4.1 Data Flow Diagram . . . 9
  - 2.4.2 Modular Monolith . . . 9
  - 2.4.3 Structured Query Language . . . 10
  - 2.4.4 MariaDB and SQL . . . 10
- 2.5 Risk Matrices and Threshold-based Risk Classification . . . 11
- 2.6 SAHARA . . . 12
- 2.7 HEAVENS . . . 14
- 2.8 Large Language Models . . . 17
  - 2.8.1 Examples of Modern LLMs . . . 17
  - 2.8.2 Prompt engineering . . . 17
  - 2.8.3 Human-in-the-loop . . . 17

3 Related Work 18
- 3.1 Literature Search Strategy . . . 18
- 3.2 Previous Work on the Tool . . . 18

3.3 HEAVENS-Based Risk Assessment . . . 19
3.4 SAHARA-based Security Analysis . . . 19
3.5 Adapted SAHARA To Distributed Grids . . . 19
3.6 Threat Tool Evaluations . . . 20
3.7 Attack Tree Risk Assessment . . . 21
3.8 LLM Support for Threat Modeling . . . 21
3.8.1 Structured LLM prompting . . . 22
3.9 Summary of Gaps . . . 23

4 Research Methodology and Design Analysis **24**
4.1 Design Science Research Approach . . . 24
4.1.1 Iteration 1: Method-Agnostic Architectural Design . . . 24
4.1.2 Iteration 2: Support features and Evaluation of LLM Integration Opportunities . . . 25
4.1.3 Mapping of Reaserch Questions to Iterations . . . 25
4.2 Method for RQ1: Method-Agnostic Representation . . . 26
4.2.1 Selection of Frameworks . . . 26
4.2.2 Comparative Structural Analysis . . . 27
4.2.3 Identify Common Elements . . . 27
4.2.4 Analysis of Scoring and Expression Logic . . . 28
4.2.5 Identification of Method Coupling . . . 28
4.2.6 Derived Architectural Requirements . . . 30
4.3 Method for RQ2: Supporting features and Collaboration . . . 31
4.3.1 Analysis of Collaboration Challenges . . . 31
4.3.2 Derived Requirements for Collaborative and Interoperable Risk Assessment . . . 32
4.4 Method for RQ3: Exploration of LLM Integration . . . 33
4.4.1 Selection Procedure . . . 33
4.4.2 Second Literature Review . . . 33
4.4.3 Derived Requirements for LLM implementation . . . 33
4.5 Expert Interview . . . 34
4.6 Summary . . . 35

5 Implementation **36**
5.1 System Overview . . . 36
5.2 Data Model . . . 37
5.2.1 Entity Structure and Core Entities . . . 37
5.2.2 Dimensions . . . 39
5.2.3 Parameters and Parameter Values . . . 40
5.3 Relational Database and MariaDB . . . 40
5.4 Database Design . . . 41
5.4.1 Stored procedures . . . 41
5.4.2 Connection Management . . . 41
5.4.3 Hierarchical *dimension* Structure . . . 42
5.5 Backend Architecture . . . 42
5.5.1 REST Architectural Style . . . 42
5.5.2 Backend Structure and Flask Implementation . . . 45
5.5.3 Security considerations . . . 46
5.6 GUI Design . . . 46
5.6.1 The fronted Architecture . . . 46
5.6.2 Usability Considerations . . . 47
5.6.3 Concurrency model . . . 47
5.7 Supporting Features . . . 48

5.7.1 Building Expressions and Lookup Tables . . . 48
5.8 LLM Implementation . . . 49
5.8.1 Supporting Feature: Data flow Diagram . . . 49
5.8.2 Import and Export . . . 49
5.8.3 Integration Architecture . . . 49
5.8.4 Human in the Loop Validation . . . 51

6 Evaluation **53**
6.1 Design Science Research Perspective . . . 53
6.2 Evaluation Objectives . . . 53
6.3 Method-Agnostic Design Assessment . . . 54
6.4 Extensibility and Method Flexibility . . . 56
6.4.1 Chosen Risk Assessment Methods . . . 56
6.5 Multi-User and Concurrent Assessment Support . . . 60
6.5.1 Test of Multiple Users . . . 60
6.5.2 Performance Evaluation . . . 61
6.6 LLM Risk Suggestion Reliability . . . 62
6.6.1 System description . . . 64
6.6.2 Scenario 1: Complete Risk Sheet . . . 64
6.6.3 Scenario 2: Partial Risk Sheet . . . 67
6.6.4 Scenario 3: System Description Only . . . 70
6.6.5 Summary of the LLM risk suggestions . . . 73
6.7 Interview Results . . . 74
6.8 Summary of Findings . . . 74

7 Discussion **75**
7.1 Results . . . 75
7.1.1 RQ1: Method-Agnostic Representation . . . 75
7.1.2 RQ2: Collaboration, Interoperability, and Supporting Features . . . 77
7.1.3 RQ3: LLM-based Risk Suggestion Support . . . 78
7.2 Method . . . 79
7.3 Evaluation . . . 80
7.3.1 Source Criticism . . . 80
7.3.2 Standard Benchmarks . . . 80
7.3.3 Limited Evaluation . . . 80
7.3.4 LLM reproducibility . . . 80
7.3.5 Language Perspectives . . . 80
7.4 Social and ethical perspective . . . 80

8 Conclusion **82**
8.1 Future work . . . 83

Bibliography **85**

A Appendix A **90**
A.1 Implementing mapping and lookup . . . 90
A.2 Functionality of the Framework . . . 92
A.2.1 Building a Framework . . . 92
A.3 Agnosticity . . . 94
A.3.1 Implementing SAHARA . . . 94
A.3.2 Implementing HEAVENS . . . 96
A.4 Risk Assessment Methods Implementation . . . 98
A.4.1 FMEA . . . 98
A.4.2 FAIR . . . 101

A.4.3 HazOp 102
A.4.4 HTMM 102
A.4.5 OCTAVE Allegro 102
A.4.6 PHA of Canada 103
A.4.7 What-If Analysis 103
A.5 Multiple User Test 104
A.6 Expert interview questions and answers 110

List of Figures

2.1 Data Flow Diagram with trust boundary 10
2.2 DFD icon definitions 10
2.3 Figure of the Modular Monolith architectural pattern 11
2.4 Conceptual overview of the SAHARA method 14
2.5 The phases and workflow when using the HEAVENS framework [**Islam**] 16

4.1 Overview of the methodology used to derive the proposed method-agnostic architecture. 26
4.2 Overview of the tool requirements, illustrating the workflow across Phase 1A (Safety Risk Analysis) and Phase 1B (Cybersecurity Risk Analysis). 28

5.1 Main system components and their communication paths. 37
5.2 ER diagram of the MariaDB database of the proposed artifact. 38
5.3 Example of a hierarchical ES structure consisting of three layers of *dimension*s. A *dimension* can contain either Parameters or additional sub-*dimension*s, allowing nested aggregation and structured risk calculation. 40
5.4 Sequence diagram illustrating the configuration workflow in the TARA/HARA tool. The diagram shows how users define *dimension*s, create or select parameters, and assign value scales through the frontend. These actions are propagated through the REST-based backend to the database layer, where stored procedures are used to persist the configuration. 44
5.5 Sequence diagram illustrating the parameter-based risk evaluation process. The diagram shows how parameter values are iteratively submitted and processed across the system layers, and how stored procedures in the database are used to aggregate these values and compute a final risk level. 45
5.6 Component diagram illustrating the layered architecture of the TARA/HARA tool. The system is structured into four layers: the React-based frontend, the Flask REST API backend, the Python database access layer, and the MariaDB database. Each layer communicates strictly with the layer directly below it. 47
5.7 Data Flow Diagram of the Authentication Service showing external entities, processes, data stores, and identified security risks. 49
5.8 AI-generated risk suggestions based on system context and existing inputs, including severity classification and explanatory rationale. The user can review and selectively include suggestions, ensuring human-in-the-loop validation. 52
5.9 Human-in-the-loop workflow for AI-generated risk suggestions. The system validates AI confidence scores before presenting suggestions to the user for manual review and acceptance. 52

6.1 Response time percentiles (P50, P95, P99) for selected API requests under 20 and 50 concurrent users. 62
6.2 Reference risk assessment sheet extracted from external source [**etsi_tvra_2017**] . . 63
6.3 First sequential LLM evaluation run using previously accepted AI-generated risks as additional context. 65

6.4 Second sequential LLM evaluation run using previously accepted AI-generated risks as additional context. . . . 66
6.5 Second sequential LLM evaluation run using previously accepted AI-generated risks as additional context.. . . . 67
6.6 First iteration of LLM-generated risk suggestions for a risk assessment sheet with a subset of risks. . . . 68
6.7 Second iteration of LLM-generated risk suggestions for a risk assessment sheet with a subset of risks. . . . 69
6.8 Third iteration of LLM-generated risk suggestions for a risk assessment sheet with a subset of risks. . . . 70
6.9 Summary of initial empty risk assessment. . . . 71
6.10 Summary of iterative LLM-generated risk suggestion. . . . 72
6.11 LLM-generated risk suggestions based on system description alone. . . . 73

A.1 Implementation workflow for defining lookup tables and ranges. . . . 91
A.2 Initial steps in creating a framework: defining dimensions. . . . 92
A.3 Defining parameters and expressions in the framework. . . . 93
A.4 Final steps: creating and configuring the RAM. . . . 94
A.5 Implementation of SAHARA. . . . 95
A.6 Implementation of HEAVENS. . . . 97
A.7 FMEA 1 98
A.8 FMEA 2 98
A.9 FMEA 3 99
A.10 FMEA 4 100
A.11 FMEA 5 101
A.12 FAIR Loss Table 101
A.13 FAIR Multiple Scenario Table 102
A.14 Hazard and Operability Study implementation 102
A.15 HTMM implementation 102
A.16 OCTAVE Allegro Evaluation Sheet 1 102
A.17 OCTAVE Allegro Evaluation Sheet 2 103
A.18 OCTAVE Allegro Evaluation Sheet 3 103
A.19 PHA of Canada Risk Assessment Method 103
A.20 What-If Analysis method for qualitative risk assessment through structured questioning 104
A.21 User A inviting a user to a project. . . . 105
A.22 User A after inviting a user to a project. . . . 106
A.23 User A baseline before editing. . . . 107
A.24 User B baseline before editing. . . . 107
A.25 User A while editing row. . . . 108
A.26 User A after saving edit. . . . 108
A.27 User B after changes saved by user A. . . . 109

List of Tables

2.1 STRIDE threat categories used for threat identification [**microsoft_stride_threat_model**] 7
2.2 Threat modeling tools and supported threat categories and security analysis frameworks . . . 9
2.3 Risk estimation matrix [**ISO14121-2-2012**]. . . . 11
2.4 Illustrative example of threshold-based risk classification. . . . 12
2.5 Security Level (SecL) determination according to the SAHARA method .[**macher2014combined**] . . . 13

3.1 Risk Assessment Parameters and Scoring Criteria [**thorsen2022combined**] . . . 20

4.1 Summary of derived requirements . . . 35

6.1 Survey questions and corresponding AI features. . . . 55
6.2 Survey results for AI feature evaluation. . . . 55
6.3 FMEA representation in the tool . . . 57
6.4 What-If Analysis representation in the tool . . . 57
6.5 RAM at the public health Agency of Canada representation in the tool. . . . 58
6.6 HTMM representation in the tool. . . . 58
6.7 OCTAVE Allegro representation in the tool . . . 59
6.8 FAIR representation in the tool . . . 59
6.9 Summary of Method Representation Scores . . . 60
6.10 Performance comparison between 20 and 50 concurrent users. . . . 61

A.1 Questions and answers part 1. . . . 111
A.2 Questions and answers part 2. . . . 112
A.3 Questions and answers part 3. . . . 113

1 Introduction

Modern Information Technology (IT) systems and Operational Technology (OT) systems are increasingly connected. In domains such as energy systems, automotive, and transportation services, physical processes are tightly coupled with software-controlled components, networked communication, and cloud-based services. This development has created complex cyber-physical systems in which cybersecurity incidents may affect not only information assets, but also operational processes and physical safety.

Risk assessment provides a structured way to identify weaknesses, evaluate threats and hazards, and define appropriate mitigation measures. However, as systems grow in scale, complexity, and interconnectedness, traditional approaches to risk assessment become more difficult to maintain. Without structured tool support, the process of identifying, categorizing, documenting, and mitigating risks can become prone to human error, lack traceability, and become difficult to keep consistent throughout the system life cycle.

A particular challenge is that existing tools, standards and, specific methodologies are often tightly coupled, limiting their flexibility and long-term applicability. A threat model can be understood as a structured representation of potential adversaries, attack vectors, system assets, and vulnerabilities within a defined system context. Different threat modeling and risk assessment methodologies define how these elements are identified, categorized, analyzed, and mitigated. However, tools that are designed around one specific methodology may be difficult to adapt to new system architectures, interdisciplinary workflows, or combined safety-security analysis. Therefore, there is a need for a flexible and threat method agnostic tool that can support multiple risk assessment approaches within a unified, consistent framework. In this thesis, a method agnostic tool refers to a tool that is not tied to one specific risk assessment method, but is capable of deploying, representing and, supporting multiple threat method.

The need for such flexibility is particularly relevant in IT/OT environments, where technologies, stakeholders, and regulatory requirements evolve over time. Due to the increasing integration of software into complex systems, the risks of attacks and failures have to be analyzed [1, 2]. New methods for analyzing security and safety are continuously being developed [3, 4, 5], while existing tools are often adapted to one specific method or workflow. Despite increased adoption of threat modeling practices, limitations remain in the flexibility and long-term usability of existing threat modeling tools. Since tools are tightly coupled to specific methodologies, on the other hand, standards are not revised often, thus, the methods

are not updated often neither. Yet, new technology is introduced that needs to be adapted to, e.g., usage of cloud in energy domain. This creates the need for a more generalizable and adaptable tool that can support new risk assessment methods adapted to analyze both cybersecurity and functional safety that can support different threat modeling and risk assessment approaches [6].

By decoupling the logic in a tool from a specific method, it becomes possible to maintain a single source of risk assessment data while allowing different specialists to apply the analysis methods most relevant to their specific domains. This agnostic architecture is also a prerequisite for effective co-analysis of functional safety and cybersecurity. By not forcing a choice between a safety-centric approach like Hazard Analysis and Risk Assessment (HARA) [7] or a security-centric one like STRIDE [8], a method-agnostic tool can bridge the gap. Furthermore, such tool may allow safety engineers and cybersecurity experts to collaborate within the same environment, ensuring that the inter-dependencies, such as cyberattacks leading to a physical grid failure are captured. Finally, a model-agnostic approach may also improve interoperability between practitioners by making it easier to exchange, reuse, and adapt risk assessment methods across projects and methods.

1.1 Motivation

The main motivation is the need to support co-analysis of cybersecurity and functional safety. Safety-oriented methods such as Hazard Analysis and Risk Assessment (HARA) focus on hazards and functional safety, while cybersecurity-oriented approaches such as STRIDE focus on threats and malicious actions. In cyber-physical systems, these perspectives may interact and therefore a tool that supports multiple methods within a shared framework may help analysts document and reason about such dependencies.

Furthermore, the thesis also explores the potential role of Large Language Models (LLMs) to assist in threat identification and analysis, potentially increasing the efficiency of the assessment process. Threat identification often depends on expert knowledge and can be time-consuming. Recent advances in LLMs offer new opportunities for knowledge extraction and reasoning. LLMs may provide support by suggesting candidate threats, assets, or risk scenarios. However, their use in safety- and security-critical contexts introduces significant challenges related to reliability, trustworthiness and accountability. Therefore, this thesis investigates LLM-based support as a human-supervised feature rather than as an autonomous risk assessment mechanism.

1.2 Aim

The aim of this thesis is to investigate how commonly used Threat Analysis and Risk Assessment (TARA) frameworks can be represented and supported by a software tool for energy communities. The work focuses on enabling a coordinated analysis of risks related to both functional safety and cybersecurity, taking into account interactions between OT and IT.

More specifically, the thesis aims to design and implement a proof-of-concept prototype of a threat-model agnostic tool that supports structured risk assessment processes. The prototype should allow different analysis methods, including safety-oriented and cybersecurity-oriented analyses to be conducted in the same tool. In addition, the work aims to explore how such a tool can support collaboration between stakeholders, facilitate import and export of assessment data and enable continuous risk management over time. In the context of this thesis, collaboration refers to the ability of multiple stakeholders to contribute to a shared risk assessment while preserving consistency and traceability. Interoperability refers to the ability to exchange, reuse, or structure assessment information across methods, projects, or stakeholders.

As a secondary objective, the thesis explores whether LLM-based support can be integrated into the prototype to assist human analysts in identifying threats and risk scenarios.

1.3 Research Questions

This thesis addresses the following research questions (RQs) covering representation, architecture, collaboration, interoperability, and LLM integration:

1. RQ1: To what extent can a common HARA/TARA framework be represented within a method-agnostic software architecture that supports both functional safety and security analysis?
2. RQ2: How effectively does the proposed artifact support collaboration and interoperability among stakeholders involved in risk assessment processes?
3. RQ3: What are the opportunities and limitations of integrating LLM-based support into a method-agnostic HARA/TARA tool for threat identification and risk analysis?

1.4 Approach

To address the identified challenges, this thesis adopts a Design Science Research (DSR) approach focused on the iterative development and evaluation of a software artifact. The work is structured into two primary iterations, which together contribute to answering the RQs defined in Section 1.3.

In the first iteration, a literature review and requirement analysis are conducted to identify the core components of a method-agnostic TARA framework. Established methodologies, such as STRIDE, HEAVENS and SAHARA, are analyzed to extract their structural and logical elements. The objective of this phase is to abstract these elements into a unified and extensible data model capable of representing multiple HARA/TARA approaches within a shared architectural structure. This iteration primarily addresses RQ1. The resulting architectural foundation enables the design and implementation of a modular software artifact that explicitly decouples the system model from method-specific threat modeling logic. Requirements related to collaboration, traceability, and interoperability are also identified in this phase, forming the basis for the features evaluated in relation to RQ2.

The second iteration focuses on supportive functionality, mainly brought up in RQ2 and RQ3, and explores how LLMs could be integrated into the TARA tool to support structured threat identification and analysis. Rather than implementing full AI functionality, this phase investigates suitable integration points within the risk assessment workflow. Particular emphasis is placed on maintaining human oversight in any potential AI-assisted process.

Finally, the developed artifact is evaluated through a series of technical tests, comparisons with existing threat assessment frameworks and semi-structured interviews in order to assess its practical applicability. The technical tests evaluate whether the prototype can represent selected risk methods and verifies the interoperability in the tool. Comparisons with selected existing frameworks evaluates coverage against concepts from HARA/TARA frameworks. The semi-structured interviews evaluate the usefulness and supporting features of the tool. The evaluation as a whole examines the tool's ability to represent the interplay between IT and OT risks, its support for concurrent and collaborative risk assessment, and its overall interoperability between stakeholders. This evaluation contributes directly to answering RQ2 and RQ3. Additionally, the findings from the exploration of LLM integration are assessed in relation to their feasibility and alignment with safety-critical risk assessment requirements. More details on the method are provided in Section 4.1.

1.5 Delimitations

The scope of the thesis is limited to the design and implementation of a proof-of-concept tool for supporting Threat Analysis and Risk Assessment (TARA) in the context of IT and OT systems. Due to time constraints and broad scope of the research area, several delimitations have been made.

Although, the tool follows a method-agnostic approach, given the large numbers of varied risk assessment methods that exist, only a subset have been selected and tested in this thesis. For example, SAHARA and HEAVENS are used to demonstrate the extensibility and the flexibility of this tool. Furthermore, the developed artifact is a proof-of-concept, focusing more on the demonstration of architectural feasibility rather than a production ready system, ready to be delivered to the industry. Therefore, aspects such as large scale deployment and extensive real world validation are outside the scope of this work. Finally, the integration of LLMs is limited to a feasibility exploration. The LLM functionality is implemented and evaluated within the tool as a supportive feature for risk identification, rather than a fully autonomous component. All LLM-generated suggestions are assumed to require human review and validation before being incorporated into an assessment.

2 Background

This chapter provides the required background for the thesis and serves as a foundation for the rest of the thesis. It introduces concepts related to threat modeling, risk assessment, cybersecurity an functional safety. Together with the architectural and technical foundations underlying the proposed method-agnostic TARA/HARA tool.

2.1 RISE

RISE Research Institutes of Sweden (RISE) is a state-owned research organization whose mission is to strengthen the competitiveness and innovative capacity of Swedish industry and society through the development of sustainable, technology-driven solutions. RISE operates at the intersection of academia, the public sector and industry providing applied research, testing, and certification services across a wide range of technical domains. The unit involved in this project consists of researchers and experts with backgrounds in cybersecurity and safety engineering. Their expertise has been utilized throughout the different phases of the design science research process applied in this thesis.

2.2 Terminology

This section will establish central concepts and terminology. The work combines concepts from both cybersecurity and functional safety. An *asset* refers to an system component, resource and/or, information considered valuable and requires protection. A *threat* is a potential cause of an unwanted incident that may exploit a system weakness. A *vulnerability* is a weakness or flaw that can be exploited by a threat actor or lead to hazardous situations [9].

Within the functional safety context a *hazard* refers to an potential source of harm caused by an malfunctioning system behavior. An *Risk* is defined as the combination of the likelihood of an event occurring and the severity of its consequences. In cybersecurity-related assessments, risk often includes factors such as attacker capability, exploitability, and potential impact. *Likelihood* describes the probability or feasibility of a threat or hazardous event occurring, while *impact* describes the resulting consequences on safety, security, operational performance, or other protected assets. A *mitigation* refers to a safeguard, control, or countermeasure intended to reduce the likelihood or impact of identified risks [9].

2.2.1 CIA Triad

The CIA triad (Confidentiality, Integrity, Availability) is a well-known foundational model in cybersecurity. This model serves as a foundation when designing threat modeling tools. The model itself describes the essence of security aspects of a system [10].

- **Confidentiality** ensures that information is accessible only to authorized entities and is protected against unauthorized disclosure.
- **Integrity** ensures that data and system functions are protected against unauthorized modification or manipulation.
- **Availability** ensures that systems and data remain accessible and operational when required.

These properties are often used to reason about the potential impact of threats and to classify security objectives in threat modeling and risk assessment.

2.2.2 Threat Analysis and Risk Assessment (TARA

Threat Analysis and Risk Assessment (TARA) is a structured cybersecurity risk assessment method commonly used in the cyber-physical and industrial systems. The purpose is to define and identify potential attack scenarios and to determine appropriate cybersecurity controls. Unlike HARA, which focuses on hazards caused by unintentional failures, TARA addresses risks arising from intentional and malicious actions.

A typical TARA process consists of several different steps, that includes an system definition, asset identification, attack path analysis, mitigation definition, and risk evaluation. The process begins with the identification of valuable assets and system functions. Followed by analyzing possible threats, vulnerabilities and vectors of attack targeting the asset. The result of an TARA process is often an structured classification and prioritization of cybersecurity risks, which then can support decision making regarding security requirements and mitigation. TARA methods are commonly utilized to support compliance with cybersecurtiy standards and to improve the resilience of IT and OT systems. In the context of this thesis, TARA is very relevant since the proof-of-concept tool aims to support a flexible and method-agnostic risk assessment workflow, across both functional safety and cybersecurity domains. Therefore, understanding how TARA frameworks structure assets, threats, risks, and mitigation is an central role to the design of the artifact proposed.

2.2.3 HARA

Hazard Analysis and Risk Assessment (HARA) is an approach that focuses on identifying safety hazards and evaluating the associated risks [7]. The main purpose of HARA is to identify hazardous events and assess their associated risks by analyzing severity, exposure and controllability. HARA focuses exclusively on functional safety and addresses hazards resulting from unintentional failures rather than malicious or intentional actions, which are handled by separate cybersecurity oriented analyses [7]. The result of a HARA process is typically a classification of hazardous events according to risk parameters such as severity, exposure, and controllability. HARA only focus on functional safety and addresses hazards resulting from unintentional failures rather than malicious or intentional actions, which are handled by separate cybersecurity oriented analyses.

2.2.4 STRIDE

Microsoft's STRIDE approach is a threat modeling methodology commonly used to systematically identify security threats during system design and analysis. STRIDE considers six

categories of threats: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege [8]. Rather than serving as a formal threat taxonomy, STRIDE provides a structured framework that helps analysts identify potential threats affecting different system components and interactions. STRIDE primarily focuses on identifying first-order threats [11].The approach often uses graphical representations of the system under analysis, such as Data Flow Diagrams (DFD) or Unified Modeling Language (UML) to get an overview of each component [8]. Each component, data flow, and interaction can then be analyzed according to the STRIDE threat categories to identify potential security threats and vulnerabilities. In addition, trust boundaries are commonly introduced to distinguish trusted parts of the system from untrusted entities or communication paths, In addition, trust boundaries are commonly introduced to distinguish trusted parts of the system from untrusted entities or communication paths, as discussed later in Figure 2.1.

Table 2.1: STRIDE threat categories used for threat identification [8]

Threat Category	**Description**
Spoofing Identity	An attacker pretends to be another user, service, or system component.
Tampering with Data	Unauthorized modification of data at rest, in transit, or during processing.
Repudiation	A user or attacker can deny having performed an action due to insufficient logging or accountability.
Information Disclosure	Confidential or sensitive information is exposed to unauthorized parties.
Denial of Service	System availability is disrupted or degraded, preventing intended use.
Elevation of Privilege	Occurs when an attacker gains permissions beyond those intended for their role

2.2.5 Attack Trees

Attack trees are a structured method for modeling security threats [12]. An attack tree represents an attacker's objective as the root node, while subordinate nodes describe alternative or necessary steps required to achieve that objective. Attack trees are commonly used in threat modeling and risk assessment to analyze possible attack paths and identify vulnerabilities in a system. By decomposing complex attacks into smaller steps, the method helps security analysts understand how different attack scenarios may occur and where mitigation measures can be applied.

2.3 Threat Models

Threat modeling is a systematic method used to identify, analyze, and prioritize security threats toward a system during its design and development phases. By abstracting a system into a simpler model, often consisting of components, data flows and assets, modeling enables a structured examination of how threats may arise and the following consequences. The typical process involves identifying valuable assets, analyzing possible attack vectors and threats, and documenting potential security concerns and system weaknesses. The activity of creating and using threat models is commonly performed iteratively as systems evolve and as architectural changes, new requirements, or emerging threats alter the threat landscape [6].

2.3.1 Existing Threat and Risk Assessment (TARA/HARA) Tools

To understand the landscape of threat modeling and risk assessment, an inventory of both commercial and open-source tools has been compiled. These tools are used to systematically identify system design vulnerabilities and suggest mitigation's. The tools have several differences both technically and in their user interfaces, but they share several characteristics:

- Method-based identification: Many of the tools rely on well-established frameworks such as STRIDE to ensure a comprehensive analysis of potential threats.
- Architectural focus: The analyses in these tools are often based on defining the architecture via Data Flow Diagrams (DFDs).
- Reporting and Documentation: A central function is the generation of traceable documentation used to demonstrate regulatory compliance and provide a basis for security-related decision-making.
- Mitigation Management: Beyond just identifying threats, these tools suggest specific security controls (mitigations) to reduce identified risks to an acceptable level.

Table 2.2 presents a comparison of the identified tools and the threat modeling approaches they support. Some of the tools do not have a check mark because they are general-purpose security platforms rather than dedicated threat modeling tools, or they utilize proprietary/-custom methodologies that do not strictly adhere to a single standardized framework like STRIDE or LINDDUN. The tools that are presented in Table 2.2 were provided by RISE from their previous investigations of threat modeling and risk assessment tools. Information regarding supported methodologies and functionality was collected from the official documentation and websites of the respective tools. The columns in Table 2.2 indicate the support for different threat modeling concepts and approaches, where STRIDE [13] refers to the STRIDE threat categorization model, LINDDUN to an privacy oriented threat modeling, OWASP TMT to support for OWAS Threat Modeling concepts [14] , CIA to confidentiality [10], integrity, and availability-oriented analysis, and Custom to proprietary or tool-specific methodologies.

In the context of this thesis, TARA is particularly relevant because the proposed tool aims to support flexible and method-agnostic risk assessment workflows across both cybersecurity and functional safety domains. Therefore, understanding how TARA frameworks structure threats, risks, assets, and mitigation strategies is central to the design of the proposed artifact.

Table 2.2: Threat modeling tools and supported threat categories and security analysis frameworks

No.	Tool	STRIDE	LINDDUN	OWASP TMT	CIA	Custom
1	Microsoft Threat Modeling Tool [13]	✓				
2	THREATGET [15]	✓				
3	OWASP Threat Dragon [14]	✓	✓	✓	✓	
4	IriusRisk Community [16]	✓		✓		✓
5	Eclipse Capella (+DARC viewpoint) [17]					✓
6	PyTM [18]	✓				
7	Threat Composer (AWS) [19]	✓				
8	SD Elements [20]	✓		✓		✓
9	ThreatModeler [21]	✓		✓		✓
10	SystemWeaver [22]					✓
11	Threats Manager Studio [23]	✓				
12	Varonis Data Security Platform [24]					
13	CAIRIS [25]					✓
14	Threagile [26]	✓				✓
15	Yacraf [27]					✓

2.4 Modeling Concepts and System Architecture

2.4.1 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of how data flows through a system. It commonly includes processes, data stores, external entities, data flows, and trust boundaries [28]. DFDs are commonly used in threat modeling to provide a structured overview of the system that supports systematic identification of security threats for example using STRIDE [6].

Figure 2.1 illustrates the functional architecture of a system using a DFD. The model adheres to the Microsoft Threat Modeling tool conventions by utilizing standardized elements to represent system logic.

The diagram defines the system's Trust boundary, which serves to isolate internal components, such as authentication services and data processing units, from untrusted external entities. The information is routed to a persistent Data Store. This decomposition into discrete processes and flows enables a systematic security analysis using the STRIDE methodology to identify potential vulnerabilities at each interaction point.

Figure 2.2 summarizes the core components commonly used in Data Flow Diagrams and their role in threat modeling.

2.4.2 Modular Monolith

A modular monolith is a software architecture pattern that strategically combines the simplicity of a traditional monolithic structure with the benefits of microservices. Unlike a standard monolith where all components are tightly coupled, a modular monolith organizes the system into loosely coupled modules with well-defined boundaries and explicit dependencies [29]. The choice of a modular monolith is central to achieving a structured code base for easy

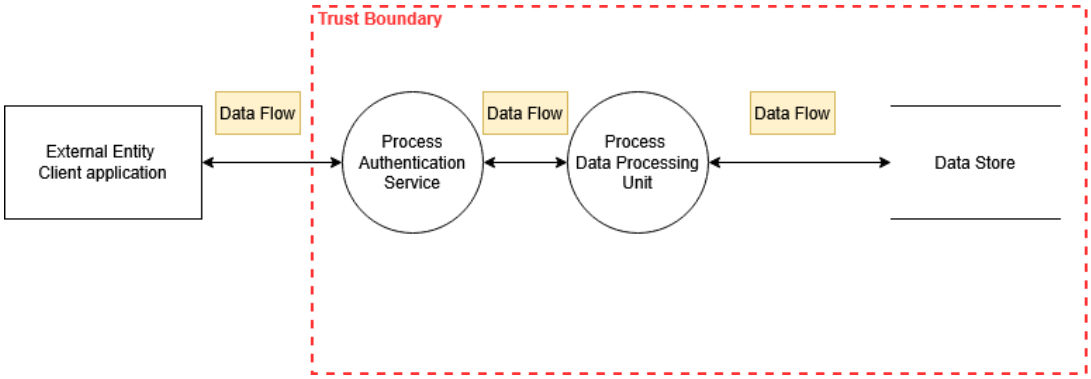


Figure 2.1: Data Flow Diagram with trust boundary

Element	Appearance	Meaning	Examples
Process		Active component that processes data.	Code written in C, C#, Python, PHP
Data Flow		Communication of data between processes, data stores, or external entities	Network connections, HTTP, RPC
Data Store		Component where data is stored persistently.	Files, databases
External Entity		Actor or system outside the modeled system	Customers

Figure 2.2: DFD icon definitions

maintenance and further development in an open-source setting. Figure 2.3 illustrates the modular monolith design pattern.

2.4.3 Structured Query Language

Structured Query Language (SQL) is a domain-specific language used to define, manage, and query data in relational database management systems (RDBMS). In the context of this thesis, SQL is relevant because the proposed tool relies on structured storage of assets. In this project, SQL serves as the the primary interface for interacting with the system's central data store, namely the MariaDB server and constitutes a foundational component of the system architecture

2.4.4 MariaDB and SQL

MariaDB server is an open-source RDBMS that provides data storage and access through a SQL interface. Within the system architecture, MariaDB functions as the central data store, supporting the persistence and retrieval of application data. The choice to retain MariaDB as the backend database is motivated by its proven compatibility with the existing technical

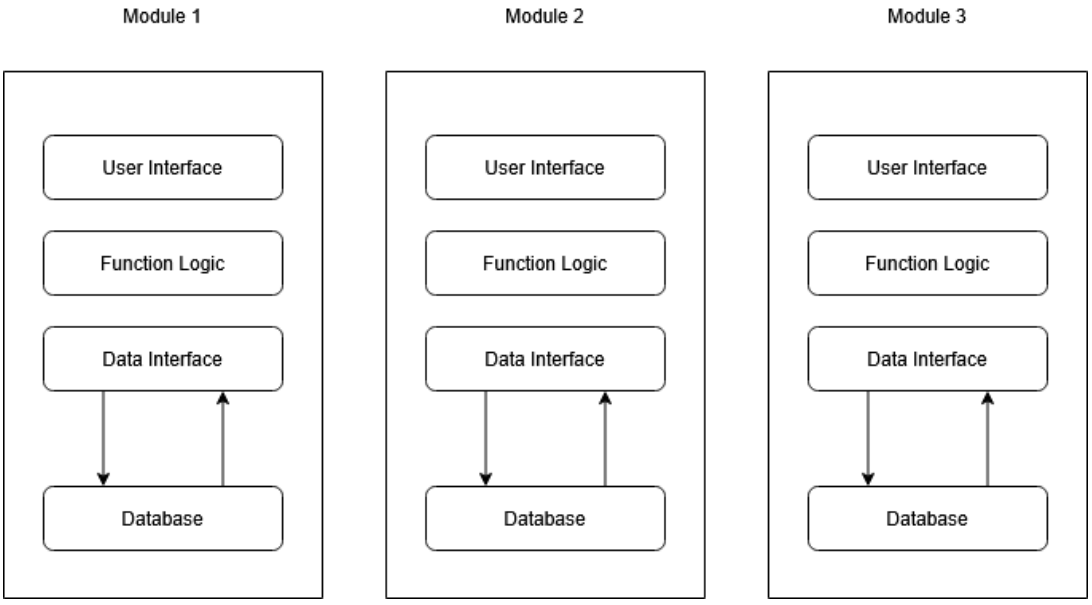


Figure 2.3: Figure of the Modular Monolith architectural pattern

infrastructure, as well as its established use in functional demonstrations in prior work. In the context of this thesis, MariaDB plays a critical role in ensuring continuity and consistency, thereby enabling the developed TARA tool to remain compatible with existing energy community models. SQL is a domain-specific language used to define, manage, and query data in RDBMS. In the context of this thesis, SQL is relevant because the proposed tool relies on structured storage of assets.

2.5 Risk Matrices and Threshold-based Risk Classification

Risk matrices are commonly used tools in risk assessment to support the evaluation and prioritization of hazardous situations. A risk matrix most often combines two dimensions of risk. The purpose of the matrix is to provide a structured and transparent method for estimating risk levels in different operational situations or tasks. This in turn enables consistent risk assessment and decision-making regarding the need for countermeasures, where situations classified as high risk typically require immediate mitigation, while lower risk levels may be considered acceptable or require less extensive control measures. The example shown in Table 2.3 is based on guidance from ISO/TR 14121-2 [30] and illustrates how combinations of probability and severity lead to different qualitative risk classifications.

Table 2.3: Risk estimation matrix [30].

Probability of occurrence of harm	**Severity of harm**			
	Catastrophic	**Serious**	**Moderate**	**Minor**
Very likely	**High**	**High**	**High**	Medium
Likely	**High**	**High**	Medium	Low
Unlikely	Medium	Medium	Low	Negligible
Remote	Low	Low	Negligible	Negligible

While the matrix in Table 2.3 represents a purely qualitative classification scheme, many methodologies extend this structure by assigning a numerical score interval to each param-

eter category. Such mappings allow qualitative assessments (e.g., minor, moderate, serious, catastrophic) to be associated with defined numerical ranges.

Table 2.4: Illustrative example of threshold-based risk classification.

Risk class	**Risk score interval (RS)**
High	$RS \geqslant 8$
Medium	$8 > RS \geqslant 5$
Low	$5 > RS \geqslant 3$
Negligible	$3 > RS \geqslant 0$

The example shown in Table 2.4 illustrates how qualitative classifications can be transformed into numerical values for structured aggregation. This numerical result does not represent a exact risk measurement but rather a constructed index. This is important for the proposed tool, since the architecture must support configurable parameters, aggregation rules, and threshold-based classification schemes.

2.6 SAHARA

SAHARA (Security-Aware Hazard Analysis and Risk Assessment) is a method that integrates th STRIDE threat modeling approach with the automotive Hazard Analysis and Risk Assessment (HARA) process defined in ISO 26262 [31]. The purpose of SAHARA is to systematically identify security threats that may have an impact on functional safety and to incorporate them into the safety analysis process. Although SAHARA was developed in an automotive context, it is relevant to this thesis because it provides an example of how cybersecurity threats can be connected to safety-oriented hazard analysis. The SAHARA method consists of two main phases.

Phase 1: Security Threat Identification and Classification

In this first phase, the security threats are identified using the STRIDE methodology, seen in Section 2.2.4. Each identified threat is then quantified using three parameters:

- R – representing required resources.
- K – representing required know-how.
- T – representing the criticality of the security threat.

Instead of estimating probabilistic likelihood, the SAHARA approach assesses the attacker's capability through the parameters R and K. The parameter T then represents the potential damage and impact of the threat. A threat classified with $T = 3$ indicates high security relevance and possible safety impact. The combination of R, K, and T results in the determination of a Security Level (SecL). The SecL is derived using a matrix-based classification scheme analogous to the Automotive Safety Integrity Level (ASIL) determination in HARA. The SecL serves as a prioritization measure for security mitigation efforts [31].

In the SAHARA method, lower values of R and K indicate attacks that require less resources and expertise to perform, while higher values indicate more difficult attacks. Consequently, lower R and K values contribute to higher Security Levels (SecL).

Phase 2: Integration into HARA

In the second phase, only threats classified with the highest criticality level ($T = 3$) are considered safety-relevant and are transferred into the HARA process. These security-induced scenarios are treated as reasonably foreseeable misuse.

Table 2.5: Security Level (SecL) determination according to the SAHARA method .[31]

Condition	**Resulting SecL**
$T = 0$	0
$5 - R - K + T \geqslant 7$	4
$5 - R - K + T = 6$	3
$5 - R - K + T = 5$	2
$5 - R - K + T = 4$	1

The resulting hazardous situations are then evaluated using the standard HARA parameters: Severity (S), Exposure (E), and Controllability (C). Through this evaluation, an Automotive Safety Integrity Level (ASIL) is determined, and corresponding safety goals are defined.

By integrating security threat analysis into the safety assessment, SAHARA enables the identification of additional hazardous situations that would not be captured by a traditional HARA approach alone. Thus, the method provides a structured mechanism to harmonize security and safety analysis during early system development. An overview of the SAHARA approach can be seen in Figure 2.4.

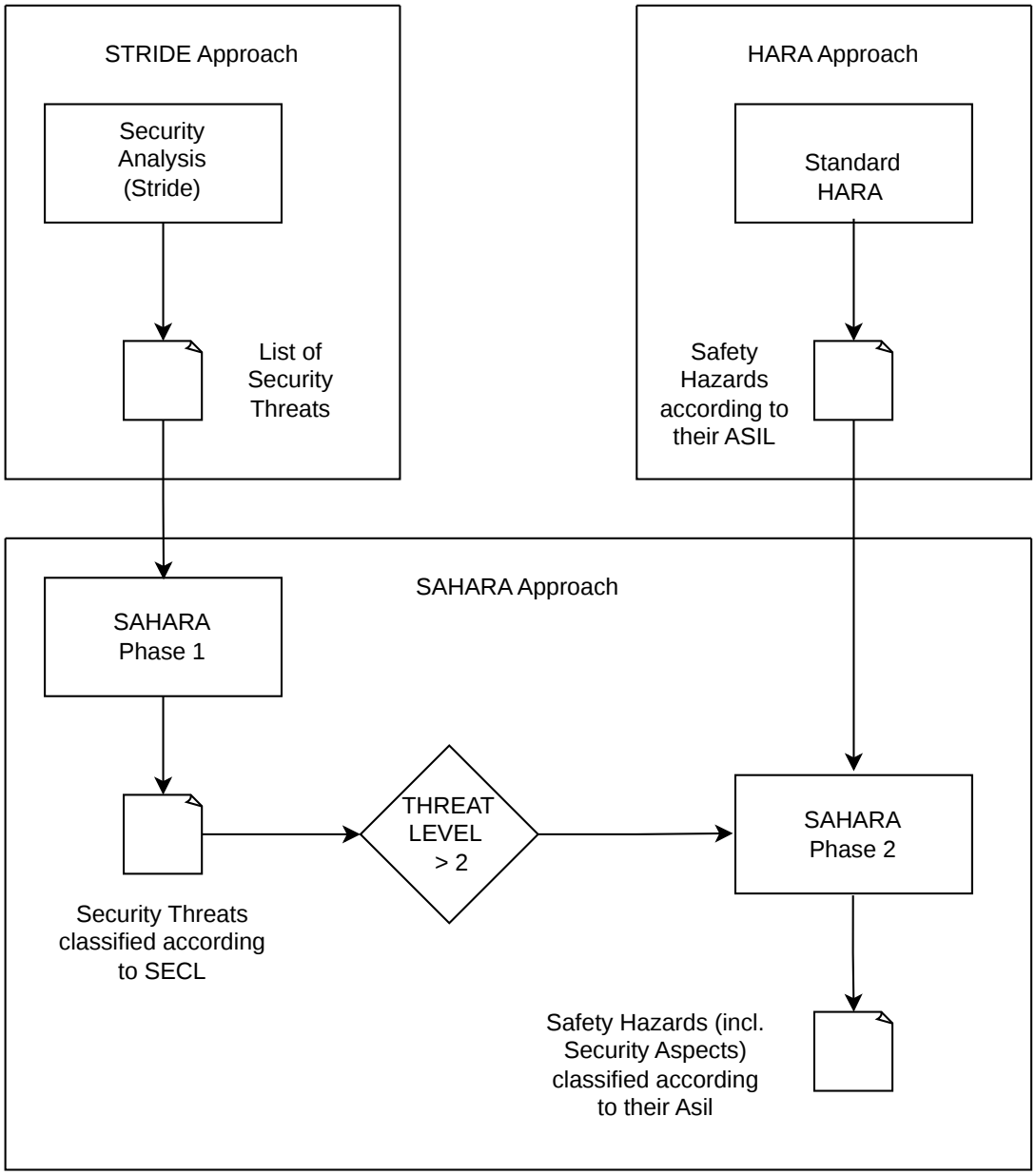


Figure 2.4: Conceptual overview of the SAHARA method

2.7 HEAVENS

HEAVENS is a Threat Analysis and Risk Assessment (TARA) framework developed for automotive embedded systems. Its purpose is to systematically identify cybersecurity threats, assess their risks and then derive high-level security requirements for vehicle systems [1]. Modern vehicles are becoming increasingly more connected, automated and software-intensive, which will in turn increase their attack surface. As a result, cybersecurity threats can no longer be considered as isolated information technology issues, but may also influence critical vehicle functions. Within the automotive domain, functional safety engineering is supported by comprehensive standards such as ISO 26262 [1]. The first security objective of the HEAVENS project is to ensure the safety of vehicle occupants and other road users to prevent both denial-of-use scenarios and unauthorized vehicle modifications that may compromise safety. The second objective is to prevent financial impact by for example fraudulent transactions

and/or theft [32]. The third objective is to keep the operational performance as intended by disallowing outside disturbances to impact performance and protecting internal functions. The last objective is to ensure the fulfillment of legislation and to ensure privacy of users and other parties [32].

The workflow of the HEAVENS framework consists of four main phases: Define System under Evaluation, Threat Analysis, Risk Assessment, and Identification of security requirements. These phases can be seen in Figure 2.5. In the first phase, the system is defined in the appropriate setting. This means that a system can be as large as a whole architecture and some small software dependent on several components. The next phase is threat analysis, which is in turn divided into identification of assets and identification of threats. During asset identification, the objective is to identify system assets and their associated security properties with respect to confidentiality, integrity, and availability. These assets subsequently serve as input to the threat analysis and risk assessment activities. Following asset identification, potential threats affecting the identified assets are analyzed and documented. The pairs of assets and threats are then used as an input for the third phase, risk assessment. The main goal of the risk assessment phase is to use the pairs given from the previous phase to determine security levels (SL), which is an evaluation of what level of security each asset needs. The SL for each component is derived by three main activities; estimating the threat level (TL) and estimating the impact level (IL) and combining them. The threat level represents an assessment of the probability that a specific threat will be realized and the IL, the level of potential damage to the invested parties if the threat is realized. Finally, the last phase is to identify security requirements using the security levels. These identifications happen at a high level, leaving the detailed technical solutions for later [1].

HEAVENS is particularly relevant to this thesis because its separation of threat level, impact level, and final security level provides a clear example of hierarchical risk aggregation. This makes the framework useful for evaluating whether the proposed method-agnostic tool can represent method-specific scoring logic through configurable parameters, expressions, and aggregation structures.

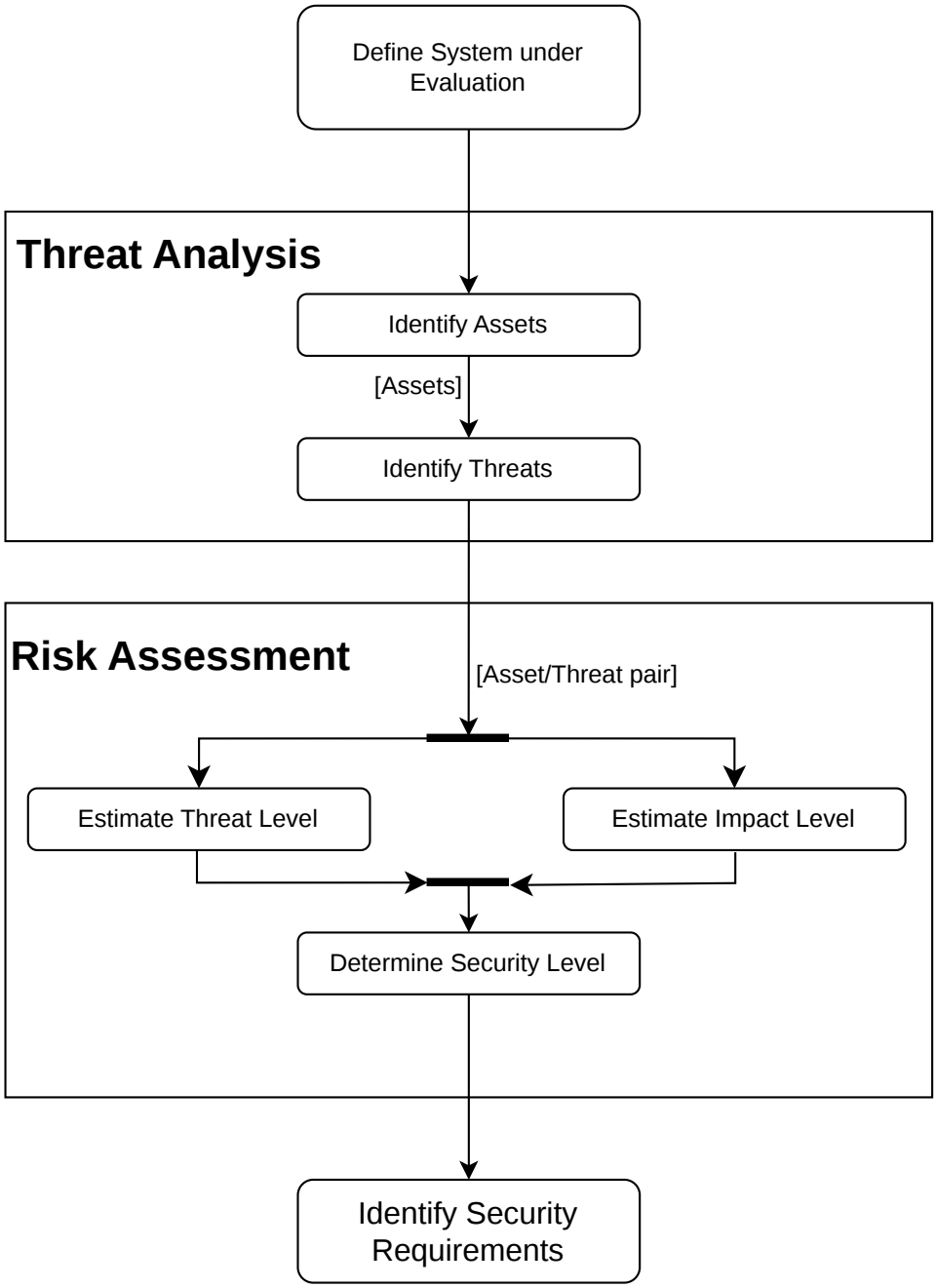


Figure 2.5: The phases and workflow when using the HEAVENS framework [1]

2.8 Large Language Models

Large language models (LLMs) are neural network-based models that are trained on a large amount of text to predict and generate natural language. These models typically learn to model the probability distribution over sequences of tokens, enabling them to then perform tasks such as text generation, question answering and summarization [33].

Many modern LLMs are based on transformer architectures, which rely on attention mechanisms to capture dependencies between elements in a sequence [34].

In the context of threat modeling and risk assessment, LLMs could be used as decision-support tools to suggest potential threats, generate risk scenarios, and support structured analysis workflows. However, their outputs require human review because they may be incomplete, incorrect, or unsupported by the provided system context.

2.8.1 Examples of Modern LLMs

In recent years, several large-scale LLMs have been developed by both research organizations and industry. Examples include models such as GPT (Generative Pre-trained Transformer) developed by OpenAI, Claude developed by Anthropic, and Gemini developed by Google. These models are often trained on large sets of data and fine-tuned for specific applications that include conversational agents, code generation, and decision support systems. While many modern LLMs share transformer-based foundations, they may differ in training, model scale and alignment techniques.

2.8.2 Prompt engineering

Prompt engineering is the practice of structuring and designing input prompts to guide the behavior of pre-trained language models, and then adapt them to specific tasks [35]. By crafting prompts carefully, it is then possible to influence how the model interpret instructions and generate outputs without modifying the underlying model. In the context of this thesis, prompt engineering is important because the LLM component relies on structured prompts to support threat identification and risk analysis.

Since the proposed LLM component is intended to assist with structured threat identification and risk analysis, prompt engineering becomes important for guiding the model toward context-relevant, structured, and reproducible outputs.

2.8.3 Human-in-the-loop

Human-in-the-loop refers to approaches where human expertise is integrated into AI systems to improve the transparency, reliability, and performance. Instead of relying on fully automated decision making, human-in-the-loop involves human interaction at key stages of the systems process to review, validate, modify, or reject AI-generated outputs before they are acted upon. In the context of this thesis, human-in-the-loop is important because LLM-generated risk suggestions may be incomplete, incorrect, or insufficiently grounded in the system context. Enabling interaction between human experts and the model, through iterative question answering or feedback loops, can be used to refine the accuracy. Furthermore, human oversight supports accountability [36].

3 Related Work

This chapter reviews existing research, highlighting it's importance and relevance to the objectives of this thesis.

3.1 Literature Search Strategy

A literature search was conducted to identify research relevant to threat modeling, safety, and cybersecurity risk assessment methods, tools for HARA/TARA processes, and LLM-assisted threat identification. The primary databases used were Google Scholar and the Linköping University library search engine.

The search process used combinations of keywords including "threat model", "model-agnostic", "TARA", "HARA", "risk assessment", and "LLMs". Searches was primarily limited to peer-reviewed journal articles, conference papers, and academic publications published in English. Preference was given to publications published within the last 10 years, although older foundational works were also included when considered relevant.

Search results were first screened based on title and abstract. Publications were included if they addressed threat modeling methodologies, risk assessment frameworks, HARA/TARA-related processes, or applications of LLMS in cybersecurity and safety analysis. Publications were excluded if they were not directly related to the research topic, lacked sufficient technical detail, or focused on unrelated application domains.

The selected literature was subsequently analyzed and used to establish the theoretical background and motivation for this thesis.

3.2 Previous Work on the Tool

This thesis is a continuation of a previous bachelor thesis by Gustafsson and Hagel [37], conducted in collaboration with RISE. Gustafsson and Hagel developed an initial proof-of-concept HARA/TARA risk assessment tool aimed at supporting both cybersecurity and functional safety analysis within a unified framework. The artifact they developed demonstrates that structurally different risk assessment methods could be represented through configurable dimensions, parameters, formulas, and evaluation sheets. However, the implementation was limited in terms of scalability, interoperability, and collaborative support. Building

on the findings made in Gustafsson and Hagel's work, this thesis extends the original work through several changes and additions to the database, a redesigned web-based architecture, support for method-agnostic risk representation, improved concurrent multi-user support and exploratory LLM-assisted risk identification functionality.

3.3 HEAVENS-Based Risk Assessment

Islam et al. [1] propose a risk assessment framework suited for automotive systems called HEAVENS. The authors emphasize the need to integrate cybersecurity with functional safety processes. The approach used defines security levels by evaluating both the likelihood of a threat and its potential impact, resulting in a comprehensive framework for risk assessment of integrated software in automotive systems and in alignment with ISO 26262 [38]. The HEAVENS framework provides a structured approach for deriving security requirements during the development process.

Sandberg et al. [39] focus on tailoring and improving HEAVENS to increase the time efficiency of the method. The authors consider some parts of the HEAVENS methodology to be time-consuming. They therefore propose to make risk assessment faster by clarifying unclear parameters and breaking complex parameters into sub-parameters. The parameter "window of opportunity" is divided into "accessibility means" and "asset exposure time". The parameter financial impact is divided into several sub-parameters (e.g. safety impact, operational impact, number of affected vehicles, cost of correction, reputation loss) which are later on combined using a financial model [39].

Lautenbach et al. [40] analyze the gap between the HEAVENS method and the ISO/SAE 21434 standard. The authors propose a set of methodological updates to close this gap. They describe the gap-related problems as learnability, model customization, process efficiency and accuracy. The proposed changes include making parameter scales more intuitive, normalizing threat and impact calculations to support customization, and introducing clearer analysis steps such as damage scenarios, attack path analysis, and refined definitions of ambiguous parameters. Overall, the proposed changes aim to improve consistency, efficiency, and practical usability while maintaining alignment with ISO/SAE 21434.

These studies are relevant to this thesis because they show that even established methods require adaptation, clarification, and customization in practice. The HEAVENS framework provides a foundation for the developed method-agnostic HARA/TARA tool. HEAVENS, together with other frameworks, was analyzed to identify common processes and components that could be abstracted into reusable building blocks.

3.4 SAHARA-based Security Analysis

The SAHARA approach was first introduced by Macher et al. [2] as a method for integrating security threat analysis into the HARA process. In their case study, the authors reported that the application of SAHARA identified approximately 34% more hazardous situations compared to a conventional HARA analysis alone. This demonstrates that cybersecurity threats may introduce safety-relevant scenarios that would otherwise remain undetected in a conventional HARA analysis. SAHARA is therefore relevant to this thesis as it is used to validate the method-agnostic design of the proposed tool. The framework supports both SecL-based security classification and ASIL-based safety assessment within a unified analysis process.

3.5 Adapted SAHARA To Distributed Grids

In a paper by Thorsén et al. [41] an adapted version of the SAHARA framework was used to do risk assessment for intelligent distributed grids. In the paper, authors explain that the original SAHARA framework uses the ISO 26262 standard for its definition of functional

safety. In the adapted version of SAHARA, Thorsén et al. use the quantitative risk graph method to better comply with the general functional safety definitions in IEC 61508 [41]. The quantitative risk graph method differs from SAHARA's safety risk assessment method in that the focus is on ensuring safety for people instead of focusing on the protection for the equipment [41]. This method, not unlike the SAHARA method, also quantifies the risks into four different categories, as seen in Table 3.1. When this is done for each component a risk class (CI) is calculated [41]. Finally, all threats with a CI of 2 or above are analyzed, which results in several safety goals based on security considerations that are added to the main safety goals defined in the second part of SAHARA.

This work supports the motivation for a method-agnostic tool because it demonstrates that established safety-security methods often need to be modified when transferred to a new domain. The work also shows an example of how this adoption could be done and what functionality the tool needs to support this adoption.

Table 3.1: Risk Assessment Parameters and Scoring Criteria [41]

Parameter	**Description**	**Score Range**
Severity of Harm (Se)	An evaluation of how serious the potential injury or health damage could be, including the overall impact on a person's well-being.	1 (minor harm) – 4 (severe harm)
Frequency of Exposure (Fr)	An assessment of how often and for how long individuals are exposed to the hazard. All operating modes of the machinery and work practices are considered, including the potential for long-term health effects.	2 (rare exposure) – 5 (very frequent exposure)
Probability of Occurrence (Pr)	An estimate of the likelihood that a hazardous event will occur and cause harm to people, property, or the environment. Considerations include accessibility of the hazard zone, duration and frequency of access, and the number of individuals exposed.	1 (very unlikely) – 5 (very likely)
Avoidability of Harm (Av)	An evaluation of how feasible it is to prevent or reduce harm once a hazardous situation arises. Factors include the skill level of exposed persons, speed of hazard development, risk awareness, and human capability to react based on experience and knowledge.	1 (avoidable), 3 (partially avoidable), 5 (unavoidable)

3.6 Threat Tool Evaluations

Existing threat modeling tools provide useful support for structured threat identification, but their design choices influence usability, extensibility, collaboration, and methodological flexibility. Therefore, tool evaluations are relevant to this thesis as they reveal limitations that motivate a method-agnostic and collaborative TARA/HARA tool.

Flå et al. [42] describe in their paper an extension of the Microsoft Threat Modeling Tool for smart grid environments. The authors introduce a dedicated smart grid template that is based on STRIDE and DFDs, enabling automated generation of cyber-threats in both IT and OT components. The tool the authors developed demonstrates that traditional, tool-assisted threat modeling can be applied to complex cyber-physical energy systems . However, the approach proposed by the authors is closely aligned with a specific threat modeling methodology and toolchain.

Bygdås et al. [43] compare the Microsoft Threat Modeling Tool and OWASP Threat Dragon Modeling Tool. The authors' study analyzes differences in usability, threat generation capabilities, reporting support, and integration into the software development life cycle. The authors also highlight a trade-off between the complex nature of the Microsoft Threat Modeling

Tool and the more user-friendly but less comprehensive Threat Dragon. Furthermore, a more comprehensive tool supports increased modeling detail, but at the cost of higher complexity and learning effort . The authors conclude that none of the tools are perfect. Both tools rely on model-specific threat identification approaches such as STRIDE, the CIA triad, and LINDDUN, and primarily support static report-oriented threat modeling workflows [43].

Hwang et al. [6] evaluate five threat modeling tools (SPARTA, MS Threat Modeling Tool, PyTM, IriusRisk, OWASP Threat Dragon) focused on usability. The authors used a Goal-Question-Metric (GQM) approach based on ISO 9241-11. They derived several usability criteria and applied them to evaluate the five threat modeling tools. Their results show that IriusRisk achieves the highest overall usability mainly due to strong software development life cycle (SDLC) integration and collaboration support. This work is valuable for our study, especially RQ1, as it highlights usability and workflow limitations in existing, method-specific tools, supporting our motivation for a method-agnostic TARA framework that emphasizes flexibility, collaboration and usability across different threat modeling approaches.

These evaluations provide important insights into the limitations and design trade-offs of existing threat modeling tools. From these evaluations we find that most current threat modeling tools do not support model-agnostic risk assessment or integration of safety-oriented analyses such as HARA.

3.7 Attack Tree Risk Assessment

Gao et al. [44] propose a risk assessment approach for SCADA systems based on Attack Countermeasure Trees (ACT), an extension of traditional attack trees that explicitly models both attacks and defensive countermeasures. Risks are evaluated by combining attack likelihood, defensive capability, and impact assessed through the CIA triad. This work is particularly relevant as it addresses cybersecurity risk assessment in critical energy infrastructure.

Similarly, Hoseini et al. [45] use attack trees to systematically model and prioritize security threats in AI/ML systems, focusing on violations of confidentiality, integrity, availability, and privacy.

While both studies demonstrate the effectiveness of attack-tree-based approaches for domain-specific risk assessment, they are built around particular methodologies. In contrast, our work aims to provide a method-agnostic TARA framework that supports the application and combination of multiple threat and risk assessment methods within a unified tool.

3.8 LLM Support for Threat Modeling

LLMs have been explored for three main purposes in threat modeling. Fregnan et al. [46] present a chat-based LLM assistant designed to support both cybersecurity experts and non-experts. The authors propose a system that uses a Retrieval-Augmented Generation (RAG) architecture to then retrieve relevant information from the threat modeling documentation and generate responses to user queries. The authors evaluate the system using several approaches: traditional natural language processing (NLP) metrics, expert-based human evaluations, and automated LLM evaluation methods. Their results show that an LLM-based chat bot can provide accurate responses. The authors, however, highlight challenges related to AI hallucinations.

Von der Assen et al. [47] propose ThreatFinderAI, an automated threat modeling framework designed for AI-based systems. The authors' approach follows a more asset-centric methodology and leverages knowledge from OWASP AI, ENISA, and MITRE to automatically identify threats. The framework is demonstrated in a scenario involving the integration of a Large Language Model in a customer service application.

Kaviyil et al. [48] investigate how ChatGPT can support the threat modeling process. In their study, they made a system model of a cloud-based microservice application hosted on

AWS and used input for ChatGPT to generate potential threats and mitigation measures. The results they presented demonstrated that ChatGPT can suggest threats and security protocols efficiently. However, the authors also emphasize that human expertise is still required for validation.

Some research has also explored the use of LLMs to give support to threat modeling automatically. Hossain [49] proposes an AI-driven system that analyzes user stories within the SDLC to then generate STRIDE- based threats and corresponding security objectives and tasks. This work demonstrates that LLMs are capable of producing structured and comprehensive reviews. However, this proposed approach is focused on cybersecurity and relies on user stories as the main input, limiting the applicability in broader risk assessment contexts.

In an article by Elsharef et al. [50], the authors also explored the use of LLMs to support automated threat modeling. They propose an approach that is LLM-based, that aims to reduce the amount of manual labor. In their system, the authors combine natural language processing methods with an LLM to extract relevant information from design documents and vulnerability data. The LLM is then used to answer key threat modeling questions such as identifying potential security threats in the system. In their results, they show that LLMs can assist effectively in the identification of relevant information and support the threat modeling process. They conclude in their article that over 75% of the responses meet the human evaluation expectations. However, the approach the authors have made focuses on assisting analysts through question answering rather than generating or refining risk assessments.

Recent research has explored the use of LLMs to support and automate parts of the threat modeling process. Several studies demonstrate that LLMs can assist in identifying threats, generating mitigation suggestions, and analyzing system descriptions or design documentation. Approaches such as those proposed by Fregnan et al. [46], Von der Assen et al. [47], Kaviyil et al. [48], Hossain [49], and Elsharef et al. [50] show that LLMs can improve efficiency and reduce manual effort in threat analysis workflows. At the same time, multiple authors emphasize limitations related to hallucinations, reliability, and the continued need for human expert validation. In several of these approaches, the LLM takes a primary role in generating threats or performing automated analysis. In contrast, this thesis investigates how LLMs can be integrated into a method-agnostic HARA/TARA tool as a supporting component within a structured risk assessment workflow. Rather than replacing the analyst, the LLM is used to analyze system descriptions together with existing human-created risk assessments in order to identify potentially overlooked risks within a selected methodological framework. The generated risks are therefore treated as suggestions that require human review and validation before inclusion in the assessment process. This approach aims to support a more iterative and context-aware risk analysis process while maintaining human oversight and reducing the impact of unreliable or hallucinated AI-generated results. Additionally, while much of the existing work utilizes OpenAI's ChatGPT models, the proposed artifact implementation instead integrates Anthropic's Claude model. This choice allows for evaluation of similar functionality using an alternative LLM provider, contributing to a broader understanding of how different LLM systems can support risk analysis tasks.

3.8.1 Structured LLM prompting

In recent work by Kongurai [51], the application of LLMs for domain specific contract analysis in engineering contexts is explored. A central contribution of the study is the use of structured prompting techniques, where prompts are carefully designed to guide the model through well-defined analytical tasks rather than relying on more generic instructions. Specifically, the authors decompose the task into several sub-tasks such as extracting relevant technical information and evaluating compliance against predefined requirements, while also constraining the expected output format. This approach embeds domain knowledge and reasoning criteria directly into the prompt, improving both consistency and interpretability of the results.

This structured prompting paradigm is highly relevant to the development of the proposed method agnostic HARA/TARA tool in this thesis. In the proposed system, LLMs are integrated to process system descriptions alongside manually identified risks, with the goal of suggesting additional hazards, threats, or risks. Inspired by Kongurai's [51] methodology, the AI functionality is designed around structured prompts that explicitly define the task context, system description and existing risk, the objective, risk identification, and the expected output format. By framing the problem as a guided, multi-step reasoning task rather than an open generation problem, the approach aims to align the LLM's behavior with established safety analysis workflows. This not only improves the relevance of generated risks but also supports traceability and consistency, which are critical in safety critical domains using HARA and TARA. Consequently, the incorporation of structured prompting serves as a key design principle for enabling reliable AI-assisted risk analysis within the tool.

3.9 Summary of Gaps

The reviewed literature showed that several risk assessment methods exist, but because they are applied to different use cases and domain-specific systems, they often need to be adapted to some extent. These modifications and changes are necessary to adopt the methods effectively and allow the methods to achieve their intended purpose of identifying and evaluating risk. Existing tool evaluations show that existing tools are often closely coupled to specific methods or workflows constraining the ability to adapt the risk assessment method in specific domains. This limits the flexibility and usefulness of the tools in cases where the ability to adapt the methods is crucial to get meaningful evaluations of risk. LLM studies demonstrate that LLMs can be very effective and reduce the amount of manual labor necessary to identify and evaluate risk. However, there is a consensus across several different independent papers that human oversight is critical when conducting LLM analysis of risk. Based on these observations, a gap remains for a method-agnostic software tool that can represent multiple safety and cybersecurity risk assessment methods, support collaboration and traceability, and explore LLM-assisted risk identification as a human-supervised support feature. This thesis addresses this gap by designing and evaluating a prototype architecture for a method-agnostic HARA/TARA tool.

4 Research Methodology and Design Analysis

This chapter first introduces Design Science Research as the overarching methodological framework. It then describes the analytical process used to derive architectural requirements, the methods used to evaluate collaboration and interoperability support, and the procedure used to explore LLM-based support for threat and risk identification.

4.1 Design Science Research Approach

This thesis adopts Design Science Research (DSR), outlined by Hevner et al. [52] as the main methodology. DSR is particularly suitable for research problems that require the design and evaluation of innovative artifacts intended to solve identified practical challenges.

Within this paradigm, knowledge is generated through the iterative creation and assessment of an artifact in a relevant application context. In accordance with established DSR guidelines, the research follows a build-and-evaluate cycle, where the artifact serves both as the means of intervention and as the primary vehicle for knowledge generation. Rather than relying solely on observational analysis, insights are derived from the design decisions, implementation process, and systematic evaluation of the developed solution. The artifact developed in this thesis consists of a prototype tool designed to support method-agnostic HARA and TARA. The research process is structured into two iterative phases, each addressing specific aspects of the research problem and corresponding research questions. The presentation of these two iterations follows.

4.1.1 Iteration 1: Method-Agnostic Architectural Design

In this thesis, the DSR process is operationalized as follows. The practical problem is the limited flexibility of existing threat analysis and risk assessment tools when applied to evolving energy community contexts. The objective of the solution is to design a method-agnostic artifact capable of representing selected safety and cybersecurity risk assessment methods. The artifact is developed as a proof-of-concept prototype and evaluated through technical tests, framework-based comparison, and expert feedback. The first iteration addresses RQ1 and parts of RQ2 by focusing on the design and development of a method-agnostic system architecture. It consists of:

- A structured literature review of established threat and risk assessment methodologies, including STRIDE, SAHARA, HEAVENS.
- An analytical abstraction process to identify common structural elements across these methodologies.
- The design and implementation of a modular, method-agnostic architecture.
- The development of a functional proof-of-concept prototype.

RQ1 is addressed by abstracting and representing a TARA framework within a unified architectural model. RQ2 is addressed by designing the system architecture based on the requirements derived from the literature review and analysis of the existing HARA/TARA methodologies. These requirements emphasize the need to support collaboration, interoperability, and concurrent assessment between different stakeholders and methods.

The evaluation in this first iteration is primarily technical and analytical. It includes architectural validation, feasibility analysis, and a assessment of whether existing methodologies such as SAHARA and HEAVENS can be recreated within the proposed structure.

4.1.2 Iteration 2: Support features and Evaluation of LLM Integration Opportunities

The focus of the second iteration, including supporting features, was identified based on expert feedback collected at the end of the first iteration, which is explored in Section 6.3. This iteration addresses RQ2 and RQ3 by investigating opportunities for additional support functionality and analytically exploring the potential integration of LLMs into the proposed artifact. First an literature review on LLM-assisted risk modeling was conducted, where the focus was on identifying existing approaches and best practices for implementing an LLM functionality. The reviews examined:

- Current applications of LLMs in threat modeling.
- Structured prompting techniques for domain-specific tasks.
- Human-in-the-loop validation mechanisms.
- Challenges related to AI hallucinations and reliability in safety-critical domains.

These findings then informed the design requirements for an LLM integration within the proposed tool.

4.1.3 Mapping of Reaserch Questions to Iterations

The research design ensures that each RQ is systematically addressed within the DSR process, as follows:

1. RQ1 is addressed through architectural abstraction and prototype implementation.
2. RQ2 is addressed through system design choices enabling collaboration and interoperability.
3. RQ3 is addressed through qualitative expert evaluation of the proposed tool, including feedback on the potential and feasibility of LLM integration, as well as testing the capability of LLMs to support existing risk assessments.

Together, these two iterations provide both a validated technical artifact and a grounded analytical assessment of future AI-supported extensions.

4.2 Method for RQ1: Method-Agnostic Representation

This section describes the methodology used to answer RQ1. The objective was to identify common concepts and structures across selected HARA/TARA frameworks and abstract them into a method-agnostic representation. The analysis was conducted through a series of steps, including framework selection, comparison of structural elements and workflows, and the derivation of architectural requirements. The following subsections describe each step of the analysis process.

The methodology for RQ1 consists of a sequence of analysis steps that progressively abstract common concepts from existing HARA/TARA frameworks. Figure 4.1 provides an overview of the process. The following subsections describe each step in detail.

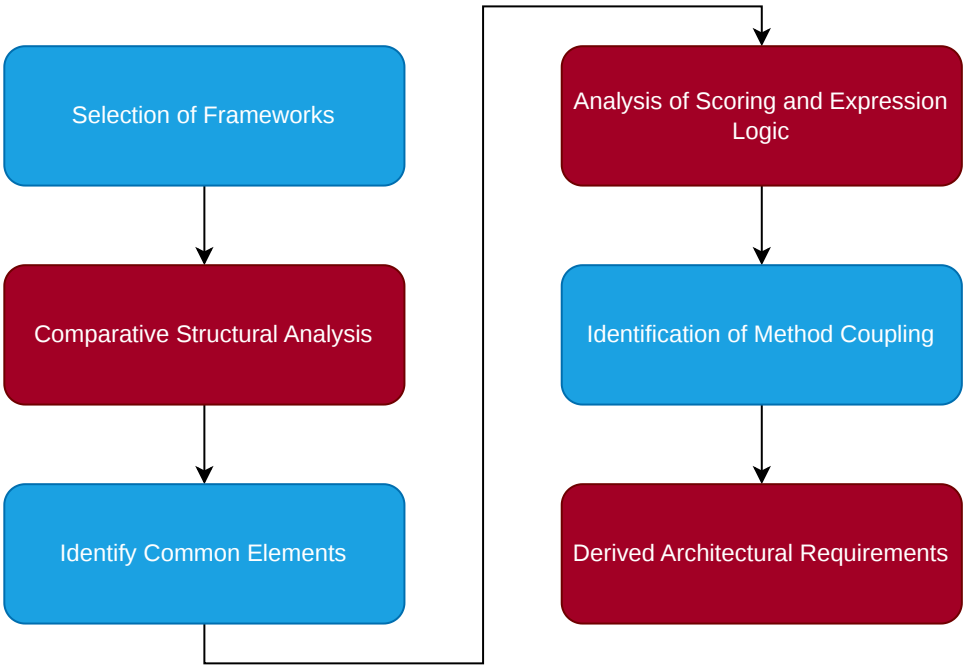


Figure 4.1: Overview of the methodology used to derive the proposed method-agnostic architecture.

4.2.1 Selection of Frameworks

Although the objective of this thesis is to design a method-agnostic TARA/HARA architecture capable of representing multiple risk assessment methodologies, a bounded set of representative frameworks was selected for structural analysis and abstraction. The purpose of this selection was not to re-implement specific standards, but to extract generalizable modeling principles applicable across heterogeneous safety and security assessment approaches.

HEAVENS and SAHARA were selected as the primary methods used to evaluate the tool due to their structural diversity and explicit integration of threat identification and risk quantification mechanisms. SAHARA represents a unified safety-security approach in which cybersecurity threats identified using STRIDE are quantified using parameter-based evaluation and subsequently integrated into a functional safety analysis. In contrast, HEAVENS follows a security-centric workflow where threat level and impact level parameters are independently assessed and combined to derive security levels.

The selection was further informed by the overview of existing threat modeling tools presented in Table 2.2, which indicates that most current tools are tightly coupled to specific methodologies. This methodological coupling limits extensibility and interoperability. Consequently, the analytical focus of this thesis was directed toward identifying deeper structural patterns such as asset representation, parameter hierarchies, aggregation logic, and result classification, that are independent of any particular domain standard.

The chosen frameworks therefore serve as structurally distinct reference models enabling abstraction toward a unified, method-agnostic representation rather than as prescriptive targets for implementation.

4.2.2 Comparative Structural Analysis

To enable abstraction toward a method agnostic architecture, the selected frameworks were decomposed into their fundamental structural components. A comparative analysis was conducted, focusing on identifying the recurring modeling constructs rather than comparing procedural workflows or domain-specific terminology. Each framework was examined with respect to how it represents assets, threats or hazards, evaluation parameters, aggregation logic, and final risk classification outputs.

In SAHARA [2], cybersecurity threats are first identified using STRIDE and subsequently quantified through structured parameters such as *Level of Knowledge*, *Required Resources* and *Threat Criticality*. These parameters are aggregated to derive a *Security Level*, and threats exceeding defined thresholds are propagated into a functional safety assessment stage. In contrast, HEAVENS [40] separates threat identification from risk quantification through independent threat level and impact level parameters, which are later combined to derive a security level classification.

Despite differences in workflow and terminology, all analyzed frameworks do share a parameter driven evaluation structure, where qualitative assessments are transformed into classification outputs. This observation shows that the variation between the frameworks lies not in their building blocks, but in the interpretation and configuration of their evaluation parameters. This finding represents a key insight of this thesis, that the underlying structure of the TARA/HARA methodologies is largely consistent. Whilst the underlying differences arise with the configuration and interpretation of different parameters. This supports the feasibility of designing a unified architecture, where the analytical focus can be shifted from a procedural workflow to a structural element enabling the design of a then unified, method-agnostic representation.

4.2.3 Identify Common Elements

To ensure comparability across frameworks, the procedural workflows of SAHARA and HEAVENS were mapped into generalized assessment phases. These phases include: system or asset identification, threat and/or hazard identification, parameter-based evaluation, aggregation of evaluation results, and derivation of risk classification, and/or safety goals. This mapping confirms that methodological differences primarily affect execution logic rather than structural composition.

An example of how these flows need to be implemented in the tool can be seen in Figure 4.2.

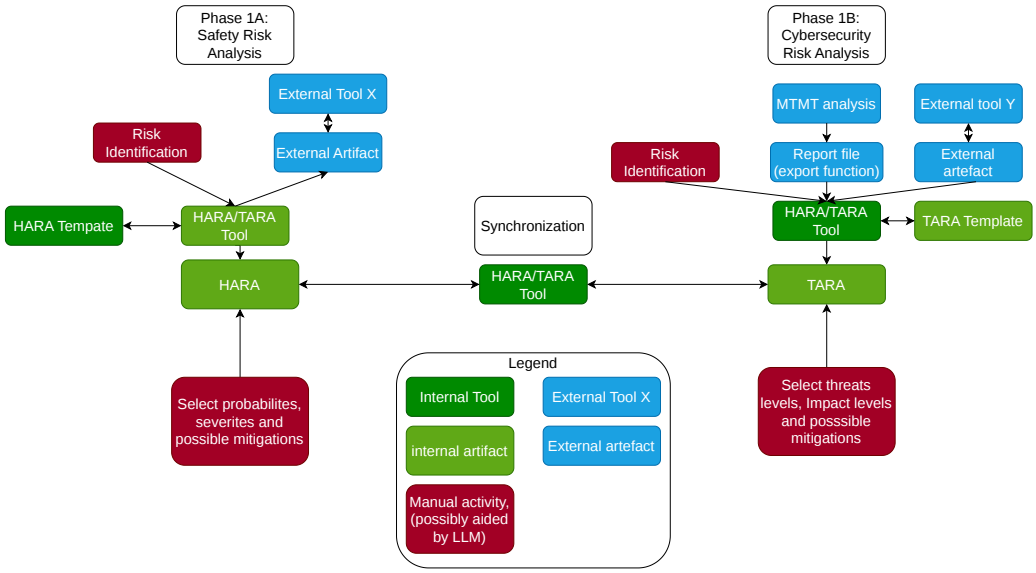


Figure 4.2: Overview of the tool requirements, illustrating the workflow across Phase 1A (Safety Risk Analysis) and Phase 1B (Cybersecurity Risk Analysis).

4.2.4 Analysis of Scoring and Expression Logic

The analyzed frameworks (such as HEAVENS and SAHARA) rely on structured evaluation parameters combined through predefined aggregation mechanisms, such as matrices, weighted calculations, or hierarchical composition. Risk matrices are commonly used to assign and combine different parameters such as impact and likelihood to then derive qualitative risk levels. Weighted calculations assigns different importance to parameters and compute a combined score that is based on predefined weights. Hierarchical composition structures parameters in multiple levels, where the values are aggregated stepwise from lower to higher-level input indicators. Despite methodological variation, all frameworks depend on configurable parameter hierarchies and aggregation rules. This reinforces the feasibility of abstracting scoring logic into a unified, method-agnostic representation.

4.2.5 Identification of Method Coupling

The analysis also examined the degree to which existing workflows and toolchains are structurally coupled to specific methodologies. As identified in the related work presented in Chapter 3, most existing threat modeling tools are designed around a specific methodology. This coupling was found to arise from implementation choices that embed method-specific logic directly into the tool architectures and not from fundamental differences. Many of the tools work in a similar way and pattern, regarding evaluation structure, i.e., how parameters and dimensions are structured to analyses specific risks. This finding, reinforces the case for a configurable and method-agnostic design, where the underlying data model remains independent of any particular assessment standard and the methodology is instead expressed through configuration of *parameters*, *dimensions* and aggregate expressions of *dimension-* and *parameter values*.

Identification of Common Structural Elements The analysis identified a set of common structural elements shared across methodologies. Threat or hazard identification mechanisms differ in terminology but follow a comparable pattern: assets are exposed to potential adverse events categorized according to a predefined taxonomy. Whether through STRIDE

categories or hazard definitions, each framework establishes a structured relationship between assets and potential harmful events.

Asset Representation Across the analyzed frameworks, the assets constitute the primary unit of analysis. Although definitions vary between functional components and logical processes, each methodology requires a structured representation of the system before evaluation. For the proposed architecture, this implies that assets must be represented independently of any specific risk method.

Threat Modeling Structures Although different frameworks use different terminology, they all follow the same basic pattern. First, a system component or asset is identified. Then, potential harmful events affecting that asset are defined. In cybersecurity oriented approaches, these events are often referred to as threats and may then follow taxonomies. In safety oriented approaches they are described as hazards.

Parameter-Based Risk Scoring The frameworks use defined parameters to evaluate risk. These parameters may represent likelihood, severity, required resources, impact, and exposure, depending on the methodology. Each parameter is assigned a value within a predefined scale, in turn allowing qualitative judgments to be translated into structured inputs. Although the specific parameters differ between frameworks, the overall principle is consistent; that risk is evaluated by assigning values to clearly defined factors and then combining them according to specific rules or formulas.

Hierarchical Aggregation Logic In all frameworks, individual parameter values are not used directly as final results. Instead, they are combined step-by-step to produce higher-level evaluations. For example, intermediate values such as threat level or impact level are calculated first and then combined into a final risk classification. This multi-step structure shows that risk assessment is typically hierarchical. Therefore any general architecture must support nested or stepwise formulas/calculations rather than only single level scoring.

Risk Classification and Decision Support The frameworks ultimately produce a final classification used to guide decisions. These may be expressed as security levels, safety integrity levels, or other defined risk categories. Even though naming conventions may vary, the purpose remains the same, i.e., to summarize the evaluated parameters into a discrete level. For the proposed architecture, this means that final assessment outputs will be similar, if not close to identical, which means the architecture only have to consider naming conventions between different risk methodologies.

Considered Design Alternatives Two primary architectural design alternatives were considered. Modular monolith and Microservices. Microservices architecture structures the system as a collection of small and loosely coupled independent services that can communicate through lightweight mechanisms such as APIs [53]. While this approach does enable the scalability and flexibility that is required for this thesis, it also introduces an increase of system complexity due the the distributed deployment. In contrast, modular monolith provides a more structured system with clearly defined modules. In this project, a modular monolith was chosen to reduce the complexity and with the aim of maintaining maintainability and extensibility, which is important if this proof-of-concept tool is to become open source. A modular design also supports future open-source development by enabling contributors to work on individual components with minimal impact on the overall system. In conclusion, the system developed follows a layered architecture, that consists of a frontend, backend and a database layer. Within that backend layer a modular monolith design is applied, where the system is structured into well defined modules with clear responsibilities.

Method-Specific Hard-Coded Architecture This approach involves implementing each risk assessment method directly in code. This would allow close alignment with each standard and straightforward implementation for a limited number of methods. However, this approach would make it difficult to add new methodologies without modifying the core system. It would also reduce flexibility and increase long-term maintenance effort. Finally, this would conflict with the thesis objective of supporting method-agnostic risk assessment.

Plugin-Based Method Modules Another alternative is to create separate modules for each risk assessment method. In this model, each methodology would be implemented independently while sharing a common system model. This improves modularity but still requires method-specific logic to be implemented separately. It may also introduce inconsistencies between modules and may not fully solve the problem of structural coupling.

4.2.6 Derived Architectural Requirements

Based on the comparative analysis and the evaluation of design alternatives, a set of architectural requirements was defined to guide the system design. These requirements reflect the common structural patterns identified across frameworks and ensure that the proposed architecture remains flexible and method-independent. These requirements will also be used during the evaluation to show if the design of the tool meets these.

Req. 1 Support for Hierarchical Calculations.

Req. 2 Configurable and Parameterized Scoring Mechanisms.

Req. 3 Extensibility and Method Flexibility.

Req. 4 Traceability and Transparency.

Req. 1 Support for Hierarchical Calculations

The architecture must support multi-stage calculations. Since all analyzed frameworks combine parameter values in steps to derive higher-level results, the system must allow intermediate results to be calculated and then reused in subsequent evaluations.

The functionality is evaluated by implementing a set of predefined risk assessment scenarios consisting of multiple calculation stages. The system-generated intermediate and final results are compared against manually derived reference calculations.

The requirement is satisfied if (i) the system correctly computes all intermediate and final values for a predefined set of test cases covering at least two different assessment methods and (ii) at least one multi-stage calculation.

Req. 2 Configurable and Parameterized Scoring Mechanisms

Risk parameters, value scales, and calculation rules must be configurable to support different methodologies.

The evaluation is performed by configuring multiple distinct scoring models (e.g., HARA and TARA-inspired configurations) and applying them to the same input dataset.

The requirement is satisfied if: (i) scoring parameters can be modified without code changes, and (ii) the system produces different, method-consistent outputs based on configuration changes.

Req. 3 Extensibility and Method Flexibility

Different stakeholders may rely on different risk assessment methodologies or frameworks. The system should therefore support multiple assessment approaches within the environment, without the enforcement of a single workflow. Furthermore, the architecture must support the addition of new risk assessment methods without requiring a redesign of the core system. This will require flexible parameter structures and configurable aggregation logic, allowing different risk assessment methods to operate on a shared system representation.

The evaluation is conducted by introducing an additional risk assessment method into the system and documenting the required implementation effort and architectural changes.

The requirement is considered fulfilled if: (i) a new method can be integrated without modifying the core data model, and (ii) integration requires only configuration or isolated module changes.

Req. 4 Traceability and Transparency

It must be possible to trace each final risk classification back to its underlying parameter values and intermediate calculations. This is necessary to ensure that risk decisions can be reviewed, validated, and justified, particularly in safety-critical contexts. These architectural considerations provide the foundation for the prototype implementation described in Chapter 5.

The evaluation is performed by selecting computed risk results and tracing them back through the system to their originating parameters and intermediate steps.

The requirement is satisfied if all final results can be: (i) decomposed into intermediate steps, and (ii) linked to original input values in a reproducible manner.

4.3 Method for RQ2: Supporting features and Collaboration

To address RQ2, this study investigated how collaboration and interoperability may arise in existing threat analysis and risk assessment workflows and how a supporting software tool can mitigate these possible challenges. The analysis focuses on the identification of structural limitations in current threat modeling tool and the architectural requirements that enable collaborative and risk assessment processes.

4.3.1 Analysis of Collaboration Challenges

The analysis focuses on identifying structural limitations in existing threat modeling tools that may hinder collaborative risk assessment. Particular attention was given to workflow assumptions, methodological coupling, and mechanisms for sharing assessment results between stakeholders. Moreover, focus on identifying where information may become fragmented, duplicated, or difficult to trace when multiple stakeholders participate in the same assessment.

Interdisciplinary Coordination Challenges

Risk assessment in complex systems typically involves several different stakeholders and users, with different areas of expertise. For example, cybersecurity specialists, system architects, safety engineers, and operational personnel. They all may contribute to the analysis. These stakeholders often rely on different analysis approaches and may focus on different aspects of the system's risks. When collaboration is then performed using separate tools or isolated workflows the information must then be exchanged manually through for example

documentation or reports. This process may therefore result in the risk assessment becoming fragmented and more difficult to maintain consistent risk management and to track how individual risk evaluations influence the final assessment.

4.3.2 Derived Requirements for Collaborative and Interoperable Risk Assessment

Based on the identified challenges, a set of architectural requirements was identified to support collaborative and interoperable risk assessment workflows. These requirements describe the capabilities that a supporting tool must provide in order to enable coordinated analysis between multiple stakeholders.

Req. 5 Multi-User and Concurrent Assessment Support.

Req. 6 Documentation and traceability of values.

Req. 7 Project-Based Isolation and Shared Context.

Req. 5 Multi-User and Concurrent Assessment Support

Risk assessment in complex environments are often done by multiple stakeholders and users, working possibly in different aspects of the system simultaneously. The tool must therefore support parallel analysis activities, allowing users to evaluate assets, parameters and threats concurrently without interfering with each others work. Supporting concurrent assessments also enables distributed teams to then collaborate on the same system model while maintaining a consistent representation of a risk analysis.

The evaluation of this requirement will be conducted in by letting several Locust-agents add risks at the same time.

The requirement is satisfied if several users can add risks without: (i) interference from other added risk, and (ii) the tool does not disturb the workflow when presenting risk from another user.

Req. 6 Documentation and traceability of values

When performing HARA/TARA analysis, values are assigned to different risk parameters. To support collaboration and later review, it is important that the reasoning behind assigned values can be documented and traced. This enables analysts to understand how specific evaluations and risk classifications were derived during the assessment process.

The evaluation of this requirement will be how well the proposed artifact can represent current frameworks annotation-strategies.

The requirement is satisfied if the tool can add annotation: (i) in line with current frameworks annotation-strategies (ii) without limiting or disturbing other logic of the risk assessment framework.

Req. 7 Project-Based Isolation and Shared Context

Risk assessments may be conducted for multiple systems or case studies within the same tool environment. In practical settings, several stakeholders may also collaborate on different assessments simultaneously. Therefore, the system must support project-based isolation to ensure that individual analyses remain separated while still allowing shared access for authorized project members.

Compared to traditional spreadsheet-based approaches, this structure improves organization and supports collaborative risk assessment workflows across multiple projects.

The evaluation of this requirement will be conducted by testing the shared project function.

The requirement is satisfied if multiple users can share and use a project without: (i) affecting unintended users (ii) limiting or disturbing other logic of the risk assessment framework.

4.4 Method for RQ3: Exploration of LLM Integration

To address RQ3, expert input was collected after the completion of the first iteration through a structured expert workshop. The experts are employees at RISE with experience in safety and cybersecurity. During this session, the domain experts evaluated several potential AI and LLM integration concepts for the developed tool. The purpose of the workshop was to identify which AI-supported functionalities were considered most valuable and feasible for integration into the system. Participants were presented with a set of candidate AI implementation ideas and were asked to vote on the concepts they believed would provide the greatest benefit within the context of TARA-based risk assessment. The outcome of this expert evaluation was used to prioritize AI-supported features for potential implementation in the prototype system.

4.4.1 Selection Procedure

Each participant, where 5 participants answered the survey, reviewed the proposed AI integration concepts and indicated their preferred alternatives through a voting process. The concept receiving the highest level of support among the participants was selected for further development and implementation in the prototype. The proposed integration concepts are shown later in Table 6.1, Section 6.

4.4.2 Second Literature Review

To explore the potential implementation of an LLM component in the proof of concept tool, an iterative literature study was conducted on AI-assisted threat modeling, automated risk identification, and decision support systems.

The literature study focused on key areas. First the use of LLMs and AI in cybersecurity risk assessment was examined. Secondly, research on LLM processing techniques for extracting and prompting relevant information. Since the proposed feature relies on the analysis of textual input. Particular emphasis was placed on the identification how LLMs can be used to analyze system descriptions and existing risk data to support the identification of additional risk that a human operator of the tool might have missed. The findings from this literature review were used to derive both functional and non-functional requirements for the proposed LLM integration, ensuring that the design of the LLM component aligns with established research and addresses known challenges in AI-assisted analysis.

4.4.3 Derived Requirements for LLM implementation

Based on the findings of the literature review, a set of non-functional and functional requirements were derived for integrating LLM into the TARA/HARA tool. These requirements reflects both capabilities and limitations of the LLM-based systems, as well established principles from decision support systems.

Req. 8 The system shall be able to provide the LLM with an optional system description and existing risk data.

Req. 9 The system shall generate candidate risks using the LLM.

Req. 10 The system should present results in a structured format, including a confidence score.

Req. 8 System Description and Risk Data Input

The LLM component in the tool must accept user provided system descriptions (text-based descriptions of the architecture) and access existing risk data from the current assessment made by the user. The system description is optional since the LLM receives some context from the existing information in the project. But with a description, the LLM operates within the specific system context rather than making a more general suggestion if no description is given.

To evaluate this requirement a case study will be conducted with a completed risk assessment sheet, where the systems description and existing risks is given to the LLM.

The requirements are satisfied if (i) the LLM generated suggestions demonstrate a solid understanding of the system and identify relevant risks. If an identified risk is relevant will be determined by the LLMs ability to identify risks within the scope of the given system description.

Req. 9 Candidate Risk Generation

The LLM shall generate candidate risks representing threats or hazards that are not already identified, while the LLM suggestions remain consistent with the selected methodology and try to avoid similar or duplicates of existing risks.

The evaluation strategy for Req. 9 is as follows: Submit test system description to the LLM component and verify that candidate risks are generated. Inspect the generated suggestions to ensure that they are non trivial and follow the expected format.

This requirement is satisfied if the generated LLM risks for all test cases demonstrate a good understanding of the system, able to replicate risks created by HARA/TARA experts based on a given system description.

Req. 10 Structured Output Format with Confidence Scoring

The generated risks from the LLM must be presented in a structured JSON format, that includes: description of the risks, the type, confidence score (low,medium,high), and the reasoning. A human-in-the-loop [36] modal interface then allows the users to accept or reject the suggestions.

The evaluation strategy for Req. 10 is based on testing the output from the LLM by then verifying JSON schema conformance. By checking that the required fields are filled. Confirming that the modal interface correctly displays and accepts the user input.

This requirement is satisfied if all of the LLM outputs conform to the defined JSON schema and the modal interface successfully processes user selections.

4.5 Expert Interview

Following the evaluation of the proof of concept tool, semi-structured interviews were conducted with domain experts to evaluate the tool from a more practical perspective. The participants consisted of two experts selected in collaboration with RISE. Both experts had extensive experience with HARA/TARA workflows and practical risk assessment experience within industry settings. The main purpose of the interviews was to explore how the artifact was perceived regarding usability, flexibility and its potential real world application. During the semi-structured interview, the experts were asked to reflect on the tools they use, workflows, challenges and to evaluate how the tool could align with their practices. The interviews explored how the agnosticism is perceived and if it can support their specific methods. These interviews were used for open-ended feedback that enabled more insights into the practical environments and requirements for further development. The collected data were analyzed to identify themes regarding usability, collaboration and the integration to real world workflows.

4.6 Summary

This chapter has presented the derivation of project requirements, based on literature analysis and expert feedback. A list of the requirements is given in Table 4.1, that were based on literature analysis and expert feedback. The identified requirements reflect the need for a flexible and method agnostic tool that is capable of supporting a structured risk assessment process. The focus were on conformability, collaboration, traceability and the potential integration of LLM-based supporting features. Together the requirements form the foundation for the design and implementation of the tool.

Table 4.1: Summary of derived requirements

Req.	**Description**
R1	Support for hierarchical calculations.
R2	Configurable and parameterized scoring mechanisms.
R3	Extensibility and method flexibility.
R4	Traceability and transparency.
R5	Multi-user and concurrent assessment support.
R6	Documentation and traceability of values.
R7	Project-based isolation and shared context.
R8	System Description and Risk Data Input.
R9	Candidate Risk Generation.
R10	Structured Output Format with Confidence Scoring.

5 Implementation

This chapter describes the implementation of the proposed method-agnostic HARA/TARA tool. Building on the requirements and design choices, that were identified in the previous chapter, this chapter explains how the systems architecture, data model and components were implemented. The focus is on demonstrating how the conceptual design has been translated into a concrete software implementation. The chapter also covers some of the supporting features and the integration of the LLM-based component, with an emphasis on how these will contribute to the flexibility and scalability.

5.1 System Overview

The prototype follows a layered architecture consisting of a React frontend, a Flask REST API backend, a Python data access layer, and a MariaDB database. An external LLM provider is integrated through the backend for AI-assisted risk suggestions. Figure 5.1 shows the main system components and their communication paths.

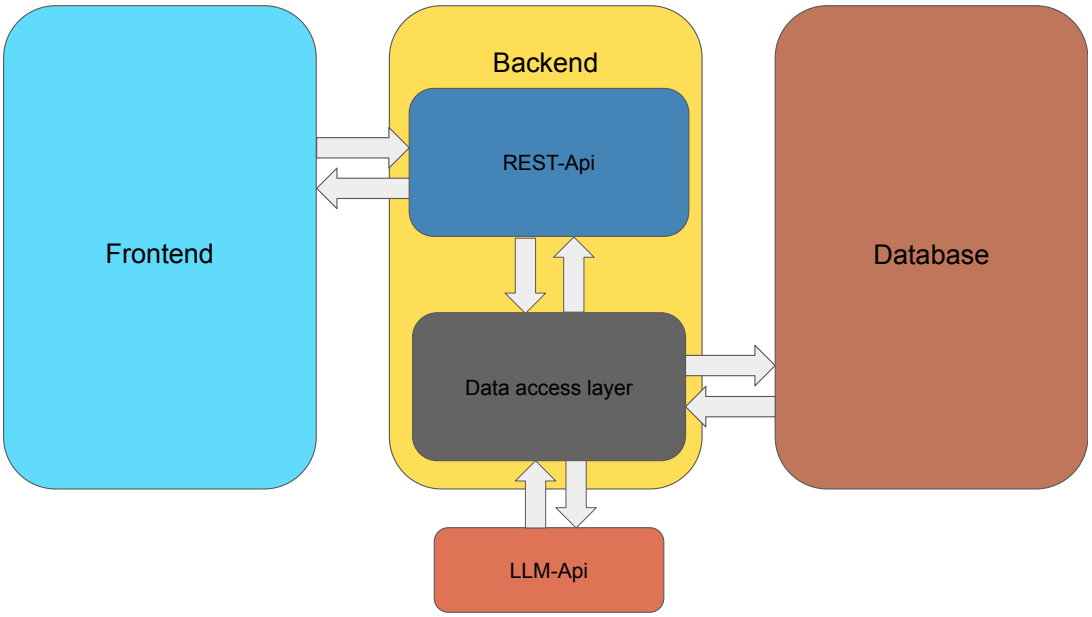


Figure 5.1: Main system components and their communication paths.

5.2 Data Model

The architectural design of the proposed TARA/HARA support tool is centered around enabling a method-agnostic, extensible and modular representation of risk assessment logic. This design choice was motivated by the observation that existing risk assessment tools are often tightly coupled to a specific methodology, making it difficult to adapt them to new frameworks or domains. By separating the representation of assessment logic from method-specific concepts, the proposed architecture enables multiple TARA/HARA approaches to be represented within a common structure while maintaining a consistent internal model. The primary architectural objective is to decouple the system model from the risk assessment methodology, thereby allowing different TARA approaches, such as HEAVENS or SAHARA to be represented within the same underlying structure. Figure 5.2 illustrates the conceptual data architecture of the system. The data model separates concerns between projects, Risk Assessment Methods (RAM), Evaluation Sheets (ES), *dimension*s, *parameters*, and their respective *parameter values*. This separation ensures that new assessment methods can be introduced without modifying the core system model.

5.2.1 Entity Structure and Core Entities

The architectural foundation of the proposed tool is structured around four primary entities: *projects*, RAM, ES, and *dimension*s. These entities define the contextual scope of a risk assessment and the methodological structure to perform it. Figure 5.2 illustrates the relationships between these entities. Relationships between the entities follow a hierarchical structure. A project may instantiate one or more RAMs. Each RAM consists of one or more ES, and each ES contains a hierarchy of *dimension*s. *dimension*s, may contain either sub-*dimension*s and/or *parameters*, while *parameters* contain multiple *parameter values* representing possible scoring alternatives.

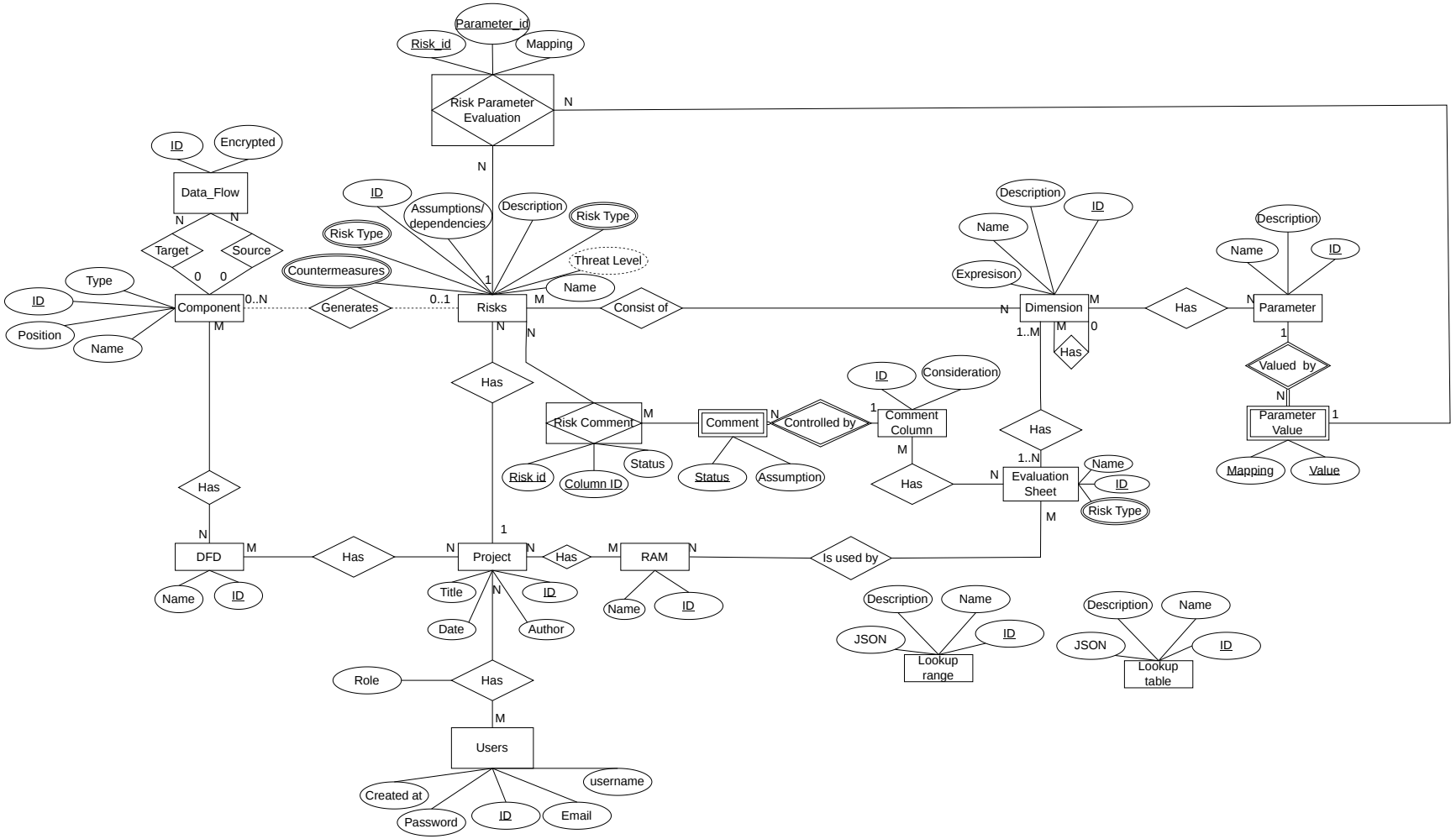


Figure 5.2: ER diagram of the MariaDB database of the proposed artifact.

Project

A project may represent a specific system, case study, or a real-world implementation that is subject to risk assessment. In the context of this thesis, a project may, for example, correspond to a subsystem within a grid or a defined OT/IT architecture. The project entity serves as the contextual container for all risk-related data, which includes identified assets, threats, hazards, and calculated risk results. By isolating the risk assessments within a certain project, the tool then enables parallel analyses of multiple systems without cross-contamination of data.

Risk Assessment Method

A RAM defines the methodological structure used to evaluate risks within a project. The RAM is designed as a reusable template rather than a project-specific instance. This allows the same RAM to be applied across multiple projects, ensuring consistency and comparability between assessments. There is also a possibility to create a copy of a created RAM should there be a need for a version not affected by other projects.

Evaluation Sheet

Each RAM consists of one or more ES. An ES represents a structured assessment unit that defines which *dimension*s and columns are used to analyze each risk. Conceptually, an ES corresponds to a scoring model or calculation framework. For example, in a HARA/TARA focused RAM, one ES may represent the calculation of an ASIL, while in a HEAVENS-based RAM, an ES would represent the determination of a SL calculated from TL and IL.

5.2.2 Dimensions

In order to support different risk assessment methods within a common architecture, the proposed data model introduces the concept of a *dimension*. A *dimension* constitutes the central abstraction layer of the proposed data model. A *dimension* represents a logical calculation unit within an ES and defines how risk-related values are structured and computed. Conceptually, a *dimension* can either contain a set of parameters and their associated parameter values or one or more sub-*dimensions*. This recursive structure enables the representation of hierarchical risk assessment models in which complex metrics are derived from multiple intermediate calculations. For example, in HEAVENS, the final SL is derived from a combination of TL and IL. In the proposed architecture, TL and IL can be modeled as sub-*dimensions* under a parent *dimension* representing SL.

The recursive structure of *dimension*s provides a flexible mechanism for representing complex risk evaluation models. Many risk assessment methodologies derive final risk levels through multiple intermediate calculations that combine qualitative and quantitative factors. By allowing *dimension*s to contain either parameters and/or other *dimension*s, the proposed architecture can represent both simple scoring schemes and multi-stage evaluation models within the same structural framework. This approach enables the tool to remain agnostic to specific methodologies while supporting a wide range of assessment approaches and maintaining a consistent and extensible internal representation of the evaluation logic.

Figure 5.3 illustrates an example of such a hierarchical construction. A root *dimension* contains two child *dimension*s, where one of them further contains additional sub-*dimension*s and parameters. This layered composition allows evaluation logic to be structured similarly to mathematical formulas. Each *dimension* may optionally contain a formula defining how its value is computed from its child *dimension*s or parameters. By using this structure, the tool can represent both qualitative and quantitative assessment approaches. Importantly, *dimension*s are independent of any specific standard. This ensures that the architecture remains method-agnostic and adaptable to evolving industry standards.

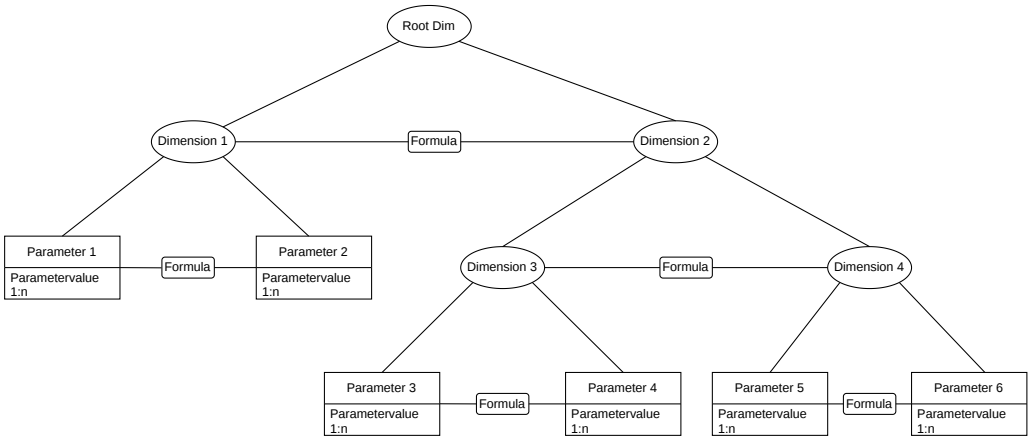


Figure 5.3: Example of a hierarchical ES structure consisting of three layers of *dimension*s. A *dimension* can contain either Parameters or additional sub-*dimension*s, allowing nested aggregation and structured risk calculation.

5.2.3 Parameters and Parameter Values

Parameters represent the atomic scoring elements within a *dimension*. While *dimensions* define structural logic, *parameters* define measurable or selectable inputs. A *parameter* consists of a name, a defined scale or value range, and a set of predefined *parameter values*. *Parameter values* correspond to selectable scoring alternatives (e.g., 1–5 scale, categorical options such as "Low", "Medium", "High", or predefined severity classes). For example, in SAHARA, *parameters* represent Required Resources (R), Required Know-how (K) and Threat Criticality (T). In HEAVENS, *parameters* represent Expertise, Equipment, and Safety factors. Each *parameter value* is associated with a numerical representation to enable automatic calculation within its parent *dimension*. This approach enables traceability between qualitative selections and quantitative results.

5.3 Relational Database and MariaDB

A relational database approach was chosen since the proposed tool relies on structured storage and management of interconnected entities such as threats, assets, *parameters*, *dimensions*, and risk assessment results. The use of SQL enables clear definition of schema, relationships, and constraints, which is essential for maintaining data integrity in systems with interconnected entities. Through features such as primary and foreign keys, SQL supports consistency and prevents invalid data states, which is critical for the correctness of the application. In this project, SQL serves as the primary interface for interacting with the system's central data store, namely the MariaDB server and constitutes a foundational component of the system architecture. The SQL-based database structure builds upon the previous proof-of-concept implementation developed by Gustafsson and Hagel [37], where configurable entities, relationships, and hierarchical structures were introduced to support method-agnostic HARA/TARA representation. In the context of this thesis, the database design was further extended and refined to improve scalability, interoperability, and support for collaborative risk assessment workflows. The use of SQL enables explicit representation of entities, relationships, and hierarchical structures used throughout the tool. This is important since the architecture relies on interconnected representations of threats, assets, *parameters*, *dimensions* and risk assessment results. Structured schemas and relational constraints assist in maintaining the consistency and traceability of the stored assessment data. MariaDB was selected as the database management system due to its performance, reliability, and compatibility

with widely used SQL standards. As an open-source system, MariaDB provides a cost effective solution while still offering robust features such as indexing, transaction support, and concurrency control. Additionally, MariaDB integrates well with backend technologies and frameworks, making it straightforward to connect the database layer with the application logic. Its support for standard SQL also ensures portability, allowing the system to be migrated or extended with minimal changes if needed.

5.4 Database Design

The MariaDB database is initialized through multiple SQL scripts, each serving a distinct domain-specific purpose. This structured approach separates schema definitions, relational constraints, stored procedures, and database views into dedicated components in order to improve clarity and maintainability.

The database schema defines the primary tables and their associated attributes, while relational constraints enforce referential integrity and define relationships between domain entities. Stored procedures and database views are used to encapsulate reusable query logic and database-related operations.

Separating database initialization into responsibility-focused scripts contributes to a clearer distinction between structural definitions and operational functionality. This improves maintainability and makes it easier to trace how core entities such as *projects*, *risks*, *parameters*, and evaluation sheets are represented within the database.

5.4.1 Stored procedures

Most database operations are encapsulated in stored procedures rather than constructed as inline SQL within the application layer. The design decision provides several concrete benefits. First, it reduces the risk of SQL injection vulnerabilities, as the Python data access layer never constructs raw query strings from user input but instead calls named procedures with typed *parameters*. Second, it establishes a stable and well-defined interface between the application and the database, meaning that internal schema changes can be absorbed at the procedure level without requiring modifications to the Python layer. Third, it centralizes query logic in a single location, improving traceability and making the database behavior easier to test and audit.

Each stored procedure follows a consistent contract: input *parameters* are validated against the current state of the database, output *parameters* communicate success or failure back to the caller, and exceptions are handled via `EXIT HANDLER` declarations where appropriate. This uniformity simplifies the data access layer considerably, as every Python function follows the same pattern of acquiring a connection, invoking a procedure and returning the result.

5.4.2 Connection Management

Database connections are managed through a connection pool initialized at application startup using `MySQLConnectionPool`. Rather than opening and closing a new connection for each incoming request, which carries significant overhead in terms of authentication and resource allocation, the pool maintains a fixed set of reusable connections, 5 in our case, that are checked out and returned as requests are processed. This approach reduces latency for individual requests and improves the overall throughput of the backend under concurrent load. The pool is initialized lazily on first use, meaning no database connection is established until the application actually requires one.

5.4.3 Hierarchical *dimension* Structure

The evaluation model supports a recursive *dimension* hierarchy, where *dimension*s can contain both Parameters and nested child *dimension*s. This structure is represented in the schema through a self referencing junction table `dimension_dimensions`, which stores parent-child relationships between *dimension* entities. Traversing this hierarchy requires a recursive query, which MariaDB supports through common table expressions with the `WITH RECURSIVE` construct. However, MariaDB 11.4 [54] and earlier versions does not support recursive CTEs within stored procedures, which necessitated moving the tree traversal query directly into the python data access layer. While this represents a pragmatic exception to the otherwise procedure-based data access pattern, it preserves correctness and avoids duplication of recursive logic across multiple stored procedures.

5.5 Backend Architecture

Python was chosen as the implementation language for the backend due to its high development efficiency, readability, and strong ecosystem support. The language's clear and concise syntax facilitates rapid development and improves code maintainability, which is particularly important in the context of this thesis where development time was limited.

In addition, Python provides well-documented frameworks for backend development, such as Flask [55]. These frameworks reduce the amount of boilerplate code required and offer built-in solutions for common backend concerns, enabling a more structured and reliable implementation.

Finally, Python was chosen as it has strong support for modularity and extensibility, which allows easy integration with databases and external services. Another advantage of this extensibility is that this thesis serves as the groundwork for a future open-source project, facilitating the addition of future functionality.

5.5.1 REST Architectural Style

The backend exposes its functionality through a RESTful API. RESTful was selected due to its stateless communication model, resource-oriented structure and compatibility with modern frontend frameworks. This architectural choice ensures loose coupling between the React-based presentation layer and the Python-based application layer.

Each core domain entity (e.g., Project, *dimension*s, Risk) is represented as an addressable resource identified by a unique URI. Standard HTTP methods are used to implement CRUD operations: `GET` retrieves resources, `POST` creates new entries, `PUT/PATCH` updates existing entities and `DELETE` removes them.

The API follows a stateless design, meaning each request contains all necessary contextual and authentication information and no session state is stored on the server. This simplifies scaling the system by adding additional server instances and improves overall system robustness.

Data exchange between client and server is performed using JSON representations, ensuring interoperability and seamless integration with the JavaScript-based frontend.

The REST abstraction further supports the model-agnostic design of the system, as risk models and evaluation parameters are retrieved dynamically via API endpoints rather than being embedded within the frontend. This prevents coupling of presentation logic to specific safety or security standards.

To illustrate how the REST-based architecture supports system configuration and risk evaluation workflows, sequence diagrams are presented in Figures 5.6 and 5.5. Figure 5.6 shows the configuration workflow, where users define dimensions, parameters, and value scales through the frontend, which are then persisted through the backend and database

layers. Figure 5.5 illustrates the parameter-based risk evaluation process, including how parameter values are submitted, aggregated, and transformed into a final risk level.

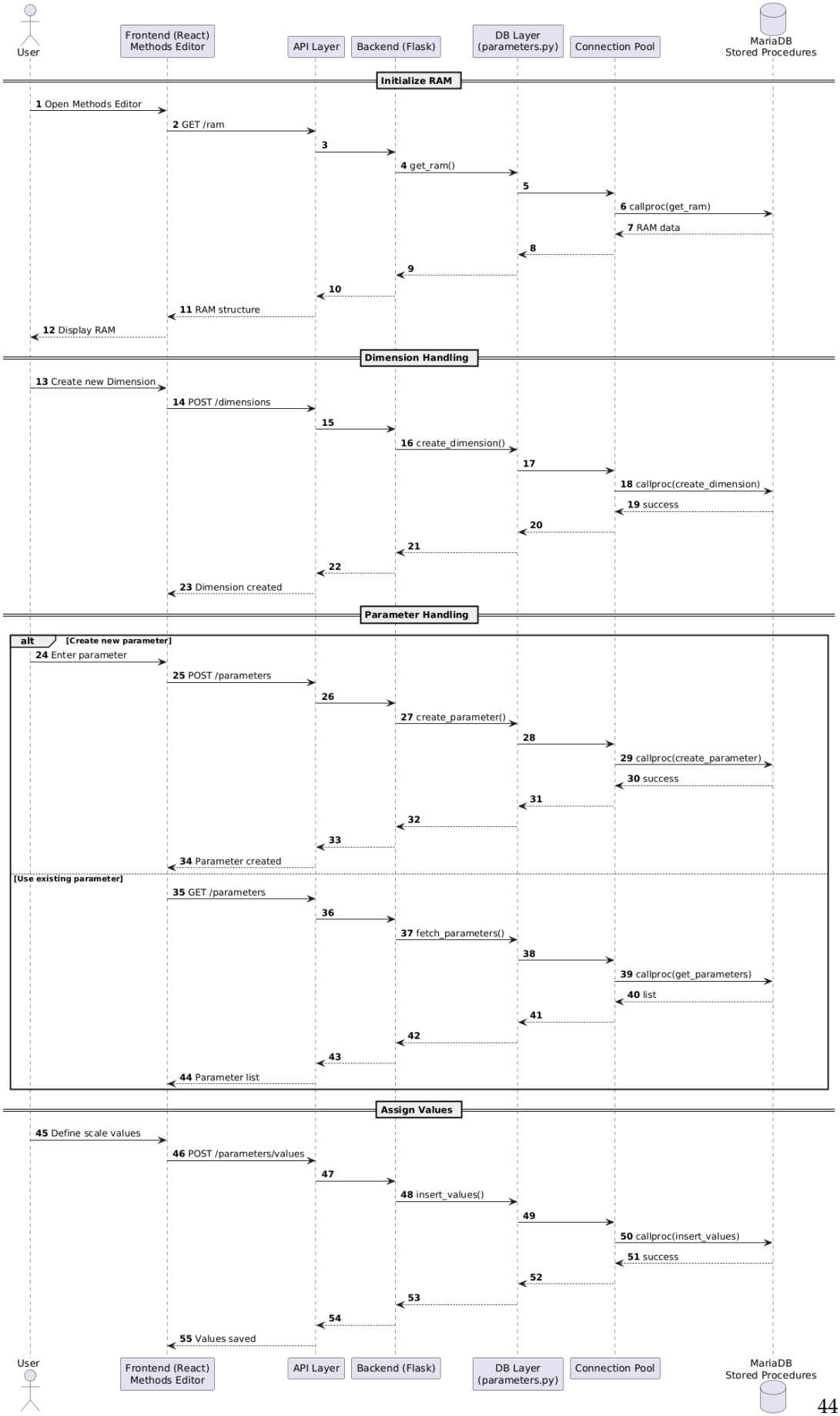


Figure 5.4: Sequence diagram illustrating the configuration workflow in the TARA/HARA tool. The diagram shows how users define *dimension*s, create or select parameters, and assign value scales through the frontend. These actions are propagated through the REST-based backend to the database layer, where stored procedures are used to persist the configuration.

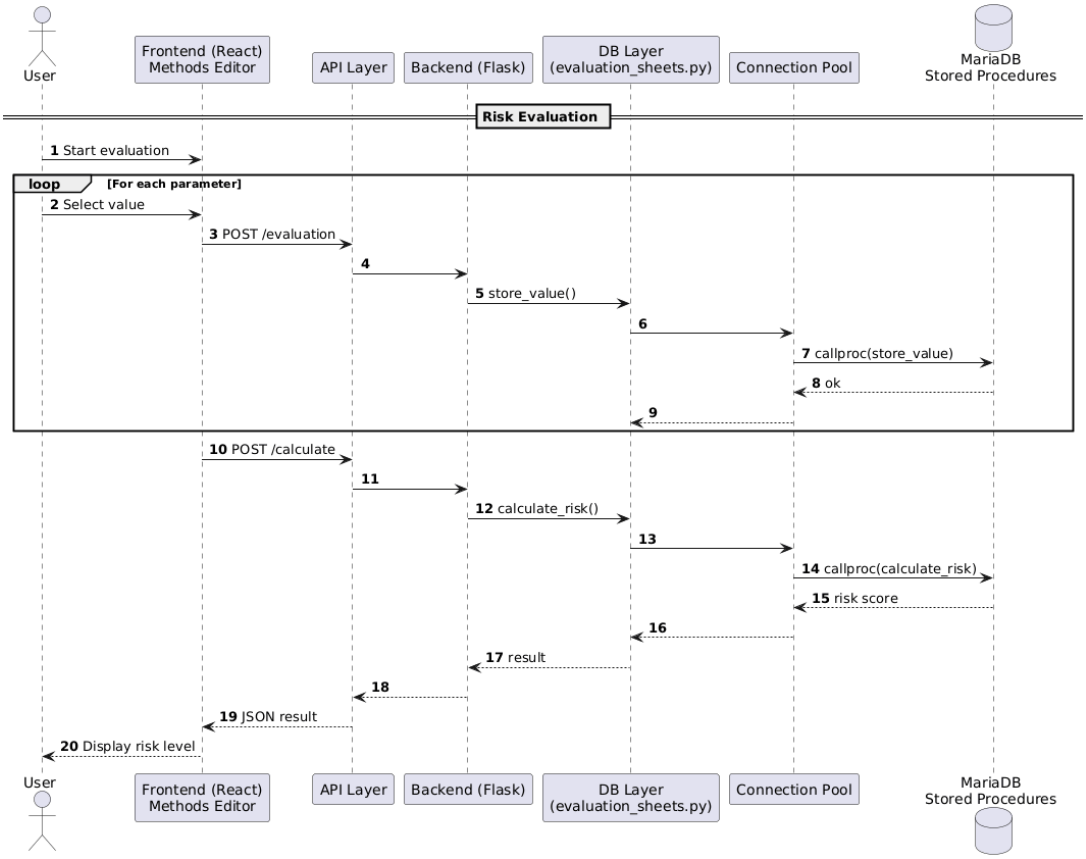


Figure 5.5: Sequence diagram illustrating the parameter-based risk evaluation process. The diagram shows how parameter values are iteratively submitted and processed across the system layers, and how stored procedures in the database are used to aggregate these values and compute a final risk level.

5.5.2 Backend Structure and Flask Implementation

The backend server is implemented using the Flask framework and structured as a dedicated Python package. The system follows a layered architecture consisting of a presentation layer responsible for REST API routing and request handling, a domain layer containing business logic and risk evaluation mechanisms and a data access layer responsible for database communication and persistence management.

To support modularization within a monolithic deployment, Flask Blueprints are used to encapsulate domain-specific functionality. Each core domain entity (e.g., Project, *dimension*s, Risk) is implemented as an independent blueprint. This approach groups routing logic, validation and related request handling code into cohesive and clearly defined modules.

Blueprints delegate business operations to the domain layer, which encapsulates risk computation logic and model-agnostic evaluation mechanisms. Persistence concerns are handled separately by the data access layer. This structure enforces separation of concerns between transport, business logic and data storage. The use of Blueprints aligns with the modular monolith design pattern, as it enforces internal boundaries between functional domains without introducing the operational complexity associated with distributed microservices. The architecture therefore achieves high cohesion within modules and low coupling between architectural layers, improving maintainability, extensibility and traceability.

Although the system follows a modular monolithic deployment model, the REST interface provides a clearly defined external boundary. This abstraction allows future decomposition into independent services without requiring fundamental changes to the client-facing API.

5.5.3 Security considerations

This prototype implements several security measures across the different architectural layers. At the authentication level JSON Web Tokens (JWT) are used to secure the API endpoints. Each request from the frontend carries a signed token in the `Authorization` header, which the backend validates before processing the request.

The operations at the database level are encapsulated, in stored procedures. This means that the Python data access layer never constructs raw SQL strings from user input. However, some exceptions exist, as discussed in Section 5.4.3. The database is not directly exposed to the public network, within the deployment environment, the MariaDB instance is only accessible from the backend container. This meaning, that external actors cannot connect to the database directly. Database credentials are managed through environment variables (`DB_HOST`, `DB_USER`, `DB_PASSWORD`) rather than being hard coded in the source code, reducing the risk of credential exposure. The prototype also supports role based access control at the project level. The `project_users` table assigns each user one of three roles: `owner`, `editor` or `viewer`.

5.6 GUI Design

The frontend of the system was implemented using React and JSX. React was selected since the proposed tool contains several interactive views and dynamically updated UI elements, including project management pages, parameter configuration interfaces, risk assessment sheets and visualization components. React enables the decomposition of the user interface into reusable and independent components, which improves code organization, maintainability, and scalability. This is particularly beneficial in projects where the interface consists of multiple interactive elements that need to be consistently managed and updated. A key advantage of React is its use of virtual DOM, which optimizes rendering performance by minimizing direct manipulation of the browser's DOM. This results in efficient updates and responsive user experience, especially in applications with frequent state changes. JSX was used to implement a lot of the reusable fronted components and dynamically render structured data that are received from the backend. Components such as assets, threats, dimensions, parameters and risk assessment results. This in turn simplified the implementation of interactive tables, forms and updated user interface dynamically, throughout the application. Furthermore, React integrates well with REST-based backends, making it suitable for the project stack as a whole. Overall, React and JSX were selected as they provide a structured, efficient, and scalable approach to frontend development, enabling the creation of a maintainable and responsive user interface.

5.6.1 The fronted Architecture

The fronted is implemented in React and follows the same modular monolith principle as the backend. Each page of the tool is a somewhat self-contained module with clearly defined responsibility, such as project management, evaluation sheet editing, *dimension* configuration and data flow diagramming. All of the API communication is separated into a dedicated `api/` layer. No page component communicates directly with the backend. Instead, named functions such as `getRisksByProject` or `updated`*`dimension`*`Expression` are called, keeping presentation logic decoupled from transport logic. Heavier UI functionality is further encapsulated into reusable components such as `EvaluationSheetView` and `ExpressionBuilder`, which can be composed into pages without duplicating logic. Styling

is scoped per module through dedicated CSS files, preventing style conflicts between components. This structure mirrors the modular monolith pattern described in Section 2.4.2, where internal boundaries are enforced through module separation rather than physical service decomposition. This separation of components are important for the modularity of the project. An overview can be seen in the component diagram shown in Figure 5.6.

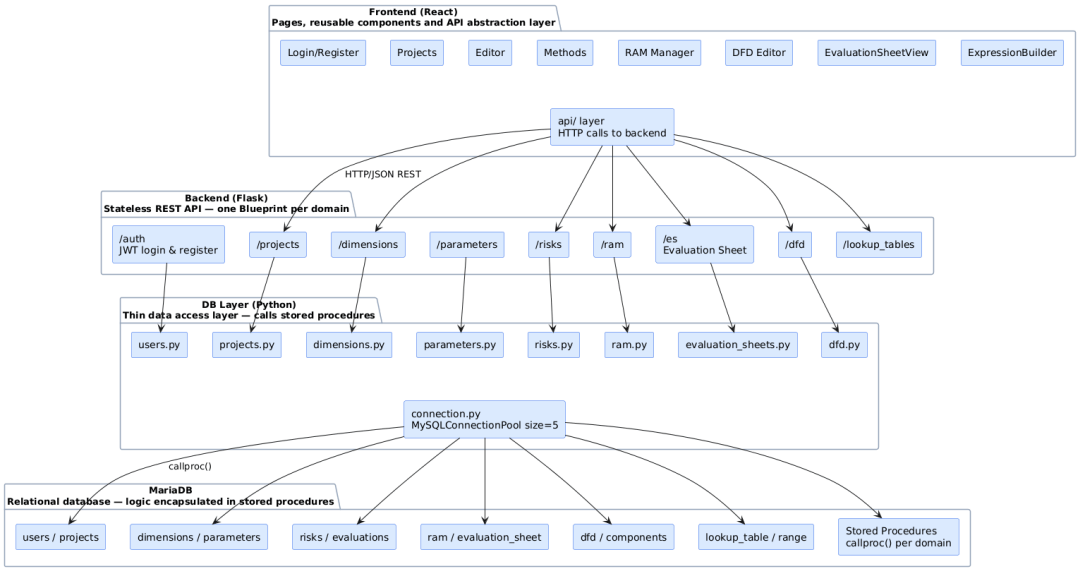


Figure 5.6: Component diagram illustrating the layered architecture of the TARA/HARA tool. The system is structured into four layers: the React-based frontend, the Flask REST API backend, the Python database access layer, and the MariaDB database. Each layer communicates strictly with the layer directly below it.

5.6.2 Usability Considerations

To reduce the amount of manual effort required when configuring new risk assessment frameworks. Many parts of the code such as the *parameter* creation form includes a set of predefined quick templates. These templates cover the commonly used *parameters* such as Likelihood, Consequence, Severity, Exposure and Controllability, which are used in frameworks such as HEAVENS and SAHARA [1], [56]. Selecting a template automatically populates the *parameter* name, description and scale values, allowing users to set up standard parameters in a single click rather than entering all values manually.

Since the tool is designed to be method-agnostic, new templates can be added directly in `parameterform.js` by extending the `PARAMETER_TEMPLATES` object with a new entry. This means that as new risk assessment methodologies are adopted, corresponding parameter templates can be added without modifying any other part of the system. However, a limitation of the current approach is that templates are hard coded in the frontend and are not configurable through the user interface. For a tool that aims to be fully method-agnostic, a more flexible solution would allow users to define and save their own templates directly in the application.

5.6.3 Concurrency model

The system supports multiple simultaneous users through a combination of a stateless backend, a database connection pool, atomic stored procedures and a client side polling mechanism. The backend is implemented as a stateless REST API where no session state is stored server-side between requests. Each request is self-contained and carries its own authentica-

tion token, allowing multiple users to interact with the system concurrently. Since no user-specific session state is maintained on the server, requests can be processed independently of one another. The database connections are managed through a connection pool initialized at application startup, on the server, using `MySQLConnectionPool`.So rather than opening a new connection for each of the incoming client requests, connections are checked out from the pool when the request is done and as a consequence reduces latency under concurrent load. At the database level, concurrency safety is partly achieved through the use of `ON DUPLICATE KEY UPDATE` in critical stored procedures, such as `add_risk_comment`. When concurrent write operations target the same row, the database automatically resolves key conflicts by performing an upsert operation rather than generating an error or creating duplicate records. This ensures that each write operation is executed atomically and that the database remains in a consistent state under concurrent load.

On the client side, the evaluation sheet implements a polling mechanism using `setInterval` that fetches the latest risk data from the Flask server every five seconds, so that if changes are made by any user these are reflected without a manual page refresh. To prevent this overwriting ongoing edits, polling is automatically paused when the local `isDirty` flag is set and resumes after one of the users saves the current edit. A known limitation is that no explicit conflict resolution exists at the application level. If two users modify and save the same risk entry simultaneously, the last write wins silently. This will then be updated for all users when the polling is active again.

5.7 Supporting Features

Following the first iteration, an expert review was conducted to evaluate the implemented functionality and identify areas for improvement. The findings from this review informed the direction of the second iteration, with a particular emphasis on enhancing supporting features (see Section 6.3).

The focus of the second iteration was therefore defined to include:

- Implementation of Lookup tables and configurable value ranges to support structured parameter selection,
- Introduction of functionality to document the rationale (motivation) associated with each Parameter Value,
- Integration of LLM-based risk determination to assist in the evaluation process.

5.7.1 Building Expressions and Lookup Tables

Many risk assessment methods rely on a combination of mathematical calculations and lookup-based evaluations. For example, standards such as ISO 26262 uses tables to derive ASIL classifications from input parameters [38]. To support a wide range of assessment methods, the tool provides a configurable expression engine capable of handling both mathematical operations and lookup-based calculations.

Mathematical expressions are defined dynamically through a JSX-based interface, allowing users to combine an arbitrary number of operators and operands. For methods that require lookup tables or value ranges, the expression engine includes dedicated lookup operators. These operators enable users to reference predefined tables or ranges as part of the calculation process.

Lookup tables and ranges can be created and managed through a dedicated interface within the tool. The configurations are stored in the database as JSON-formatted objects, as illustrated in Figure 5.2. When an expression is evaluated, the relevant lookup definitions are retrieved from the database and processed in the same manner as other operators within the expression engine.

5.8 LLM Implementation

During the expert workshop evaluation (see Appendix A.3.2), multiple LLM-assisted functionalities were proposed and evaluated by domain experts. The feature where an LLM analyzes system descriptions and existing risk assessments to identify new risks was chosen to be implemented. As mentioned in Section 3.8, much previous work has been done where AI does the majority of the work; instead, here the LLM functions as a supporting component.

5.8.1 Supporting Feature: Data flow Diagram

To support identification of different system components and their potential threat and risk relationships the tool incorporates an integrated DFD editor that was built using reactflow. The DFD interface enables users to visually model different system architectures that may include processes, data stores, external entities and system boundaries. The DFD editor component in the tool supports interactive node creation and manipulation that allows the user to drag different component types on the sheet, see Figure 5.7. As the users modify the diagram, a side panel displays the current risks associated with each component. The user can also select a certain component of a system to edit or add a risk relevant to the selected component. This design then facilitates approaches such as STRIDE, where data flow and components serve as a key part of the process. The DFD components data risk is also persistent with the database, which enables an iterative refinement of system models. This also supports reuse of component evaluations across a larger project.

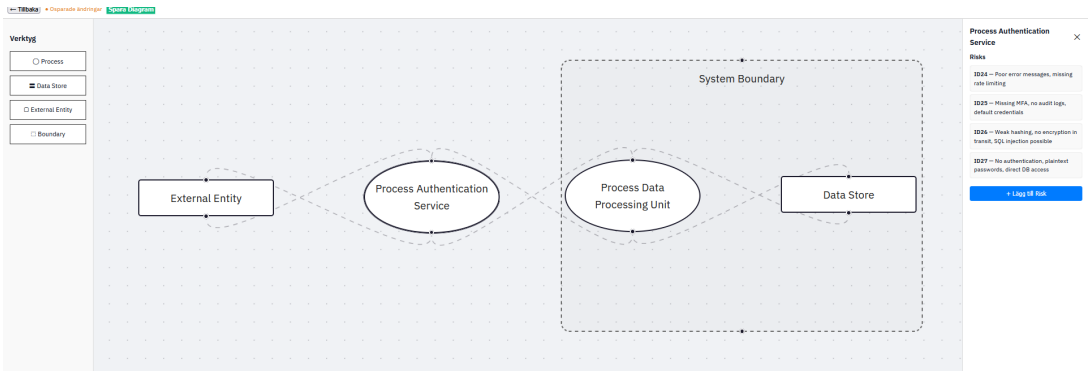


Figure 5.7: Data Flow Diagram of the Authentication Service showing external entities, processes, data stores, and identified security risks.

5.8.2 Import and Export

The tool implements two-way CSV import and export mechanisms to enable risk assessments to be integrated into existing workflows. It also supports the reuse of previously collected risk assessment data. The export functionality generates a file containing the identified risks along with the associated parameter values, expressions, comment and etc. The CSV structure preserves the hierarchical *dimension* information by including the computed expression and columns alongside with raw data values. The import functionality accepts CSV files formatted according to the export schema.

5.8.3 Integration Architecture

The LLM component is integrated into the backend system through multiple architectural layers, maintaining separation of concerns consistent with the modular monolith design pattern. When the AI-assisted suggestions are needed, the user can enter the system description input and the tool displays AI-generated suggestions for user validation. The module allows

users to accept, reject or modify the suggestions before they are integrated into the ES. The REST endpoint in the server, receives the system description and context from the frontend. The endpoint then delegates processing to the AI logic layer, where the response follows the stateless pattern.

AI Logic Layer

The `risk_ai.py` module encapsulates all LLM-related functionality:

- Prompt construction through `build_risk_analysis_prompt()`
- API communication with Claude Opus 4.1 (Anthropic)
- Response parsing and validation
- JSON schema enforcement

The Data Flow

1. User initiates AI suggestions through the modal interface
2. Frontend sends POST request to `/risks/ai_suggestions` with:
    - System description (free-text)
    - Existing risks (JSON array)
    - Framework type (SAHARA/HEAVENS/HARA)
3. Backend invokes `build_risk_analysis_prompt()` to construct structured prompt
4. Claude API processes prompt and returns structured JSON response
5. `risk_ai.py` parses and validates response
6. Suggestions are returned to frontend as JSON array
7. The user reviews suggestions in modal and accepts/rejects

The prompt is constructed to include the following components: **Task Definition**: states that the LLM should be used to identity missing risks. **Context**: Provides system architecture description and relevant framework information. **Input Data**: Lists all existing identified risks to avoid duplication. **Constraints**: Specify the input format as JSON, and schema. **Instructions**: Explicit directives regarding hallucination mitigation. The prompt explicitly instructs the LLM to analyze the system for potential threats or hazards. Compare the identified threats against the existing risk lists that has been created by the human user. Return only not existing risks to the user. Lastly, avoid speculating beyond reasonable threat scenarios.

Output Schema

The response from the AI represented in a JSON object as follows:

```
{
  "risks": [
    {
      "description": "string",
      "type": "threat|hazard",
      "reasoning": "string"
    }
  ]
}
```

where the `description` is a human readable risk statement. `type` is an classification as threat (intentional attack) or hazard (unintentional failure). Lastly an explanation for why the risk was created by the LLM and why the human expert might have overlooked it.

Example Input and Output

An example input structure entered by the user could be:

```
System description: An Smart grid energy community with distributed solar
    panels, battery storage, and IoT-based monitoring. That is remotly
    controlled through an cloud based API. Existing components: solar
    inverters, SCADA controller, Battery management system and a cloud
    backend.
```

From this system description the LLM might suggest risks such as:

```
{
  "risks": [
    {
      "description": "Man in the middle attack on IoT device communication
          to SCADA controller",
      "type": "threat",
      "reasoning": "Wireless communication between IoT devices and SCADA
          may lack encryption. This attack vector was overlooked in
          initial analysis."
    },
    {
      "description": "Loss of cloud API connectivity causing uncontrolled
          battery discharge",
      "type": "hazard",
      "reasoning": "If cloud API becomes unavailable, battery management
          system may discharge without grid feedback, causing voltage
          instability."
    }
  ]
}
```

The response from the LLM

The system applies several validation and processing steps after receiving the JSON response from Claude, to ensure data integrity and prevent duplicate entries. The response is parsed from JSON and then validated against the expected schema. If this fails it is communicated through the user with an retry request. Figure 5.8 illustrates this response.

5.8.4 Human in the Loop Validation

A critical design principle is that all LLM generated risk suggestions must undergo human approval before inserted in to the database. This design ensures that the final say is with the human user, addressing concerns about automated decision making in safety critical environments. Figure 5.9 illustrates this workflow.

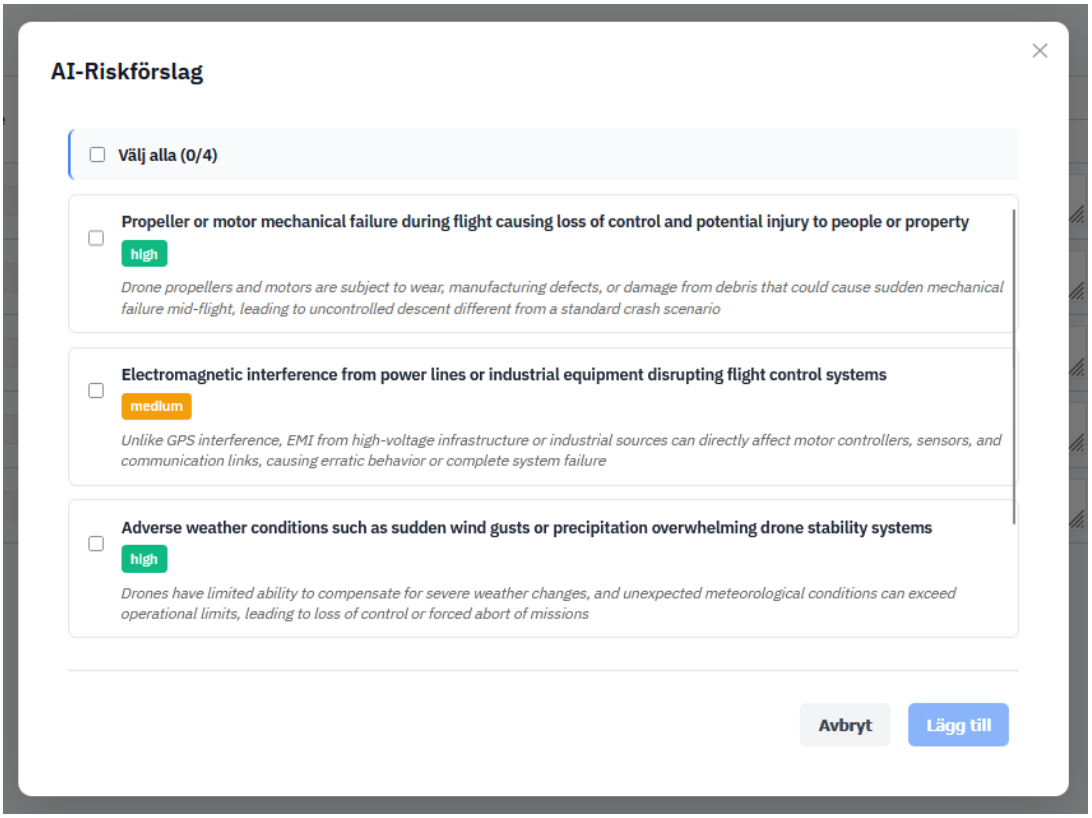


Figure 5.8: AI-generated risk suggestions based on system context and existing inputs, including severity classification and explanatory rationale. The user can review and selectively include suggestions, ensuring human-in-the-loop validation.

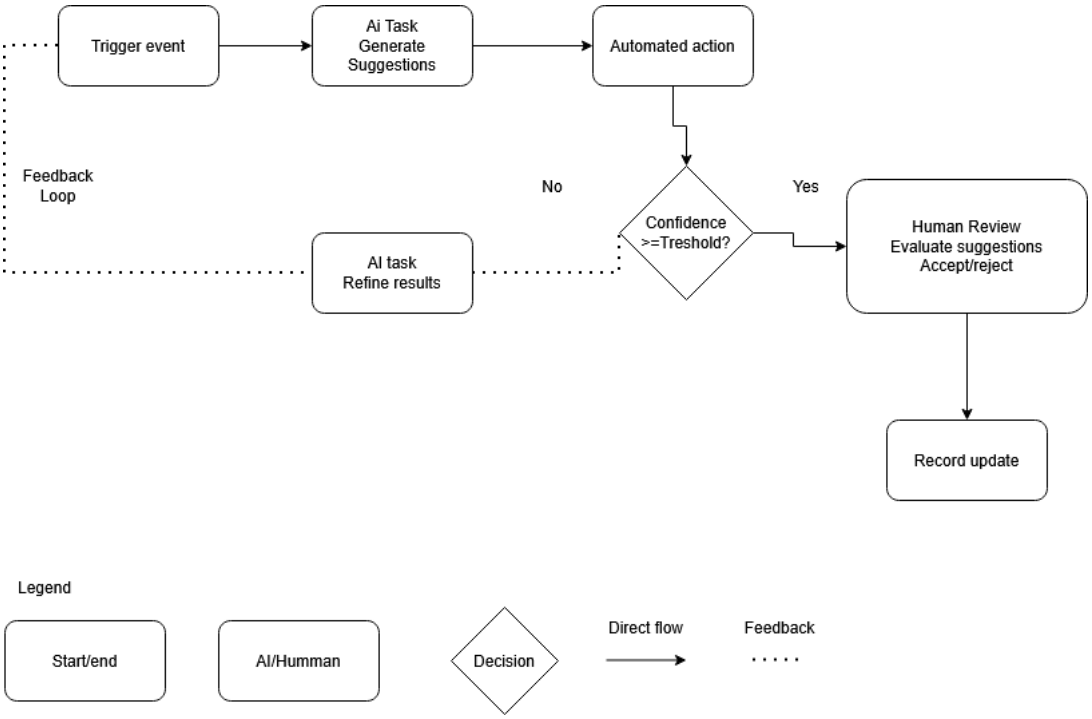


Figure 5.9: Human-in-the-loop workflow for AI-generated risk suggestions. The system validates AI confidence scores before presenting suggestions to the user for manual review and acceptance.

6 Evaluation

This chapter presents the evaluation of the proposed TARA/HARA tool. The purpose of the evaluation is to assess to what extent the developed artifact fulfills the identified requirements in Chapter 4. particularly with respect to method-agnostic representation, support for collaborative risk assessment, and the integration of LLM-based functionality. Both qualitative and quantitative methods have been used in the evaluation of the system features. The results are structured according to the research questions, and therefore provide the basis the discussion in Chapter 7.

6.1 Design Science Research Perspective

The evaluation of the artifact was conducted from a DSR perspective, where the primarily goal is to assess the utility and effectiveness of the developed artifact in solving the identified problem. In line with DSR principles, the artifact is evaluated based on the ability to support real-world risk assessment scenarios rather than purely theoretical correctness. The evaluation combines several forms of DSR artifact evaluation including demonstration of implemented methods, technical testing of system behavior, expert review of practical usefulness, and scenario-based evaluation of LLM-generated suggestions. Furthermore, the evaluation will evaluate how well the artifact satisfies the requirements set in Chapter 4. This approach ensures that the evaluation reflects both the technical design and its practical applicability.

6.2 Evaluation Objectives

The evaluation is structured around the research questions defined in Chapter 1. To ensure traceability between the research questions and the evaluation, each research question is mapped to one or more evaluation objectives:

- **RQ1 - Method-agnostic representation and extensibility:** Evaluates to what extent the tool can represent different risk assessment methodologies within a unified framework, as well as the ability of the system to adapt to new methods, parameters, and configurations without significant redesign.

- **RQ2 - Collaborative support and interoperability:** Evaluates how effectively the tool supports collaboration among multiple stakeholders and concurrent risk assessment activities.
- **RQ3 - LLM-assisted functionality:** Evaluates the reliability, usefulness, opportunities, and limitations of AI-generated risk suggestions within the analysis process.

6.3 Method-Agnostic Design Assessment

Once the core functionality within the scope of the first design iteration had been implemented, an expert review was conducted to evaluate the method-agnostic HARA/TARA tool. The experts consisted of fifteen participants from RISE, with a mix of cybersecurity and functional safety backgrounds and expert areas. These experts were selected by our supervisor at RISE. The evaluation was carried out in the form of a structured demonstration, where the implemented frameworks, specifically HEAVENS and SAHARA, were presented within the tool. Following the demonstration, an open discussion was held with the expert group to gather qualitative feedback, suggestions for improvement and reflections on the tool's applicability in practice. In addition, a separate meeting with the technical team was conducted to discuss potential directions for further development and implementation considerations for subsequent iterations.

Overall, the feedback from the expert group was positive. The participants concluded that the core functionality of the tool was well implemented and that it successfully supports the definition and application of existing risk assessment frameworks. Furthermore, the tool was considered sufficiently flexible to allow users to adapt or modify frameworks to suit specific project requirements, thereby supporting the claim that the tool is method agnostic.

Despite the positive assessment, several areas for improvement were identified. The most prominent observation was that the tool currently provides limited support for users during the analysis process. To address this, the experts suggested the introduction of additional supportive features, such as the ability for user to define lookup tables, the ability to attach comments to individual data points and mechanisms for naming and managing expressions.

In addition, several suggestions involving AI-based functionality were proposed. These suggestions were further refined and consolidated into a set of candidate features, which were subsequently evaluated through a survey. The survey contained the questions shown in Table 6.1, where participants were asked to distribute up to five points across all questions. In cases where more than five points were assigned, the responses were normalized, which is why some decimal values appear in the results. The results are presented in Table 6.2.

Table 6.1: Survey questions and corresponding AI features.

Shorthand	Survey question
CR	**Compilation report by LLM based on the current working sheet.** This function compiles a structured report of all risks in the project, highlighting critical risks and missing countermeasures while providing an easy-to-read overview. *Example:* After completing a HARA analysis, a user can generate a report with an executive summary, a severity-sorted risk table, and a section highlighting missing countermeasures.
CB	**AI chat bot that reads the project page.** This function allows querying the entire risk analysis without manual searching. *Examples:* • "Which risks have no counter assigned?" • "What is the most common hazard type?" • "What are the most critical risks and why?"
RS	**Risk suggestions from AI based on the system description.** The LLM analyzes the system description and existing risks to generate additional potential risks, which the user must review. *Example:* A user requests suggestions after entering risks; the system proposes missing risks based on analysis of the system description and current entries.
DD	**AI-based detection of duplicate risk entries.** The LLM identifies semantically similar risks even if worded differently. *Examples:* • "Loss of braking friction" vs "Brake system failure" • Identical parameters but different descriptions
CF	**AI-assisted consistency in comments and formatting.** The LLM reviews rationale fields and comments for consistency and suggests improvements. *Examples:* • Flags insufficient rationale descriptions • Suggests standardizing "N/A", "None", etc.

Table 6.2: Survey results for AI feature evaluation.

Id	CR	CB	RS	DD	CF	Total
1	0	1	3	1	0	5
2	1	1	2	1	0	5
3	1	1	2	0	1	5
4	2	1	2	0	0	5
5	0	0	3	1	1	5
6	1.18	1.47	0.59	0.88	0.88	5
7	0	0	0	2	2	4
Tot	5.12	5.47	12.59	5.88	4.88	

6.4 Extensibility and Method Flexibility

6.4.1 Chosen Risk Assessment Methods

The methods shown in List 6.4.1 were selected and tested to evaluate if they could be replicated within the tool. The methods were chosen to represent a range of risk assessment approaches, spanning from both functional safety and cybersecurity domains, as well as qualitative and quantitative methodologies. The methods were chosen based on three main criteria. Firstly that they represent either functional safety or cybersecurity perspectives or optimally both, allowing the evaluation to capture the differences between safety-based approaches and security focused approaches. Secondly, the selected methods needed to support more flexible or interpretive evaluation logic, enabling the analysis of how different scoring models and assessment approaches could be expressed within the tool. Lastly, the methods vary in complexity and characteristics, to evaluate the tools agnosticity to its best extent.

To then enable a more structured comparison between the different RAMs (risk assessment methods), a simple scoring scheme was defined and applied consistently across the evaluated features. Each feature was subsequently assessed by the thesis authors using a three-level scale reflecting the extent to which it could be represented in the tool:

- (3) - Fully supported, the feature can be implemented directly using existing functionality.
- (2) - Partially supported, the feature can be represented, but requires adaptations, workarounds, or does not fully match the original method.
- (1) - Limited support, the feature cannot be adequately represented within the current architecture.
- (N/A) - Not applicable, the feature is not relevant to the evaluated risk assessment method.

The evaluated RAMs feature include aspects such as worksheet structure, parameter representation, etc. The scoring approach provides a consistent way to compare how well different methods align with the proof of concept tools method-agnostic architecture and highlights the strength and limitations in the tool. The following methods were tested by the authors using the tool:

- **FMEA** - A widely used safety engineering method that evaluates failure through Detection, Severity and Occurrence parameters, resulting in a Risk Priority Number (RPN) [57] A.4.
- **What-If Analysis** - A qualitative brainstorming method that identifies hazards through structured scenario questioning with no risk formula [58] A.20.
- **P.H.A of Canada** - A risk analysis method that models both the causes and the consequences of a hazardous event, combining fault tree and event tree concepts [59] A.4.5.
- **hTMM** - It is a hybrid threat modeling method, a structured cybersecurity focused method combining elements from both multiple threat modeling approaches [60] A.4.3.
- **OCTAVE Allegro** - An information security risk assessment method that identifies and prioritizes security risks through threat scenario analysis [61] A.4.4.
- **FAIR** - is a risk analysis methodology that decomposes risk into hierarchical factors [62] A.4.1.

FMEA

FMEA was implemented as a single ES with three *parameters*: Severity, Occurrence and Detection on a scale of 1-10. Combined through the expression $RPN = S \times O \times D$, and a lookup table mapping RPN scores to risk levels. Free-text columns were added for Item/Function, Failure Mode, Potential Effects and Recommended Actions. Although, the structure of the free-text columns was not possible to replicate as there is no support in the tool for sub-fields in free-text columns currently.

Table 6.3: FMEA representation in the tool

Feature	Score	Comment
Worksheet structure	3	Fully supported via ES-columns
Parameters (S, O, D)	3	Fully supported
Formula (RPN)	3	Fully supported
Lookup table	3	Fully supported
Free-text columns	1	Sub-columns not supported
Overall	**2.6**	

What-If Analysis

The What-If Analysis was implemented on a single evaluation sheet with two parameters (Likelihood and Consequence) on a 1-5 scale and free-text columns for the What-If scenario, Answer and Recommendations, reflecting the standard What-If worksheet structure [58]. In this method no expression or formula is used. This can be seen in Table 6.4

Table 6.4: What-If Analysis representation in the tool

Feature	Score	Comment
Worksheet structure	3	Fully supported via ES-columns
Parameters	3	Fully supported (Likelihood, Consequence)
Formula	3	Not part of the method, qualitative only
Free-text columns	3	Fully supported (Answer, Recommendations)
Overall	**3.0**	

RAM at the P.H.A of Canada

The framework for RAM at the public health Agency of Canada is somewhat more complex in structure as it requires both Likelihood and Impact as Dimensions which is analyzed by both estimates of Likelihood and Impact as well as estimates of Uncertainty of the set values. The tool was however, sufficient to represent both the sub-dimensions as well its parameters. The values of each Dimension is then used in a Lookup tables that scales the risk. This can be seen in Table 6.5

hTMM

The hTMM framework was implemented in a single ES with the root *dimension hTMM* and three *parameters*: *Action*, *Purpose* and *Result*. Furthermore three *comment columns* were added

Table 6.5: RAM at the public health Agency of Canada representation in the tool.

Feature	Score	Comment
Worksheet structure	3	Fully supported
Parameters	3	Fully supported
Formula	3	Fully Supported
Lookup table	3	Fully Supported
Comment columns	3	Fully Supported
Overall	**3**	

which contained selectable categories of *Actor*, *Target*, and *Type of risk*. The *parameters* are then used by a lookup table which results in the Impact column seen in Figure A.15. Overall this worksheet was fully implemented without great difficulty and is very similar to the worksheet represented in the paper by Mead et al. [60].

Table 6.6: HTMM representation in the tool.

Feature	Score	Comment
Worksheet structure	3	Fully supported
Parameters	3	Fully supported
Formula	3	Fully Supported
Comment columns	3	Fully Supported
Overall	**3**	

OCTAVE Allegro

Octave Allegro was implemented with three evaluation sheets, that corresponds to the three main phases. The first sheet, represents the Threat Scenario identification, including Actor, Means, Motive and Outcome as free-text columns alongside a Probability parameter. The second sheet, represents the asset and identification stage. The third sheet, implements the Risk Analysis phase with five impact area parameters (Reputation, Financial, Productivity, Safety & Health, Fines & Legal) and a formula computing the relative risk score. The main limitation we encountered was the lack of support for constants in the expression. Since in OCTAVE Allegro, each of the parameters are assigned a user defined weight (rank) that is then multiplied by the impact. As the tool does not support this, an unweighted sum was used. This represents a deviation from the original method and is therefore identified as a candidate for future improvement. This can be seen in Table 6.7

FAIR

FAIR was implemented in the tool as a simplified representation consisting of two ES: one representing the *Multiple Scenario Table* and the other representing the *Loss Table*. The *Multiple Scenario Table* included the *parameters*: Threat Event Frequency and Threat Capability, while the *Loss Table* included the *parameters Confidentiality*, *Availability*, and *"Possession of $"*. To represent scenarios, several multiple-choice *comment columns* were introduced. For the *Multiple Scenario Table*, the columns *Asset*, *Threat Community*, *Threat Type*, and *Effect* were created, while the *Loss Table* included the different loss forms.

Table 6.7: OCTAVE Allegro representation in the tool

Feature	Score	Comment
Worksheet structure	3	Fully supported via ES-columns
Parameters	3	Fully supported (5 impact areas)
Formula	2	Supported but without dynamic weights
Dynamic weights	1	Constants not supported in expressions
Overall	**2.25**	

In the FAIR methodology, collected empirical data can be analyzed using Monte Carlo simulations to support quantitative risk assessment [62]. This functionality was not implemented in the current version of the tool, as it would require significant additional development. Consequently, the implementation should be viewed as a simplified representation of FAIR rather than a full implementation of the methodology. Supporting fully quantitative risk assessment methods could therefore be considered future work. This can be seen in Table 6.8

Table 6.8: FAIR representation in the tool

Feature	Score	Comment
Worksheet structure	3	Fully supported via ES-columns
Parameters	3	Fully supported
Comment columns	3	Fully supported
Post analysis calculation engine	1	Not supported
Overall	**2.5**	

Summary of Method Representation

The selected risk assessment methods were implemented and evaluated to determine their suitability for replication within the developed tool. The methods were intentionally chosen to represent a diverse set of approaches, spanning both functional safety and cybersecurity domains, as well as qualitative and quantitative methodologies. This diversity enabled a comprehensive assessment of the tools flexibility and general applicability as seen in Req. 1, Req. 2 and Req. 3 defined in Chapter 5.

Overall, the results indicate that the tool successfully supports the core structural elements common to most risk assessment methods, such as worksheet-based organization, parameter definition and basic computation mechanisms. Methods with relatively straightforward structures, such as What-If analysis, hTMM, and the RAM used by P.H.A of Canada, achieved full or near-full implementation, demonstrating the tool's strength in handling qualitative and semi-quantitative approaches.

In contrast, more complex methodologies revealed certain limitations. For instance, FMEA was largely supported, including Parameter handling and formula computation, but lacked flexibility in structuring free-text sub-columns. OCTAVE Allegro exposed a more significant limitation due to the absence of support for dynamic weighting in expressions, resulting in a simplified and slightly deviating implementation of the original method. Similarly, FAIR could be structurally represented; however, its reliance on advanced stochastic analy-

sis (i.e, Monte Carlo simulation) could not be replicated, highlighting a gap in post-analysis computational capabilities.

Despite these limitations, the tool demonstrated a high degree of methodological agnosticism, as it was capable of representing all evaluated methods to a meaningful extent without requiring method-specific customization. The consistent support for core features, such as parameters, lookup tables and comment fields, suggest that the tool provides a solid foundation for generalized risk assessment workflows.

The comparative scoring further reinforces these findings, with most methods achieving overall scores between 2.5 and 3, indicating good to full support. The primarily areas for improvement include advanced computational features, support for weighted expressions and more flexible data structuring options.

In summary, the evaluation demonstrates that the tool can represent a broad set of risk assessment methods, particularly those based on structured qualitative or semi-quantitative approaches. More advanced quantitative methods such as FAIR (Factor Analysis of Information Risk) revealed limitations and may require further development. Therefore, the tool can be considered method-agnostic within the scope of structured qualitative and semi-quantitative methods. As summarized in Table 6.9, Req. 1 is fulfilled since the tool was able to represent at least two different assessment methods and at least one multi stage-calculation. Furthermore, Req. 2 is fulfilled since for all tested methods, scoring parameters could be modified without code changes and the system produced method-consistent outputs based on configuration changes for nearly all methods. Moreover, Req. 3 is fulfilled as we could fully implement at least three methods without modifying the core data model and the integration of all methods only required configuration in the tool. Finally, Req. 4 is fulfilled as all final results can be decomposed into their intermediate steps and linked to the original input values as well as replicated.

Table 6.9: Summary of Method Representation Scores

Feature	FMEA	What-If	OCTAVE	P.H.A of Canada	hTMM	FAIR
Worksheet structure	3	3	3	3	3	3
Parameters	3	3	3	3	3	3
Formula/Expression	3	3	2	3	3	N/A
Lookup table	3	N/A	N/A	3	N/A	N/A
Free-text columns	1	3	N/A	3	3	3
Dynamic weights	N/A	N/A	1	N/A	N/A	N/A
Post-analysis calculation engine	N/A	N/A	N/A	N/A	N/A	1
Overall	**2.6**	**3.0**	**2.25**	**3**	**3.0**	**2.5**

6.5 Multi-User and Concurrent Assessment Support

6.5.1 Test of Multiple Users

The multiple-user functionality of the tool was evaluated using two separate user accounts, referred to as User A and User B. In the evaluation scenario, User A invited User B to a shared project within the system. Both users were then able to access the same project environment simultaneously.

To evaluate shared workflow behavior, User A performed modifications to the project data and saved the changes to the database. User B was subsequently able to observe the updated project state after the polling fetched the shared assessment data. This demonstrated that multiple users could access and interact with the same project structure through shared persistence in the backend database.

The evaluation was exploratory in nature and focused on verifying basic concurrent multi-user interaction and shared project access rather than advanced real-time synchronization or conflict resolution mechanisms. The evaluation workflow can be seen in Appendix A.5.

The evaluation provides support for Req. 5, concerning concurrent multi-user interaction, and partially supports Req. 7, related to collaborative risk assessment workflows.

6.5.2 Performance Evaluation

To validate the system's scalability, we conducted load testing with two different concurrency levels: 20 and 50 concurrent users, each over a 10-minute period using Locust [63] 6.10. The average response time is calculated as the mean of all response times across all requests during the test, giving an overall indication of system performance.

Test Results

Table 6.10: Performance comparison between 20 and 50 concurrent users.

Metric	**20 Users**	**50 Users**
Total Requests	5,964	14,862
Success Rate	83.7%	83.7%
Avg Response Time	3.65ms	3.27ms
P95 Response Time	5ms	5ms
Max Response Time	29.62ms	76.24ms

The results demonstrate that the system performance remains stable under increased load. The average response time varied only slightly from 3.65 ms to 3.27 ms as the concurrency increased from 20 to 50 users. Which is suggesting that no clear degradation was observed under the tested load. The P95 response time then remained consistent at 5ms across both the scenarios, this indicates that 95% of the user requests experience is consistent regardless of the load level. The P95 response time represents the threshold below which 95% of all requests are completed. This means that 5% of the requests took longer than this value, making it an useful indicator of worst-case performance. The increase in the maximum response time from 29.62ms to 76.24ms reflects the occasional spikes that come under a higher load, which is expected as the system handles simultaneous requests. However, the median and the P95 value indicates that these spikes are outliers affecting a small percentage of user requests.

The 83.7% success rate in both of the tests is primary due to the `/projects/get_project_rams/5` endpoint, that returned authorization errors. The errors are caused by invalid or missing authentication in the load testing setup rather than a limitation of the system itself. This endpoint alone accounted for 973 failed requests of the 991 total request failures (98.2%). This indicates that the failures are isolated to a single endpoint. Since these failures were caused by invalid or missing authentication data rather than server overload, the raw success rate should not be interpreted as a general system failure under load. These results are shown in Figure 6.1. Here, P50, P95, and P99 denote the 50th, 95th, and 99th response time percentiles, respectively. For example, P95 indicates that 95% of all requests completed within the reported response time, while only 5% took longer.

When evaluated in relationship with the requirements defined in Chapter 4, the results then demonstrate that Req. 5, which specifies the need for support for concurrent multi-user interaction, is satisfied. Since the system maintains consistent performance under increased load. Req. 6, that concerns the system responsiveness is also fulfilled since both the average and P95 response times remain low and stable across the test scenarios. Req. 7, related to

the supporting collaborative and concurrent risk assessment workflows is supported by the systems ability to handle multiple risks requests without degradation in performance.

Figure 6.1 presents the measured response time percentiles (P50, P95, and P99) for selected API endpoints under 20 and 50 concurrent users. The evaluated endpoints include risk evaluation updates (POST), retrieval of evaluation sheets (GET), and retrieval of evaluation sheet column definitions (GET).

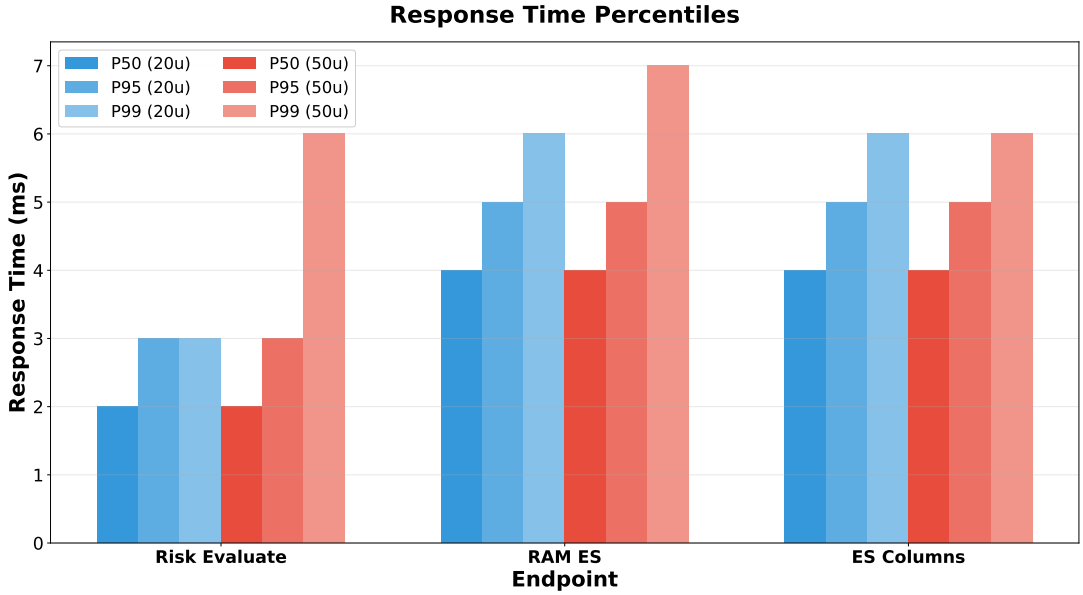


Figure 6.1: Response time percentiles (P50, P95, P99) for selected API requests under 20 and 50 concurrent users.

6.6 LLM Risk Suggestion Reliability

This section evaluates the reliability of the LLM component in the tool. The evaluation focuses on the completeness and quality of input data influence and the risk suggestions that were generated. In line with a human-in-the-loop approach described in the system design, the LLM-generated risks are considered a support for decision making, rather than authoritative outputs. Three experimental scenarios were designed to assess the performance of the LLM under different levels of input. Firstly, a complete risk sheet. Secondly, a partially completed risk sheet. And finally, the LLM is given only a system description with no predefined risks. The evaluation in this section aims to analyze the relevance and consistency of the suggested LLM risks in each scenario. The same system description is given in all three scenarios. The scenario used was based on the ETSI Threat, Vulnerability and Risk Analysis (TVRA) for Intelligent Transport Systems (ITS), as described in ETSI TR 102 893 [64]. The recreation of one of the threat tables can be seen in Figure 6.2.

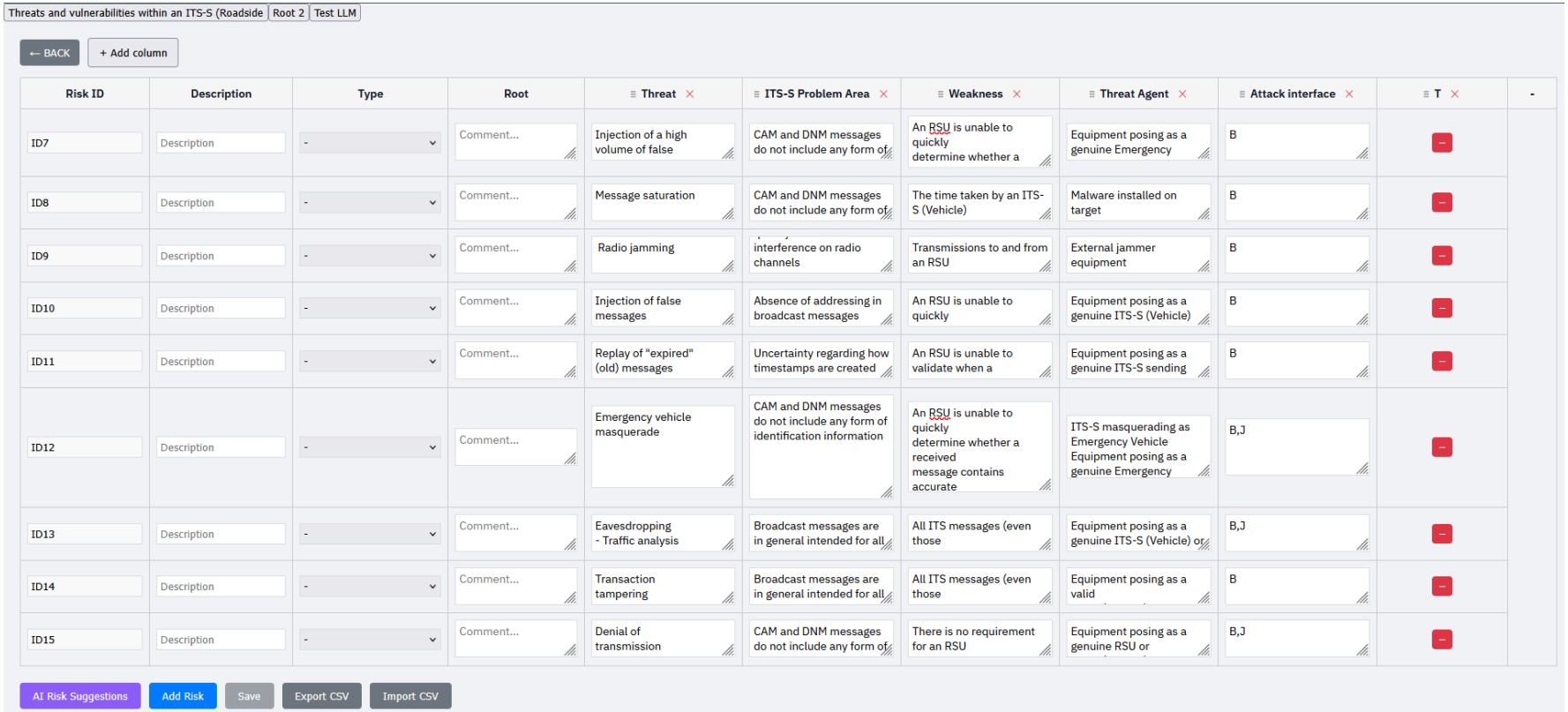


Figure 6.2: Reference risk assessment sheet extracted from external source [64]

6.6.1 System description

The following system description was used as input to the LLM in all experimental scenarios:

```
SYSTEM: Vehicle-to-Vehicle Safety Communication System (V2V)

COMPONENTS:
- Wireless 5.9 GHz communication module
- Message processor (receives CAM, DENM messages at 10 Hz)
- Local sensor data (GNSS position, vehicle speed/direction)
- Cooperative awareness and hazard warning services
- Multi-hop broadcast relay functionality

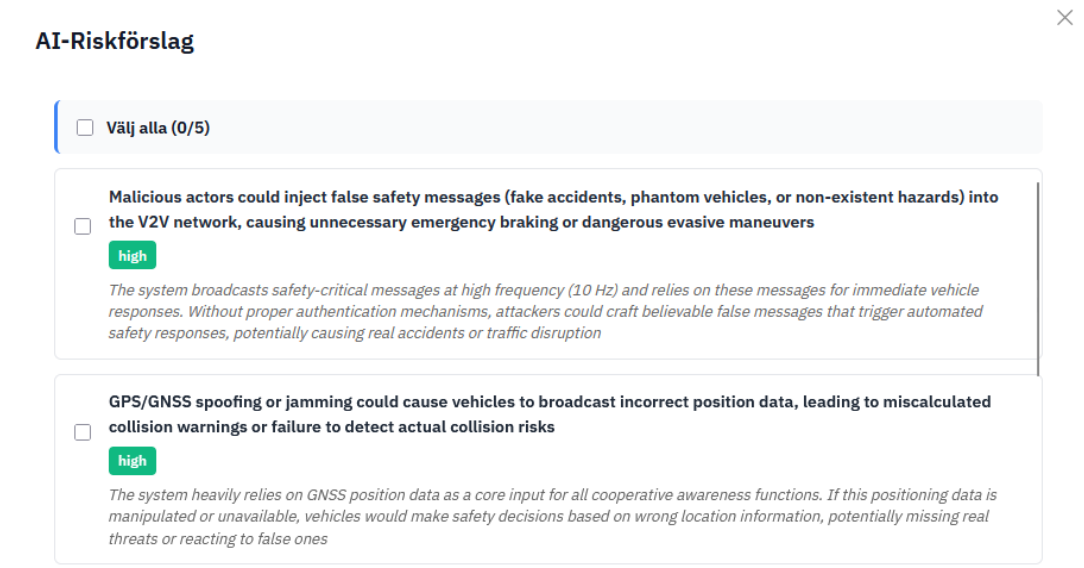
DATA FLOWS:
- Vehicle sends beaconing messages with position and status to nearby
  vehicles
- Vehicles receive safety alerts about obstacles, accidents, and hazards
- Safety-critical messages relayed through multiple hops
- Service profile determines which applications are active
```

6.6.2 Scenario 1: Complete Risk Sheet

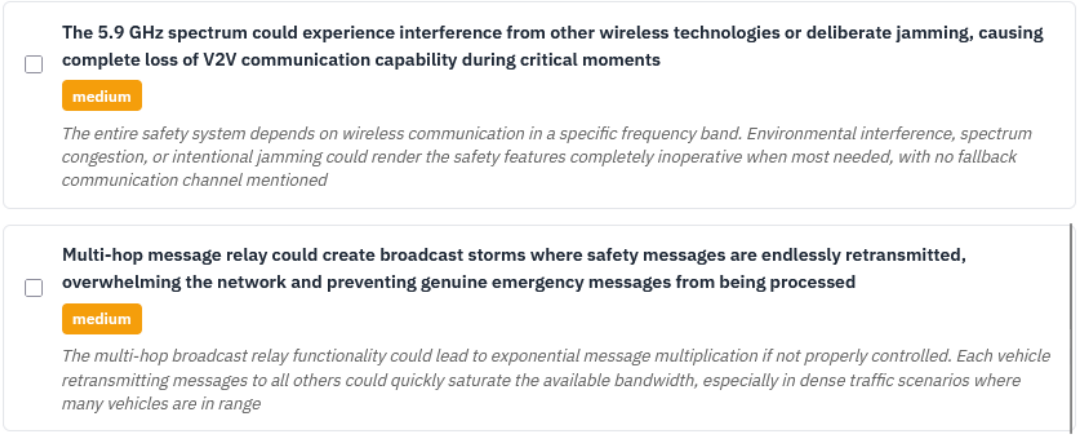
In the first scenario, the LLM was given a completed risk sheet, including threats, associated parameters and identified assets. The objective was to then evaluate if the model could contribute additional support and insights to an already complete analysis.

The results shown in Figure 6.3a display the first four risks generated from the first batch of the LLM generated risks. This pair demonstrates that the model is able to identify both communication level and data integrity related threats. In particular, the LLM highlights vulnerabilities related to the wireless communication reliability and the potential manipulation of safety-critical data. While the generated risks are relevant, several overlap conceptually with the existing predefined risks or represent variations of already known threat categories. This might indicate that, in a fully populated risk context the LLM primarily contributes by reinforcing and refining the existing risk perspectives rather than introduce entirely new risks to an already "completed" evaluation sheet.

In the second batch, shown in Figure 6.3b, the previously generated risks from Figure 6.3a were included as additional context for the LLM. The model continued to produce relevant risk suggestions. However, the results from Figure 6.3b indicates some saturation, due to an increase of similarity to the earlier outputs. With a small tendency of rephrasing or extending previously identified risk. An example of this behavior can be observed in Figure 6.3a, where risks such as "False Safety Message Injection" and "GNSS spoofing Leading to incorrect Position Data" are generated. While these represent different attack vectors, they both relate to the manipulation of safety critical data, which many of the human identified risks were centered around. This saturation effect suggests that the model becomes increasingly constrained by the existing context, reducing its ability to explore new risk scenarios.

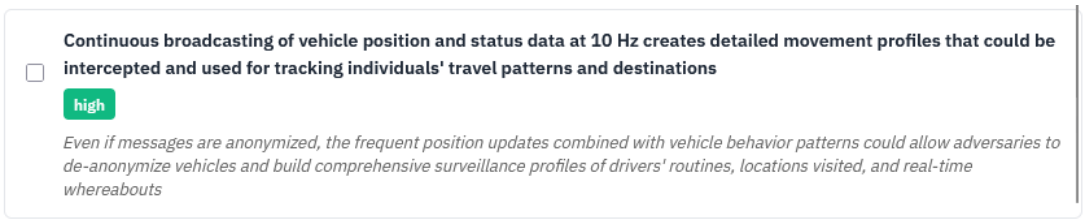


(a) Risk suggestions generated in the first LLM run

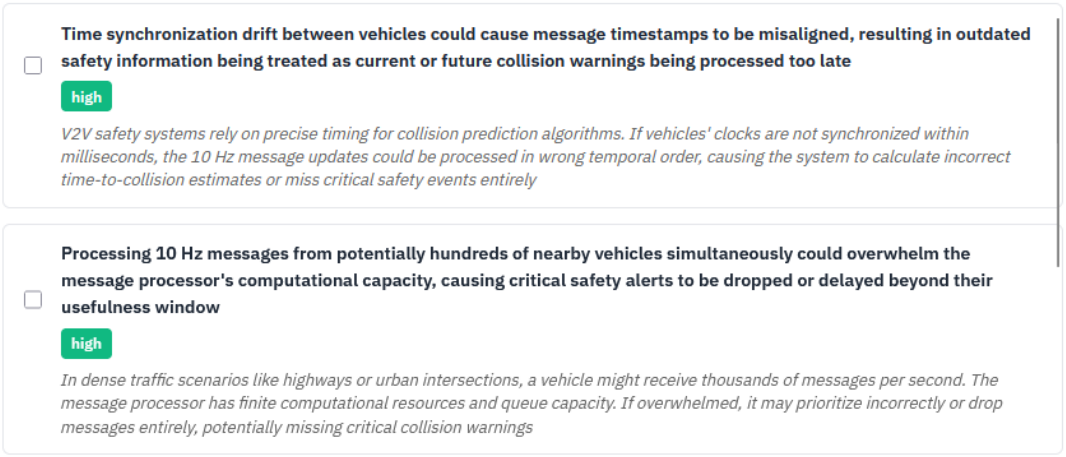


(b) Risk suggestions generated in the first LLM run

Figure 6.3: First sequential LLM evaluation run using previously accepted AI-generated risks as additional context.

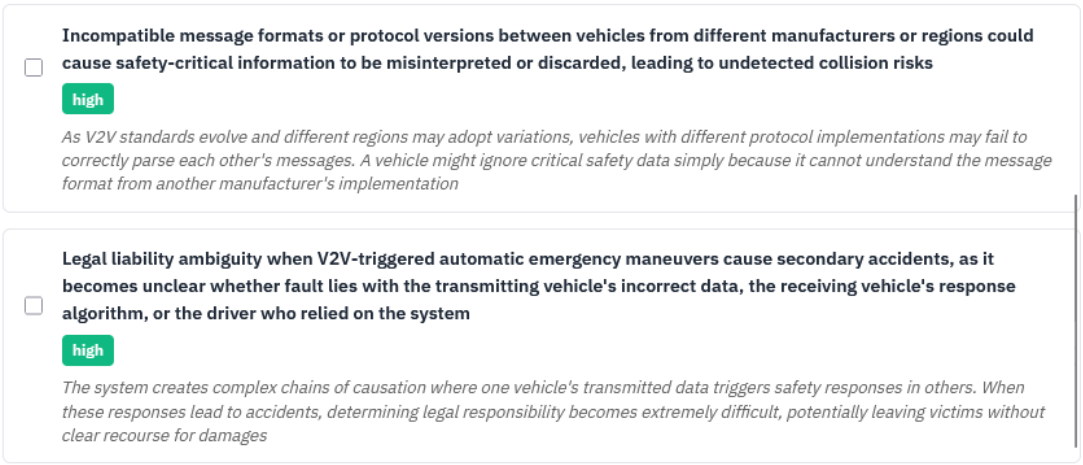


(a) First sequential LLM evaluation run.

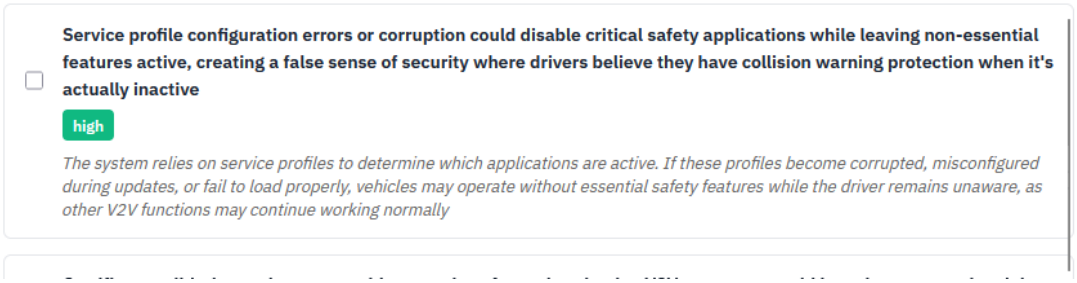


(b) Second sequential LLM evaluation run.

Figure 6.4: Second sequential LLM evaluation run using previously accepted AI-generated risks as additional context.



(a) Input including accepted risks from the first and second LLM runs



(b) Risk suggestions generated in the third LLM run

Figure 6.5: Second sequential LLM evaluation run using previously accepted AI-generated risks as additional context..

6.6.3 Scenario 2: Partial Risk Sheet

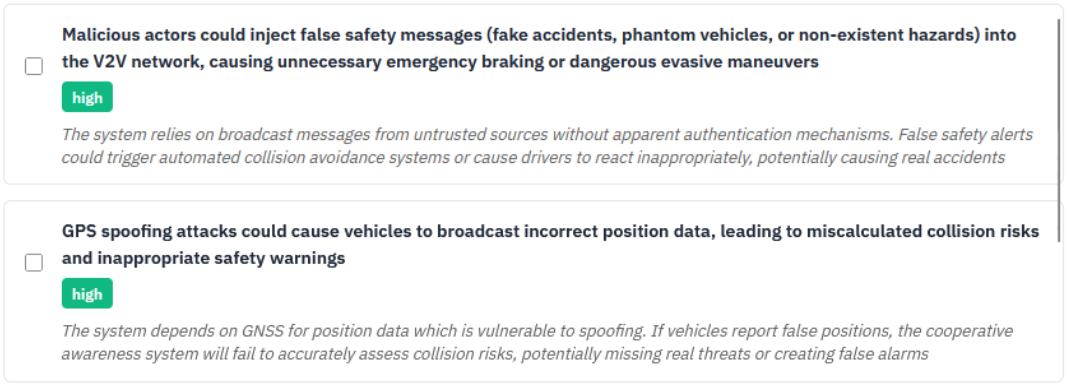
To evaluate the LLM's ability to identify relevant risks, the model was provided with a system description along with a partial set of human identified risks (the first five from the expert-defined list). This setup was intended to simulate a realistic scenario in which only incomplete prior knowledge is available. The selected risks primarily covered environmental factors, regulatory uncertainty, cryptographic key compromise, software reliability issues, and radio interference within certain spectrum (in this case 5.9 GHz).

When comparing the LLM's output to the remaining, not given, expert-defined risks, a substantial overlap can be observed. The model successfully identified several key threats that were not included in the initial prompt but are present in the ground truth. In particular, it generated risks corresponding to false message injection and impersonation, seen in Figure 6.6a (ground truth: injection of false messages and emergency vehicle masquerading), GPS spoofing, Figure 6.6a (ground truth: incorrect position data leading to miscalculated risks), time synchronization issues, and Figure 6.7a (ground truth: eavesdropping and traffic analysis). Additionally, the LLM identified system-level degradation scenarios such as message processing overload which can be observed in Figure 6.8a, which aligns with expert-identified risks concerning message saturation and denial of transmission.

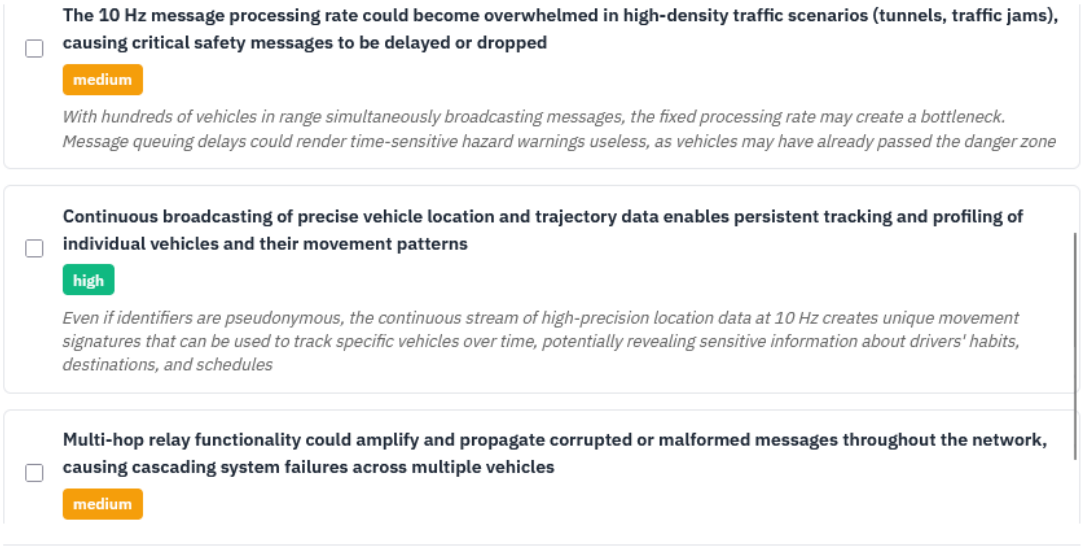
However, the comparison also reveals some discrepancies. Certain expert-defined risks were only partially captured or described at a higher level of abstraction. For example, the LLM did not explicitly distinguish between different communication-layer attacks such as radio jamming versus general interference, nor did it clearly separate transaction tampering from broader message integrity issues. Furthermore, while the LLM identified imperson-

ation and false data injection, it tended to merge multiple distinct attack vectors into broader categories, reducing granularity compared to the expert taxonomy.

Overall, the LLM demonstrated a strong ability to generalize from the provided subset of risks and recover a majority of the remaining ground truth categories. Nevertheless, its outputs exhibit a tendency toward abstraction and consolidation, indicating that while it is effective at identifying broad threat classes, it may require expert refinement to achieve the same level of specificity and completeness as manually curated risk assessment.

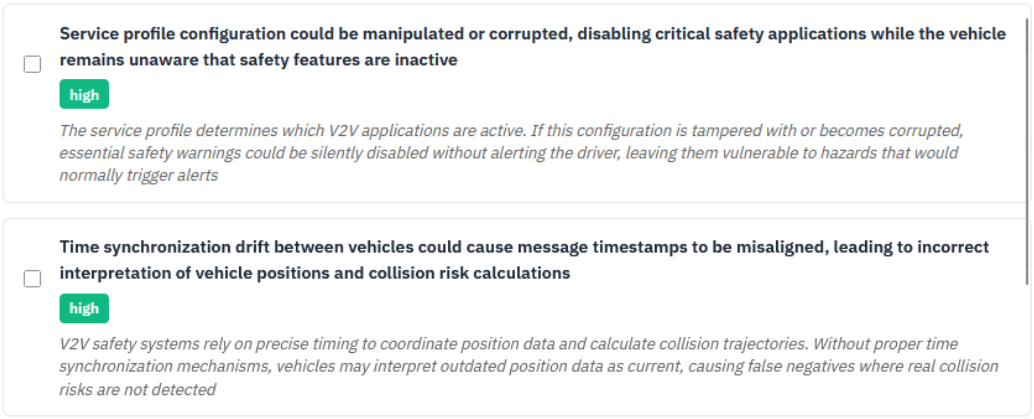


(a) Risk suggestions generated in the first LLM run.

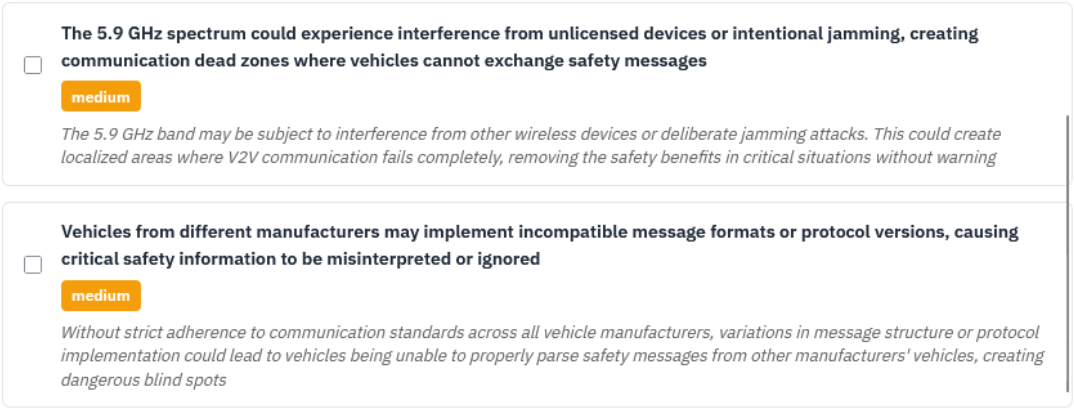


(b) Risk suggestions generated in the first LLM run.

Figure 6.6: First iteration of LLM-generated risk suggestions for a risk assessment sheet with a subset of risks.

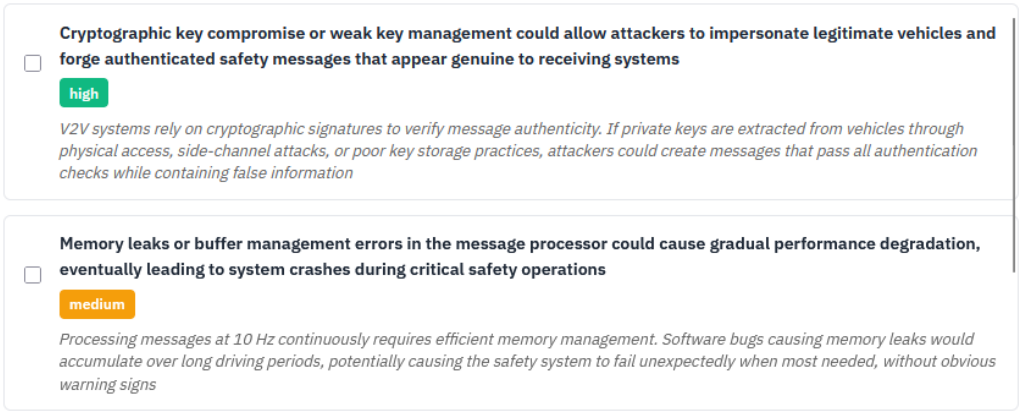


(a) Risk suggestions generated in the second LLM run.

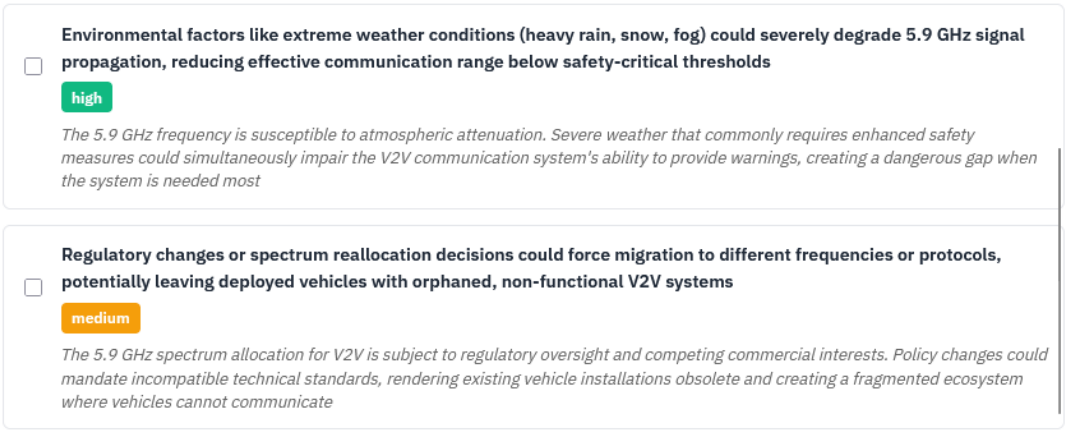


(b) Risk suggestions generated in the second LLM run.

Figure 6.7: Second iteration of LLM-generated risk suggestions for a risk assessment sheet with a subset of risks.



(a) Risk suggestions generated in the third LLM run.



(b) Risk suggestions generated in the third LLM run.

Figure 6.8: Third iteration of LLM-generated risk suggestions for a risk assessment sheet with a subset of risks.

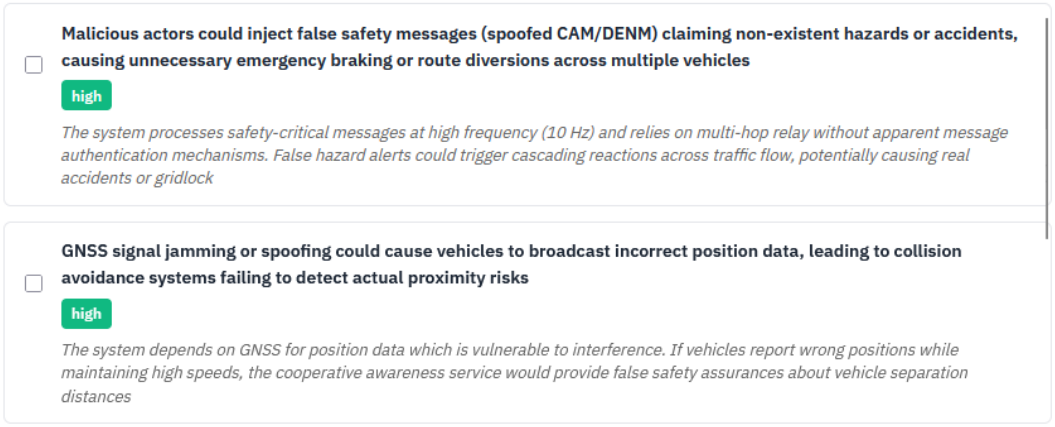
6.6.4 Scenario 3: System Description Only

In this last scenario, the LLM was only provided with the system description, seen in Section 6.6.1. The aim and objective here was to evaluate the models ability to identify risks based solely on the architecture, components and data flow.

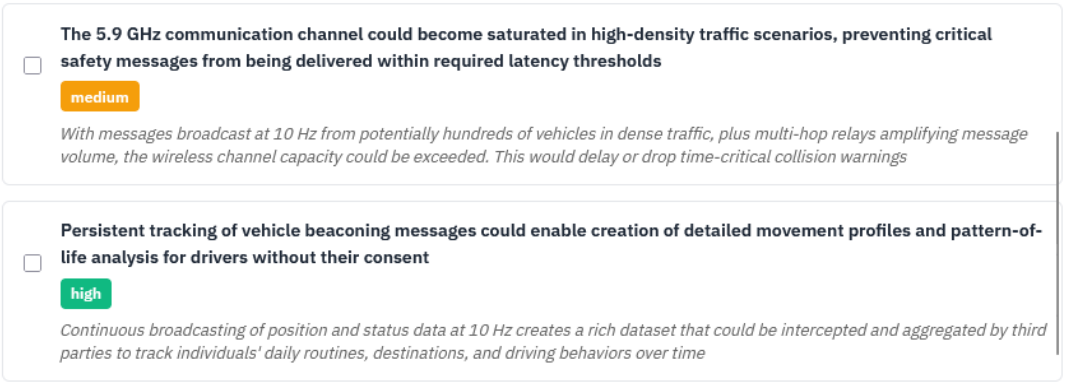
The results show that the LLM is capable of generating diverse risks that span several categories. These risks identified by the LLM includes communication failures, data integrity and performance limitation. The LLM identified risks related to environmental interference affecting the communication aspect, as shown in Figure 6.9a. The risks identified using nothing but the system description demonstrates that the LLM is able to reason about the limitations and threats to the system without relying on previously human made defined risks. Furthermore, the LLM also introduced risks that were very similar to the ones created by a human. For example, in Figure 6.10c "Malicious actors could inject false safety messages (spoofed CAM/DENM)" aligns with expert-defined risks related to false message injection.

Another similar example is shown in Figure 6.9b, which resembles previously identified risks concerning delayed or missing handling of safety messages [64]. However, the LLM also created risks that are quite broad for example "Clock synchronization errors between vehicles...", seen in Figure 6.10a which describes system-level inconsistencies rather than threats. These types of risks highlight interactions between system components rather than clearly defined threats. Overall, the results suggest that the LLM is capable of both rediscovering core human defined threats and introducing additional system level threats. This demon-

strates that the LLM component in the tool can function as a useful supplement for early stage risk identification, although some of the generated risks do require further refinement.

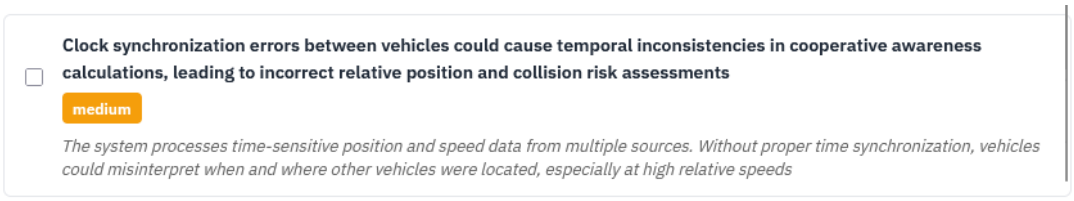


(a) Initial empty risk assessment sheet provided to the LLM.

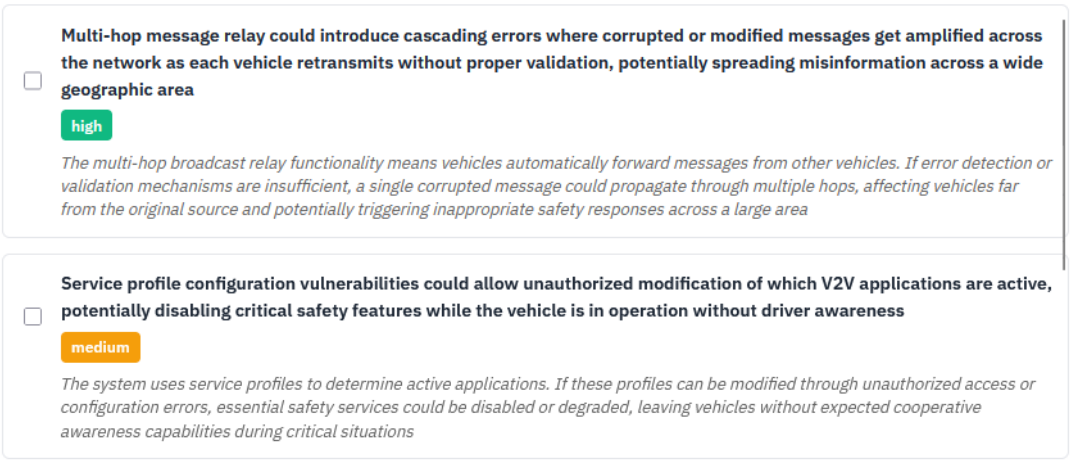


(b) Initial empty risk assessment sheet provided to the LLM.

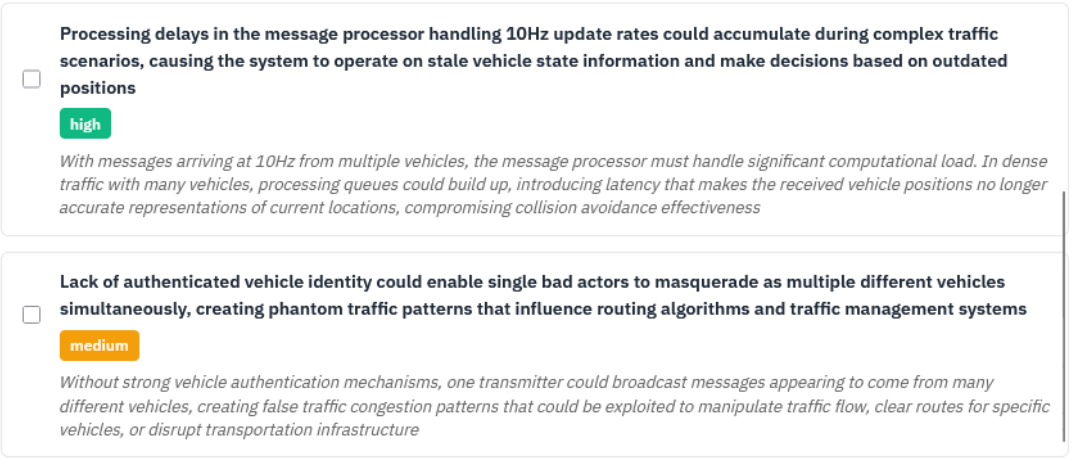
Figure 6.9: Summary of initial empty risk assessment.



(a) First iteration of LLM-generated risk suggestions.

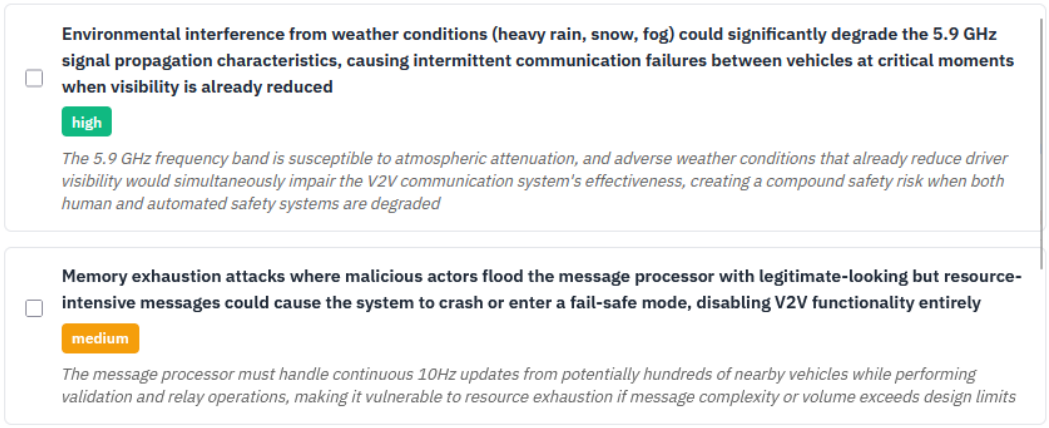


(b) First iteration of LLM-generated risk suggestions.

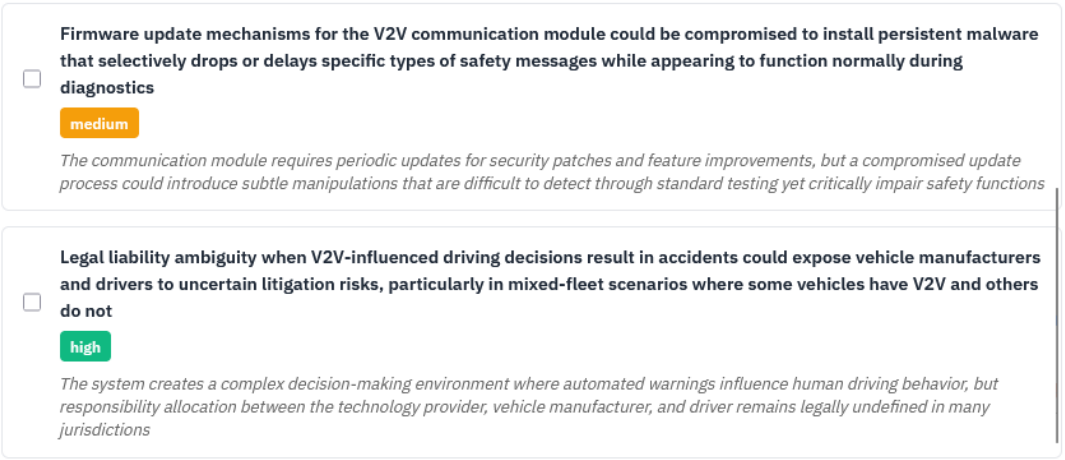


(c) Second iteration of LLM-generated risk suggestions using the previously accepted risks as additional context.

Figure 6.10: Summary of iterative LLM-generated risk suggestion.



(a) Second iteration of LLM-generated risk suggestions using the previously accepted risks as additional context.



(b) Second iteration of LLM-generated risk suggestions using the previously accepted risks as additional context.

Figure 6.11: LLM-generated risk suggestions based on system description alone.

6.6.5 Summary of the LLM risk suggestions

Overall, the reliability of the risks suggested by the LLM is strongly influenced by the amount of contextual information provided. When compared to the expert-defined ground truth, see Figure 6.2, the model performs best in partially completed scenarios, where it can infer missing risks and extend the analysis beyond the provided subset.

Across the evaluated scenarios, the LLM demonstrated the ability to identify several risks present in the ground truth. In the scenario where a subset of expert-identified risks was provided, the model recovered a majority of the remaining ground truth risks partially fulfilling Req. 7. and Req. 8. However, these were often expressed at a higher level of abstraction and lacked the specificity observed in the expert-defined entries. For example, distinct risks such as message injection, replay attacks, and impersonation were frequently grouped into broader categories such as "false data injection" or "communication integrity issues."

Similarly, in the scenario where no prior risks were provided, the LLM was still able to identify a substantial portion of the ground truth risks. However, the outputs in this case were even more generalized and less aligned with the structure and granularity of the expert taxonomy. While key themes, such as interference, spoofing, system overload, and privacy concerns, were correctly identified, the model did not consistently distinguish between

closely related but technically different risks (e.g., radio jamming versus spectrum interference, or transaction tampering versus general message manipulation).

A consistent pattern observed across all scenarios was the LLM's tendency to generate risks that closely resemble existing ones, either through rephrasing or by presenting them in a slightly altered context. This behavior was particularly evident when the model was provided with a more complete set of risks, where many of its suggestions overlapped with or redundantly extended the existing entries rather than introducing entirely new categories.

When provided with a fully completed expert analysis, the role of the LLM becomes primarily confirmatory, offering limited additional value beyond restating or slightly expanding known risks. In contrast, when only the system description is provided, the model produces broader and more abstract suggestions that capture general threat patterns but lack precision.

These findings highlight that while the LLM is effective at identifying and generalizing risk concepts, its output require expert validation to ensure accuracy, completeness, and appropriate granularity. This supports the integration of the LLM within a human-in-the-loop workflow, where domain experts refine and structure the generated risks to align with established security analysis practices. Finally, we consider Req. 10 to be successful since all of the outputs of the LLM conformed to the set JSON schema the modal interface.

6.7 Interview Results

As part of the evaluation process, semi-structured interviews were also conducted with domain experts to evaluate the tool from a more practical perspective. The focus of the interviews was to understand current workflows and evaluate whether the tool aligns with real-world use cases. The responses from the two experts, seen in Tables A.1, A.2 and A.3, show several recurring themes. Both experts described current HARA/TARA workflows as dependent on Excel-based tools, which can become difficult to manage as the size and complexity increases. Regarding the proof-of-concept tool, the two experts expressed positive impressions, highlighting the structured approach and the potential advantages over some of the existing solutions. The method-agnostic design was perceived as clear, and the idea of combining several assessment methods within a single framework was regarded as beneficial. At the same time, both of the experts mentioned that the tool still have a lot of manual input and labor, suggesting further automation and LLM-functionality could improve the usability. This indicates that, the support for collaboration and interoperability is sufficient in the tool in relation to RQ2 but additional features could further improve the workflow in the tool. Finally, both experts mentioned quality of life for the user as an area for improving suggesting clarity of flows, filtering of risk and, additional AI-functionality to assist in this.

6.8 Summary of Findings

The evaluation demonstrated that the proposed artifact was capable of representing multiple risk assessment methods within a common architectural framework. Methods including SAHARA, HEAVENS, FMEA, FAIR, HazOp, and OCTAVE Allegro could be represented without modifications to the underlying data model, indicating a high degree of flexibility and extensibility.

The evaluation also showed that the system supported concurrent risk assessment activities through a shared data model and collaborative functionality. In addition, several supporting features were implemented to facilitate assessment workflows and interoperability between stakeholders.

Finally, the LLM-based functionality showed to be a potential support for threat identification. While the generated suggestions were often useful, their quality varied depending on the available context, reinforcing the importance of human review and validation.

7 Discussion

The following chapter discusses the results and methodology of this thesis in relation to the research questions and the overall aim of the thesis. The findings from the design, implementation and evaluation of the proposed tool are analyzed with particular focus on method-agnostic representation, collaborative risk assessment, and the LLM-based support. Furthermore, the chapter reflects on the strengths and limitations of the selected research approach, the practical applicability of the artifact within real-world HARA/TARA workflows, and relevant ethical and societal considerations.

7.1 Results

The main challenge in this thesis is the limited availability of existing research on method agnostic risk assessment tools. While there exists literature on individual methodologies or specific tools such as Microsoft Threat Modeling Tool, there is comparatively little work addressing how multiple methods can be supported within a unified, methodology-independent framework. This lack of established evaluation benchmarks made it quite difficult to directly compare the results of this thesis to prior work. As a result the evaluation is therefore primary based on qualitative analysis and structural comparisons rather than standardized benchmark metrics. Therefore, it might affect the validity of the conclusions as the assessment relies on interpretation rather than a widely accepted evaluation criteria. The evaluation demonstrates the feasibility of the proposed architecture and provides evidence that the system can represent several selected risk assessment methods. However, due to the limited set of evaluated methods, the results cannot establish general support for all possible TARA/HARA approaches.

At the same time this limitation highlights the importance of the work, as it explores an area that is not yet well defined in existing literature. However, it also implies that further research is needed to establish a clearer evaluation framework for method agnostic risk assessment tools.

7.1.1 RQ1: Method-Agnostic Representation

The first RQ concerns the extent to which HARA/TARA-related concepts can be represented within a method-agnostic software architecture. The evaluation indicates that the proposed

tool can represent several structurally different risk assessment methods by abstracting common elements such as evaluation sheets, dimensions, parameters, parameter values, expressions, lookup tables, and final classifications.

The selection of evaluated risk assessment methods was not only intended to provide methodological diversity, but also to stress-test the flexibility of the developed artifact across fundamentally different ways of thinking in risk analysis. By including methods from both safety (e.g. FMEA, RAM from Public Health Agency of Canada) and cybersecurity domains (e.g. FAIR, OCTAVE Allegro), the evaluation captured differences in structure, data requirements and computational complexity.

Furthermore, the inclusion of both qualitative approaches (e.g., What-If Analysis) and quantitative or semi-quantitative methods (e.g., FMEA, FAIR) allows for more nuanced assessment of how well the tool supports varying levels of formalization in risk modeling.

The implementation of FMEA demonstrates that the tool is well suited for structured, semi-quantitative risk assessment methods that rely on relatively simple mathematical formulations. The full support for *parameters* and formula computation indicates that the tool can accurately replicate core analytical mechanics such as the Risk Priority Number calculation. However, the limitation in handling more complex free-text structures suggest that while numerical evaluation is well supported, the tool may be less effective in capturing the context required in industrial FMEA applications. This could impact usability in real-world scenarios where detailed documentation and hierarchical descriptions are essential.

The What-If analysis was represented without major limitations, highlighting the tools strength in supporting qualitative and workshop oriented risk assessment methods. Since the method relies primarily on structured questioning and expert judgment rather than formal computation, it aligns well with the tool's flexible worksheet structure. This result suggest that the tool is particularly effective in early-stage risk identification processes, where flexibility and ease of use are more critical. It also indicates potential for collaborative use cases, such as brainstorming and hazard identification workshops.

In contrast, the partial implementation of OCTAVE Allegro highlights an important limitation of the tool in terms of its inability to support weighted expressions. While the structural components of the method were successfully replicated, the absence of dynamic weighting alters the intended risk evaluation logic. This deviation illustrates a broader challenge; while the tool can represent the structure of complex methodologies, it may struggle to preserve their analytical fidelity when advanced parameter relationships are required. Consequently, results produced using such simplified implementations should be interpreted with caution.

The FAIR implementation further emphasizes the limitations of the tool in handling advanced quantitative analysis. While the structural components of the methodology could be represented, the absence of support for stochastic modeling significantly restricts its analytical depth. This suggest that the tool is currently better positioned as a modeling and structuring platform rather than full quantitative risk analysis engine. For organizations relying on data-driven risk estimation, additional external tools or integrations would likely be required.

The implementation of hTMM demonstrates that hybrid and moderately structured cybersecurity methods can be effectively supported within the tool. The combination of parameters and categorical inputs aligns well with the tool's capabilities, allowing both structured evaluation and contextual classification. This suggests that the tool is particularly well-suited for modern threat modeling approaches that balance structure with flexibility, especially in domains where strict quantitative models are less common.

Similar to the hTMM, the implementation of threat assessment framework of the Public Health Agency of Canada shows that the tool can handle multi-dimensional risk models, including those incorporating uncertainty as an explicit factor. The use of lookup tables to scale risk further demonstrates the tool's ability to support layered evaluation logic. This is an important result as it shows that the tool is not limited to simple models but can also

represent more sophisticated conceptual frameworks provided they do not rely on advanced computational mechanisms.

Cross-method results

When comparing the evaluated methods, a clear pattern emerges: the tool performs best for methods that rely on structured inputs, simple parameter relationships and limited computational complexity. In contrast, methods requiring dynamic weighting, probabilistic modeling or advanced post-processing expose limitations in the current implementation.

The identified limitations suggest several directions for future development, including support for weighted expressions, enhanced data structuring capabilities and integration of advanced computational engines. Addressing these areas would significantly expand the range of methods and frameworks that can be fully and accurately implemented.

7.1.2 RQ2: Collaboration, Interoperability, and Supporting Features

The second RQ concerns how the artifact supports collaboration and interoperability among stakeholders involved in risk assessment. The implemented supporting features contribute to this goal by enabling shared projects, concurrent access, import and export, configurable lookup tables, rationale documentation, and structured data flow modeling. The support for concurrent multi-user use is important because risk assessment is rarely performed by a single person in isolation. This functionality allows multiple stakeholders to participate in the analysis process simultaneously, which in turn reflects real-world conditions where risk assessment are rarely performed in isolation and often involve different stakeholders.

In addition, supporting features such as import and export functionality, structured data flow representation, and user-defined lookup tables and lookup ranges improve transparency, flexibility, and adaptability, thereby facilitating collaboration and interoperability across different risk assessment methods. By enabling users to create custom mappings between qualitative values, the tool can be adapted to a wide range of risk assessment approaches rather than being limited to predefined models. These features enable users to tailor the tool to different methodologies and system contexts, whilst also facilitating reproducibility and consistency across assessments. However, it is important to note that it is not feasible for a single tool to fully support all possible risk assessment methodologies and use cases. While the implemented supporting features provide a high degree of flexibility, there will always exist some method specific requirement that have not been thought of during development. As a result, certain adaptions or manual configurations might still be required. This highlights an inherent trade-off between flexibility and usability.

While the ability to define custom parameters, lookup tables and evaluation structures enables an agnostic approach, it also increases the complexity of the tool. The complexity of the tool requires users to have certain level of expertise in risk assessment methodologies to efficiently use the features. Therefore, the supporting features demonstrate strong potential in enabling adaptable and collaborative workflows, but their effectiveness is dependent on how well users understand both the risk assessment method and the configuration logic. This suggests, additional features will be needed to the ones currently in the tool. As TARA and HARA methodologies evolves, the tool must also be updated to accommodate these future changes. This reinforces the importance of maintaining flexibility in the tool and system architecture, while still ensuring that the tool remains practical for real world experts and accessible for the users.

The Performance test

In addition to the functional aspects of the the supporting features, the performance evaluation in Section 6.4.1 provides important insights on how well the system supports concurrent usage. The results indicate that the system is relatively stable even as the number of users

increases from 20 to 50. This suggests that the chosen architecture is capable of supporting collaborative workflows without an degradation in performance. However, this should be interpreted cautiously. While the results are promising it is hard to know exactly how many simultaneous users the tool needs to support at the same time in a real world scenario, where safety and threat experts operate concurrently. There is also some scaling limitation, the connection pool size of 5 is appropriate for the test range but must be increased for a larger user base. In addition, some failed requests in the load test were related to authentication issues in the test setup. Therefore, the performance results provide initial support for the architectural design choices, but they do not constitute full validation of scalability or industrial readiness.

The results indicate that the tool provides basic technical support for collaboration and interoperability. However, collaboration is not only a technical issue. The current prototype demonstrates feasibility, but further user testing and real-world deployment studies are needed to evaluate collaborative effectiveness in practice.

7.1.3 RQ3: LLM-based Risk Suggestion Support

The integration of LLMs has been explored as a supporting feature for assisting users in the risk identification process. In this thesis, the LLM functionality is implemented using the Claude Opus 4.1 model provided by Anthropic, rather than developing and training a custom model. The decision to use this pre-trained model such as Claude Opus is motivated by several practical considerations. Training a domain specific LLM requires large amount of high quality data, and continuous maintenance which we in this thesis did not have adequate time for. Within the system the LLM is used to generate additional risks from a system description with the already identified risks. The model is guided using structured prompts that constrain the output format and explicitly instruct it to avoid generating risks, ensuring a more consistent and reliable results. Furthermore, the risks are not tied to a specific methodology, allowing the user to classify them according to their chosen risk assessment framework.

Despite the advantages of the risk additions that the component brings, the evaluation that was conducted highlights several important limitations. As observed in the evaluation presented in Chapter 6, the generated risk suggestions did vary depending on the level on input context, such as if a complete, partial or non-existing system description was provided. This allowed us to conclude that in some aspects, the quality and relevance of the generated risks are highly dependent on the available risks and information that already exists. Furthermore, the results show that the model can produce generic or very non context specific risks, this was particularity observed when the LLM component were only given the system description and no existing risks. This aligns with the challenges of LLM:s, such as AI hallucinations and lack of domain specific precision. In specialized systems this limitation becomes very clear as the model in this case fail to capture some of the domain specific threats that the human expert noticed. This behavior suggests that the role of the LLM is not to replace established risk assessment methods or practices, but rather to support the threat identification phase. By acting as a source of additional candidate threats, the LLM component can assist analysts in identifying overlooked threats and accelerating the early stages of the assessment process. The identified threats must still be evaluated using the selected risk assessment methodology. This support could be particularly valuable in large and complex systems, where the number of potential threats can be substantial.

The evaluation can also be related to the derived requirements from Section 4.4.3. For Req. 8, the results clearly show that the quality of the generated risks depend on the availability of the system description and the existing risks made by a human. For Req. 9, the LLM demonstrated their relevance varied depending on the context. Finally, Req. 10 was successfully fulfilled as the LLM component produced structured outputs. However, the reliability of confidence scores was not fully validated.

From the perspective of RQ3 (see Chapter 2), the results indicate that the LLM can generate useful and relevant risk suggestions, but the outcome is strongly dependent on the information given and the presence of human validation. Therefore, the integration of the component can be considered as successful in the terms of feasibility. Furthermore, these finding reinforce the importance of human-in-the-loop where domain experts still are responsible for much of the work such as selecting the generated risks, validating and refining. In a safety critical domain this is essential to ensure that the incorrect suggestions do not negatively impact the risk assessment.

Expert assessment and usability

The expert interviews provided valuable insights into the tools usability and relevance. The results indicate that current HARA/TARA workflows are still constrained by the limitations of Excel-based tools. In particular, the experts highlighted challenges related to navigation, scalability, and data management. This supports the motivation behind the developed artifact, which aims to provide a more integrated and structured environment for risk assessments.

The experts expressed a positive reception to the tools flexibility and structure, indicating that the method-agnostic approach is not only technically feasible but also practically relevant. The practicality is important as one of the main challenges with existing tools is the rigidity. However, the experts pointed out that much work still remains with the tool to get it industry ready. This highlights an important direction of the tool for future development, particularly in automation and workflow support. The mixed perception of the LLM functionality also further reinforces the need for a human-in-the-loop approach. While the LLM generated suggestions were seen as useful, the lack of a full trust indicates that such features should be designed as supportive rather than authoritative.

The expert feedback should also be interpreted cautiously because only two interviews were conducted. While the responses provide valuable qualitative insight, they cannot be treated as representative of all practitioners. A broader evaluation with more participants, different roles, and real risk assessment workshops would be required to draw stronger conclusions about practical usability and adoption potential.

7.2 Method

The thesis applied a Design Science Research (DSR) approach, focusing on the iterative design, implementation and evaluation of a software artifact. This approach was suitable because the aim of the thesis was to design and evaluate a tool that addresses a practical problem. The DSR approach allowed knowledge to be generated through the process of building and evaluating the artifact. However, the approach also introduces some limitations that should be considered when interpreting the results. The focus on creating on a working artifact came at the expense of deeper user testing and real world evaluation. For a tool intended to support practical assessment, more extensive testing with more diverse stakeholder could strengthen the evidence of the impact and utility. The prototype was evaluated primarily through technical validation and case studies rather than comprehensive user testing with domain experts. A more robust evaluation would have included formal usability testing with multiple expert reviewers and longitudinal studies assessing the tool's effectiveness in real-world risk assessment workflows. Therefore, the results should be understood as proof-of-concept evidence rather than full validation of an industrial tool.

7.3 Evaluation

7.3.1 Source Criticism

The literature used in this thesis primarily consists of academic papers, industry standards, and technical documentation related to risk assessment methodologies such as HARA, TARA, and threat modeling frameworks. While the sources provide a good theoretical foundation, some limitations should be acknowledged. Several of the referenced methodologies originate from domain specific applications or industry standards often within the automotive and cybersecurity sector. This may introduce a bias towards some risk perspectives that were not thought of. In addition, some of the sources related to the tools and practical implementations are based on documentation and vendor provided material rather than peer reviewed research. Such sources may emphasize positive aspects of a tool while providing less detail about limitations. This introduces potential bias when comparing tool support and functionality.

7.3.2 Standard Benchmarks

The evaluation of method-agnostic support was conducted using a limited set of selected risk assessment methods. Although the evaluated methods were intentionally chosen to represent structurally different approaches, the results cannot establish general support for all possible HARA/TARA methodologies.

7.3.3 Limited Evaluation

Parts of the evaluation relies on qualitative interpretation and expert feedback rather than some standardized benchmark metrics (non which we could find). This introduces a degree of subjectivity in the assessment of method support and workflow suitability.

7.3.4 LLM reproducibility

The LLM-assisted evaluation was limited to a single system scenario and also a constrained number of generated outputs. The LLM responses are probabilistic and somewhat sensitive to prompt context. Repeated executions may also produce different suggestions. This affects the reproducibility of the generated results and limits the ability to draw generalized conclusions regarding the consistency of the LLM functionality.

7.3.5 Language Perspectives

The language used in the survey to conclude the first iteration was English. English was selected to preserve domain-specific terminology without translations. However, because the respondents English proficiency was not formally assessed, misunderstandings may have influenced the prioritization of support features. As the results of the survey were very close, some small misunderstanding could have affected which feature was selected for implementation.

The expert interviews conducted in the study were held in Swedish and the transcripts were translated into English. This could generate errors in translation and possibly misrepresent the precise meaning of the interviewees intended meaning. This should be considered when interpreting the interview results and when attempting to replicate the study.

7.4 Social and ethical perspective

The tool developed in this thesis is intended to support risk assessment processes in IT and OT systems, where incorrect analysis may have consequences for safety, security, and service

availability. This introduces several social and ethical considerations related to the use of AI-assisted analysis, the handling of sensitive system information, and the broader implications of making such tooling widely accessible.

The integration of LLM-based functionality introduces several ethical considerations. Although LLMs can support analysts by generating risk suggestions and identifying potential threats, the generated output may contain inaccuracies, incomplete reasoning, or misleading recommendations. Since the generated suggestions are often formulated in a confident and human-like manner, there is a risk that users place excessive trust in the output without sufficient critical evaluation. In the context of safety- and security-critical systems, such overreliance could contribute to incomplete or incorrect risk assessments and potentially influence important decisions. To mitigate this risk, the tool follows a human-in-the-loop approach where the LLM acts as a supplementary support function rather than an authoritative decision-maker. Final assessments and decisions remain the responsibility of domain experts. This requires that users understand both the capabilities and limitations of AI-assisted analysis and that generated suggestions are critically reviewed before being incorporated into risk assessments.

Another important consideration concerns the handling of sensitive information. Risk assessments often contain detailed descriptions of system architectures, vulnerabilities, operational processes, and potential attack vectors. Unauthorized access to such information could itself introduce security risks. The integration of external AI services may further complicate this issue if sensitive data is transmitted to third-party providers for processing. Consequently, organizations using AI-assisted risk assessment tools must carefully evaluate data handling practices, confidentiality requirements, and deployment strategies to ensure compliance with security and privacy requirements.

The possibility of releasing the tool as open source also introduces both societal benefits and potential risks. From a societal perspective, open-source availability can increase accessibility to structured risk assessment methodologies, particularly for smaller organizations, educational institutions, and communities that may lack access to expensive commercial tools. This may contribute to improved awareness and adoption of cybersecurity and functional safety practices across a broader range of organizations. At the same time, the use of an open-source risk assessment tool requires careful handling of project-specific assessment data and system information. Public access to project-specific assessment data, system models, workflows, or implementation details could potentially provide malicious actors with insights into critical assets, system architecture, or potential attack surfaces. Open-source development therefore requires careful consideration regarding transparency, documentation, and security-related implementation details. Balancing accessibility, collaboration, and security becomes an important ethical consideration for future development and deployment of the tool.

8 Conclusion

This thesis investigated how a HARA/TARA framework can be represented and supported by a method-agnostic software tool. The work resulted in a proof-of-concept artifact. The implementation demonstrates that it is feasible to design a tool with a flexible and extensive architecture that is both capable of safety and security analysis, within a combined and unified system.

The results indicate that the integration between functional safety and cybersecurity is becoming more and more necessary to analyze in modern cyber-physical systems. As IT and OT systems become more interconnected, security incidents may have influence physical processes and safety. The proposed artifact illustrates the potential advantages and disadvantages of a unified assessment environment, where risk structures and system representations can be shared between stakeholders from different domains.

From a broader perspective, the thesis contributes to the discussion on integrated safety and security assessment within cybersystems. Rather than proposing another specific domain assessment methodology, this work demonstrates how methodologies that already exist can be represented and abstracted within a unified structure. This shifts the focus from method-specific tooling towards adaptable and interoperable assessment environments that are capable of evolving alongside regulatory and industry requirements.

RQ1: To what extent can a common HARA/TARA framework be represented within a method-agnostic software architecture that supports both functional safety and security analysis? Regarding RQ1, the thesis shows that a method agnostic representation of a HARA/TARA tool can be achieved by abstracting common structural elements across different methodologies. The prototype successfully decouples the model from the method specific logic, which in turn enables the integration of frameworks such as SAHARA, HEAVENS and others within the shared architecture of the tool. The results support the feasibility of a method-agnostic architecture for structured qualitative and semi-quantitative methods.

RQ2: How effectively does the proposed artifact support collaboration and interoperability among stakeholders involved in risk assessment processes? For RQ2 , the evaluation indicates that the proposed artifact provides support for collaboration and interoperability. The system enables concurrent multi-user interactions, which allows several participants to

contribute to the same risk assessment in parallel. The integration of further supporting features such as the DFD component provides a shared structured system representation allowing for common understanding between users and stakeholders. Additionally, functionalities such as the lookup table builder and the import/export features facilitate the exchange and reuse of risk assessment methods and data across different tools and methodologies, thereby enhancing interoperability. Together, these features demonstrate that the artifact supports collaborative workflows and a consistent risk assessment process. However, the evaluation also shows that further work is needed before the tool can be considered ready for industrial deployment.

RQ3: What are the opportunities and limitations of integrating LLM-based support into a method-agnostic HARA/TARA tool for threat identification and risk analysis? The findings from the evaluation of RQ3 show that LLM-based support can be integrated into a proof-of-concept tool. The results indicate that LLMs are capable of generating relevant risk suggestions based on a system description and available risk data provided by a HARA/TARA specialist. This can assist analysts and experts in identifying threats that might have been missed or overlooked during manual analysis. However, the evaluation also highlights important limitations. The quality and reliability of the generated suggestions varied depending on the amount and quality of relevant context given to the model. The LLM sometimes produced broad, overlapping, or insufficiently specific suggestions. Therefore, the suggestions generated in the tool by the LLM must be reviewed by a human expert. The LLM component must be seen as a supplement and supportive feature rather than a replacement for human experts.

Overall, the proposed method-agnostic tool shows strong potential for supporting an integrated and collaborative risk assessment, while also highlighting the areas where further research and improvement are needed. The evaluation was conducted within the scope of a proof-of-concept prototype implementation and therefore does not fully reflect large-scale industrial deployment scenarios. Further validation in real-world environments is necessary to assess scalability, usability, and long-term integration within existing engineering workflows.

8.1 Future work

Several important research directions emerge from this work. First of all, further analysis and evaluation of the proposed artifact in real-world could provide insight into where the greatest benefit can be achieved. A formal user study could be a suitable option to give more grounded and concrete development options. Moreover, the tool currently supports a broad number of methodologies but it lacks some computational features. Future development should consider investigating the possibility of incorporating advanced capabilities such as stochastic modeling for probabilistic analysis, post-analysis calculations, weighted parameters expressions and statistical aggregation methods. Moreover, the tool is currently tailored to support qualitative risk assessment methods, these features could expand the capability of the tool considerably thereby enabling adoption across a broader user base.

Another important direction concerns collaborative functionality and interoperability. While the current implementation supports concurrent multi-user interaction, future development could investigate mechanisms such as conflict detection and resolution, version history, change tracking, and approval workflows to improve coordination and traceability in larger assessment processes. Additionally, improved interoperability through standardized APIs, import/export functionality, and integration with external risk management or requirements engineering tools could further support adoption within existing organizational workflows.

Finally, the modular LLM integration could be further expanded and evaluated. Investigating suitable existing models and the feasibility of a standardized domain specific risk

assessment model could generate interesting results. Furthermore, further integration of AI features such as in-line suggestions, report generation and chat assistant could also provide more user support functionality, alleviating the tediousness of risk assessment work.

Bibliography

[1] Mafijul Md. Islam, Aljoscha Lautenbach, Christian Sandberg, and Tomas Olovsson. "A Risk Assessment Framework for Automotive Embedded Systems". In: *Proceedings of the 2nd ACM International Workshop on Cyber-Physical System Security*. CPSS '16. Xi'an, China: Association for Computing Machinery, 2016, pp. 3–14. ISBN: 9781450342889. DOI: 10.1145/2899015.2899018. URL: https://doi.org/10.1145/2899015.2899018.

[2] Georg Macher, Harald Sporer, Reinhard Berlach, Eric Armengaud, and Christian Kreiner. "SAHARA: A security-aware hazard and risk analysis method". In: *2015 Design, Automation Test in Europe Conference Exhibition (DATE)*. 2015, pp. 621–624. DOI: 10.7873/DATE.2015.0622.

[3] Xueqin Gao, Tao Shang, Da Li, and Jianwei Liu. "Quantitative Risk Assessment of Threats on SCADA Systems Using Attack Countermeasure Tree". In: *2022 19th Annual International Conference on Privacy, Security Trust (PST)*. 2022, pp. 1–5. DOI: 10.1109/PST55820.2022.9851965.

[4] Madhab Chandra Jena, Sarat Kumar Mishra, and Himanshu Sekhar Moharana. "Introduction of Time Impact Analysis (TIA) Method for Risk Analysis to Overcome Limitations of Existing Risk Analysis Methods". In: *Environmental Claims Journal* 36.4 (2024), pp. 467–484. DOI: 10.1080/10406026.2024.2427726. eprint: https://doi.org/10.1080/10406026.2024.2427726. URL: https://doi.org/10.1080/10406026.2024.2427726.

[5] Priyamvada Saxena and Raghavendra Sharma. "A Conceptual Framework to Assess and Mitigate Cyber Threats Through Anonymization and Risk Modelling". In: *2025 9th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS)*. 2025, pp. 1–5. DOI: 10.1109/CSITSS67709.2025.11294427.

[6] Inno Hwang, Hyunsuk Cho, and Seungjoo Kim. "Deriving Usability Evaluation Criteria for Threat Modeling Tools". In: *IEEE Access* 13 (2025), pp. 65246–65265. DOI: 10.1109/ACCESS.2025.3559917.

[7] Jherrod Thomas. "HATARA: A Novel Approach by Fusion of HARA and TARA for System Safety and Security Analysis". In: *International Journal of Innovative Science and Research Technology* 9.2 (2024), pp. 381–385.

[8] Microsoft. *The STRIDE Threat Model*. https://learn.microsoft.com/en-us/previous-versions/commerce-server/ee823878(v=cs.20)?redirectedfrom=MSDN. Accessed: 2026-01-19. 2005.

[9] *ISO/IEC 27005:2022 Information Security, Cybersecurity and Privacy Protection — Guidance on Managing Information Security Risks*. ISO/IEC, 2022. URL: https://www.iso.org/standard/80585.html.

[10] Spyridon Samonas and David Coss. "The CIA strikes back: Redefining confidentiality, integrity and availability in security." In: *Journal of Information System Security* 10.3 (2014).

[11] Adam Shostack. *Threat modeling: Designing for security*. John wiley & sons, 2014.

[12] Bruce Schneier et al. "Attack trees". In: *Dr. Dobb's journal* 24.12 (1999), pp. 21–29.

[13] Microsoft. *Microsoft Threat Modeling Tool*. Accessed: 2026-05-14. URL: https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool.

[14] *OWASP Threat Dragon*. Accessed: 2026-05-14. URL: https://owasp.org/www-project-threat-dragon/.

[15] THREATGET. *ThreatGet - Threat Analysis and Risk Management*. Accessed: 2026-05-14. URL: https://www.threatget.com/.

[16] *IriusRisk Community*. Accessed: 2026-05-14. URL: https://www.iriusrisk.com/.

[17] *Eclipse Capella (+DARC Viewpoint (Cybersecurity Assets Threats Modeling analysis))*. Accessed: 2026-05-14. URL: https://github.com/eclipse-capella/capella-cybersecurity.

[18] *PyTM*. Accessed: 2026-05-14. URL: https://github.com/izar/pytm.

[19] *Threat Composer*. Accessed: 2026-05-14. URL: https://github.com/awslabs/threat-composer.

[20] *SD Elements*. Accessed: 2026-05-14. URL: https://www.securitycompass.com/devici/.

[21] *ThreatModeler*. Accessed: 2026-05-14. URL: https://threatmodeler.com.

[22] *SystemWeaver*. Accessed: 2026-05-14. URL: https://systemweaver.com.

[23] *Threats Manager Studio*. Accessed: 2026-05-14. URL: https://threatsmanager.com.

[24] *Varonis Data Security Platform*. Accessed: 2026-05-14. URL: https://www.varonis.com/products/data-security-platform.

[25] *CAIRIS (Computer Aided Integration of Requirements and Information Security)o*. Accessed: 2026-05-14. URL: https://cairis.org.

[26] *Threagile*. Accessed: 2026-05-14. URL: https://threagile.io.

[27] *Yacraf*. Accessed: 2026-05-14. URL: https://github.com/KTH-SSAS/EP2791-Cybersecurity-Threat-Modeling-and-Risk-Analysis/tree/master/YACRAF-Calculator-Tool.

[28] Arwa Y Aleryani. "Comparative study between data flow diagram and use case diagram". In: *International Journal of Scientific and Research Publications* 6.3 (2016), pp. 124–126.

[29] Ruoyu Su and Xiaozhou Li. "Modular Monolith: Is This the Trend in Software Architecture?" In: *Proceedings of the 1st International Workshop on New Trends in Software Architecture*. SATrends '24. Lisbon, Portugal: Association for Computing Machinery, 2024, pp. 10–13. ISBN: 9798400705601. DOI: 10.1145/3643657.3643911. URL: https://doi.org/10.1145/3643657.3643911.

[30] *Maskinsäkerhet – Riskbedömning – Del 2: Praktisk vägledning och exempel på metoder*. Tech. rep. SIS-ISO/TR 14121-2:2012. Swedish Institute for Standards, 2012.

[31] Georg Macher, Andrea Höller, Harald Sporer, Eric Armengaud, and Christian Kreiner. "A combined safety-hazards and security-threat analysis method for automotive systems". In: *International Conference on Computer Safety, Reliability, and Security*. Springer. 2014, pp. 237–250.

[32] Aljoscha Lautenbach, Mafijul Islam, Christian Sandberg, and Tomas Olovsson. *HEAVENS – HEAling Vulnerabilities to ENhance Software Security and Safety: Deliverable D2 – Security Models*. Tech. rep. D2. Version 2.0, Vinnova/FFI, Sweden. 2016.

[33] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. "Language models are few-shot learners". In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.

[34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).

[35] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing". In: *ACM computing surveys* 55.9 (2023), pp. 1–35.

[36] Yucheng Chu, Hang Li, Kaiqi Yang, Yasemin Copur-Gencturk, and Jiliang Tang. "LLM-Based Automated Grading with Human-in-the-Loop". In: *2025 IEEE International Conference on Teaching, Assessment, and Learning for Engineering (TALE)*. 2025, pp. 1–8. DOI: 10.1109/TALE66047.2025.11346771.

[37] Joel Gustafsson and Anton Hagel. *Designing a Model-agnostic Cybersecurity and Functional Safety Risk Assessment Tool*. 2025.

[38] *Vägfordon – Funktionssäkerhet i el- och elektroniksystem – Del 2: Ledning och hantering av funktionssäkerhet*. Tech. rep. SS-ISO 26262-2:2018. Swedish Institute for Standards, 2018.

[39] Christian Sandberg, Andreas Bokesand, and Urban Thorsson. *Tailoring the HEAVENS risk assessment methodology for improved performance*. Tech. rep. Deliverable D4.1.1. Mar. 2018. URL: https://autosec.se/wp-content/uploads/2018/03/HOLISEC_D4.1.1_v1.0.pdf.

[40] Aljoscha Lautenbach, Magnus Almgren, and Tomas Olovsson. "Proposing HEAVENS 2.0–an automotive risk assessment model". In: *Proceedings of the 5th ACM Computer Science in Cars Symposium*. 2021, pp. 1–12.

[41] Anders Thorsén, Behrooz Sangchoolie, Peter Folkesson, and Ted Strandberg. "Combined Safety and Cybersecurity Risk Assessment for Intelligent Distributed Grids". In: *CSG 2022: 16. International Conference on Smart Grids January 28-29, 2022 in Dubai, United Arab Emirates*. 2022.

[42] Lars Halvdan Flå, Ravishankar Borgaonkar, Inger Anne Tøndel, and Martin Gilje Jaatun. "Tool-assisted Threat Modeling for Smart Grid Cyber Security". In: *2021 International Conference on Cyber Situational Awareness, Data Analytics and Assessment (CyberSA)*. 2021, pp. 1–8. DOI: 10.1109/CyberSA52016.2021.9478258.

[43] Erlend Bygdås, Lars Andreassen Jaatun, Stian Brekken Antonsen, Anders Ringen, and Erlend Eiring. "Evaluating threat modeling tools: Microsoft TMT versus OWASP Threat Dragon". In: *2021 International Conference on Cyber Situational Awareness, Data Analytics and Assessment (CyberSA)*. IEEE. 2021, pp. 1–7.

[44] Xueqin Gao, Tao Shang, Da Li, and Jianwei Liu. "Quantitative Risk Assessment of Threats on SCADA Systems Using Attack Countermeasure Tree". In: *2022 19th Annual International Conference on Privacy, Security Trust (PST)*. 2022, pp. 1–5. DOI: 10.1109/PST55820.2022.9851965.

[45] Seied Veria Hoseini, Jaakko Suutala, Juha Partala, and Kimmo Halunen. "Threat Modeling AI/ML With the Attack Tree". In: *IEEE Access* 12 (2024), pp. 172610–172637. DOI: 10.1109/ACCESS.2024.3497011.

[46] Enrico Fregnan, Christian Göttel, Balz Maag, Abdallah Dawoud, and Georgios Nakas. "Leveraging LLMs Towards Assistant-based Support for Industrial Threat Models". In: *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*. 2025, pp. 1–8. DOI: 10.1109/ETFA65518.2025.11205762.

[47] Jan Von der Assen, Alberto Huertas, Jamo Sharif, Chao Feng, Gérôme Bovet, and Burkhard Stiller. "ThreatFinderAI: Automated Threat Modeling Applied to LLM System Integration". In: *2024 20th International Conference on Network and Service Management (CNSM)*. 2024, pp. 1–3. DOI: 10.23919/CNSM62983.2024.10814632.

[48] Priya Joy Kaviyil, Danny McFadden, and Ruth Lennon. "ChatGPT Created Threat Model". In: *2023 Cyber Research Conference - Ireland (Cyber-RCI)*. 2023, pp. 1–7. DOI: 10.1109/Cyber-RCI59474.2023.10671398.

[49] Shantu Asif Hossain. "Automated Threat Modeling using Artificial Intelligence on User Stories within the SDLC to Generate Security Tasks". In: *Proceedings of the 21st International Conference on Cyber Warfare and Security (ICCWS)*. 2026, pp. 694–704.

[50] Isra Elsharef, Zhen Zeng, and Zhongshu Gu. "Facilitating Threat Modeling by Leveraging Large Language Models.(2024)". In: *URL https://www. ndss-symposium. org/wpcontent/uploads/aiscc2024-16-paper. pdf,[Accessed 16-05-2025]*. 2024.

[51] Krit Kongurai. "Domain-Specific Contract Analysis: LLMs vs. Engineers in Power Plant TOR Review". In: *Proceedings of an IEEE Conference*. IEEE, 2025. DOI: 10.1109/11403579.

[52] Alan R Hevner, Salvatore T March, Jinsoo Park, and Sudha Ram. "Design science in information systems research". In: *MIS quarterly* (2004), pp. 75–105.

[53] Paolo Di Francesco, Patricia Lago, and Ivano Malavolta. "Architecting with microservices: A systematic mapping study". In: *Journal of Systems and Software* 150 (2019), pp. 77–97.

[54] MariaDB Foundation. *Download MariaDB Server*. 2026. URL: https://mariadb.org/download/ (visited on 05/28/2026).

[55] Pallets Projects. *Flask Documentation*. https://flask.palletsprojects.com/. Accessed: 2026-05-16. 2026.

[56] Georg Macher, Eric Armengaud, Christian Kreiner, and Reinhard Weiss. "A Combined Safety-Hazards and Security-Threat Analysis Method for Automotive Systems". In: *Computer Safety, Reliability, and Security (SAFECOMP 2015 Workshops)*. Springer, Sept. 2015, pp. 121–133. DOI: 10.1007/978-3-319-24249-1_21.

[57] *Analysis techniques for system reliability – Procedure for failure mode and effects analysis (FMEA)*. International Standard IEC 60812:2006. International Electrotechnical Commission, 2006.

[58] Massachusetts Institute of Technology. *What-If Analysis – Course Manual Appendix VI*. Accessed: 2026-04-13. 2024. URL: https://web.mit.edu/course/10/10.27/www/1027CourseManual/1027CourseManual-AppVI.html.

[59] Public Health Agency of Canada. *Risk assessment method at the Public Health Agency of Canada*. Accessed: 2026-04-21. 2025. URL: https://www.canada.ca/en/public-health/services/emergency-preparedness-response/rapid-risk-assessments-public-health-professionals/methods.html.

[60] Nancy Mead, Forrest Shull, Krishnamurthy Vemuru, and Ole Villadsen. *A Hybrid Threat Modeling Method*. Tech. rep. CMU/SEI-2018-TN-002. Accessed: 2026-Apr-21. Mar. 2018. URL: https://doi.org/10.1184/R1/12366992.v1.

[61] Richard A Caralli, James F Stevens, Lisa R Young, and William R Wilson. *Introducing octave allegro: Improving the information security risk assessment process*. Tech. rep. 2007.

[62] Jack Freund Jones and Jack Shapiro. *Measuring and Managing Information Risk: A FAIR Approach*. 2nd ed. Amsterdam: Elsevier, 2025.

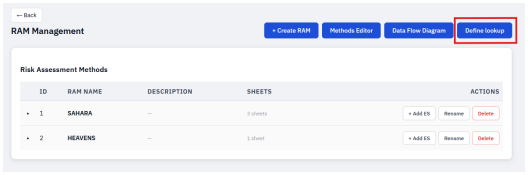
[63] Locust Developers. *Locust: A Modern Load Testing Framework*. https://locust.io/. Accessed: 2026-05-28. 2026.

[64] *Intelligent Transport Systems (ITS); Security; Threat, Vulnerability and Risk Analysis (TVRA)*. Tech. rep. ETSI TR 102 893 V1.2.1. European Telecommunications Standards Institute (ETSI), Mar. 2017.

A Appendix A

A.1 Implementing mapping and lookup

Implementing a mapping for a specific value such as ASIL or SL is a simple workflow process. The user have to define the axes, with name and range, and then add values (see Figure A.1e). Defining a range is similar in that after defining the name and the range of the mapping the user then have to set each numerical range to a specific value and label as seen in Figure A.1b. The whole workflow is represented in Figure A.1.



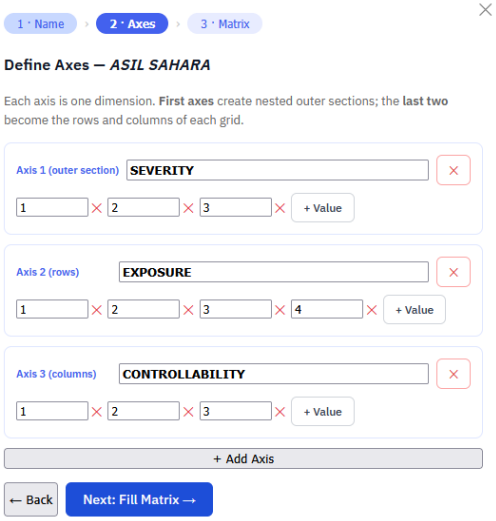
(a) Initial step to create lookup.



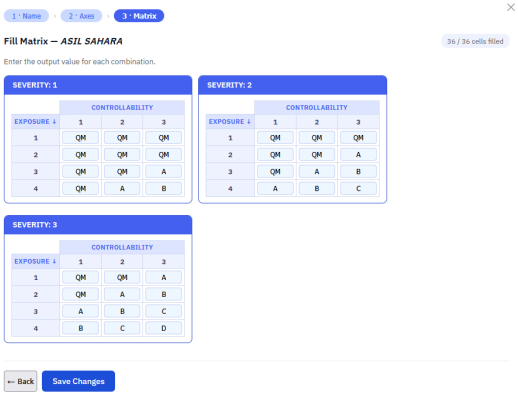
(b) Options in Lookup page.



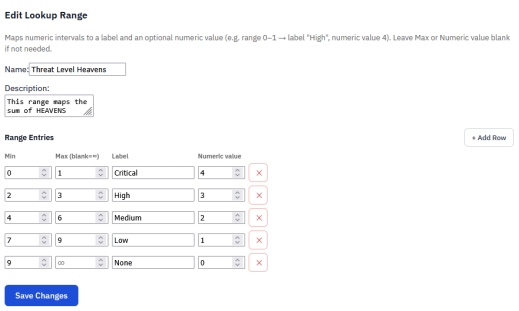
(c) Defining lookup table step 1.



(d) Defining lookup table step 3.



(e) Defining lookup table step 3.



(f) Defining lookup ranges.

Figure A.1: Implementation workflow for defining lookup tables and ranges.

A.2 Functionality of the Framework

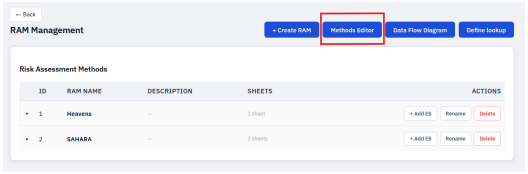
A.2.1 Building a Framework

The process of constructing a framework within the tool follows an iterative and recursive workflow. The user begins by defining a root Dimension, which enables the overall context and structure of the framework.

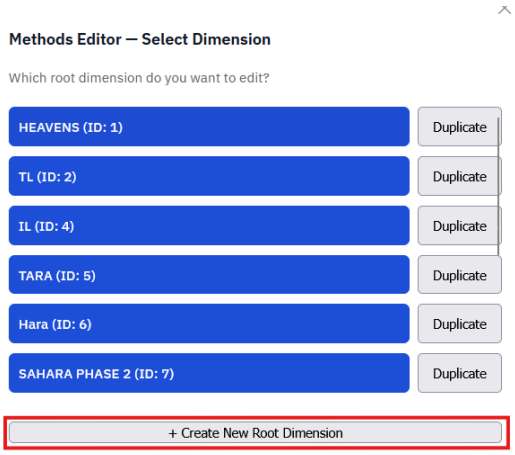
Once the root Dimension has been specified, the user proceeds by adding the remaining Dimensions, Parameters and Expressions that constitute the framework. At this stage, it is not necessary to fully refine all elements, as the tool allows for subsequent modifications and adjustments. Instead, the focus is on establishing the overall structure and relationships between components.

After the general structure has been defined, the user creates the RAM and the corresponding Evaluation sheet(s), which operationalize the framework for practical use. Once these components are in place, the framework can be finalized and applied to a project.

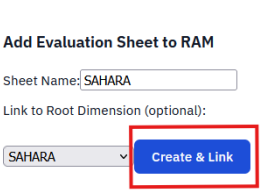
An example of illustrating this workflow i presented in Figure A.2, A.3, A.4.



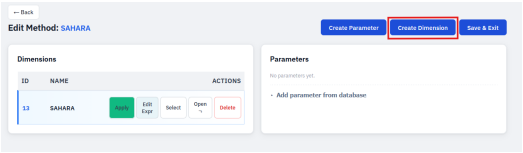
(a) Initial view of RAM.



(b) Selecting between existing or new dimension.

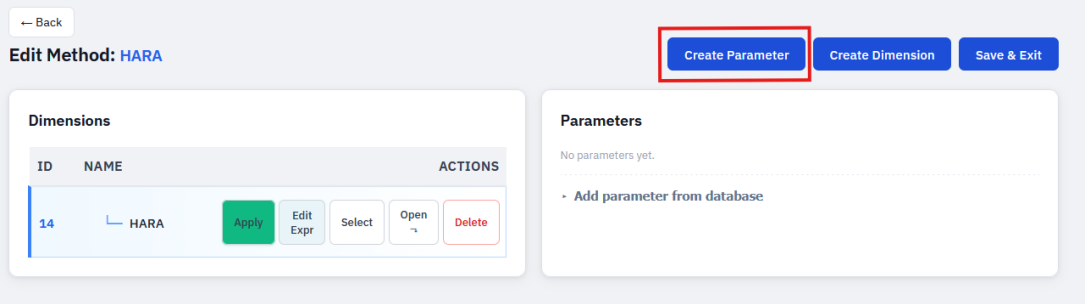


(c) Creating the root dimension.

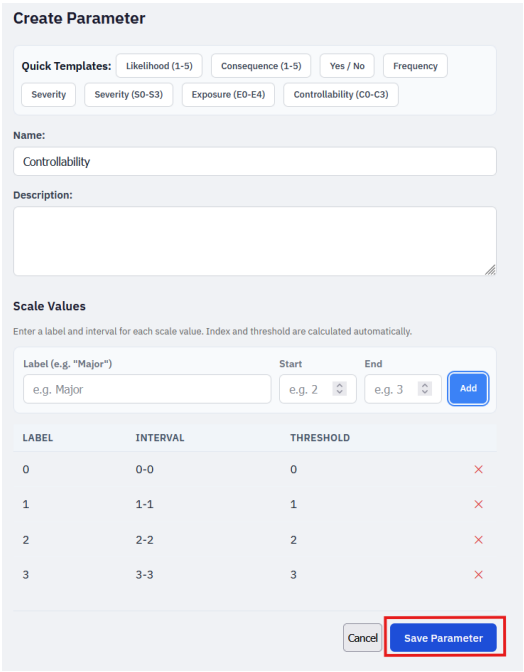


(d) Creating additional dimensions.

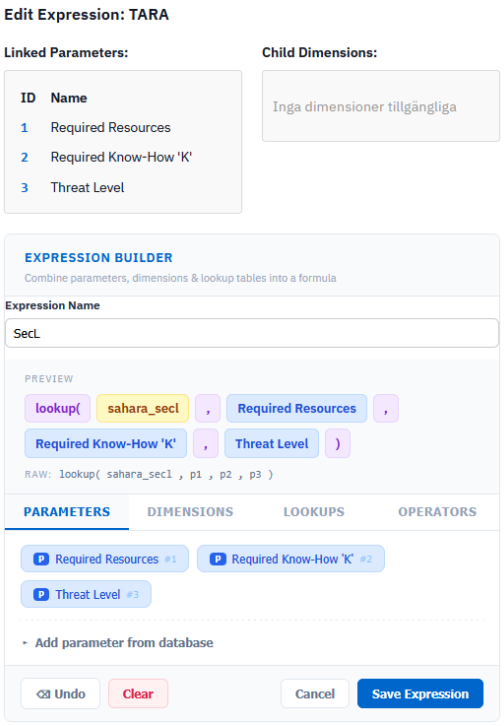
Figure A.2: Initial steps in creating a framework: defining dimensions.



(a) Creating parameters.

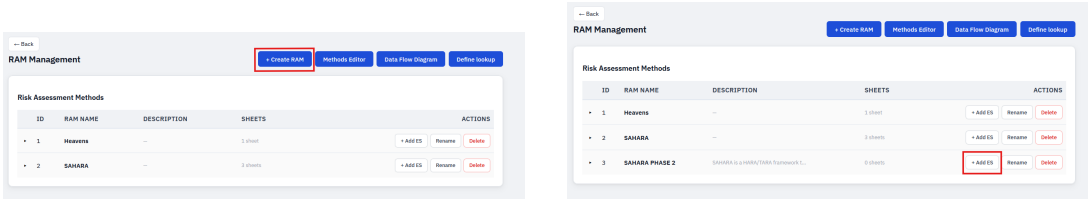


(b) Assigning parameter values.



(c) Adding expressions.

Figure A.3: Defining parameters and expressions in the framework.



(a) Creating the RAM.

(b) Adding evaluation sheets to the RAM.

Figure A.4: Final steps: creating and configuring the RAM.

A.3 Agnosticity

A.3.1 Implementing SAHARA

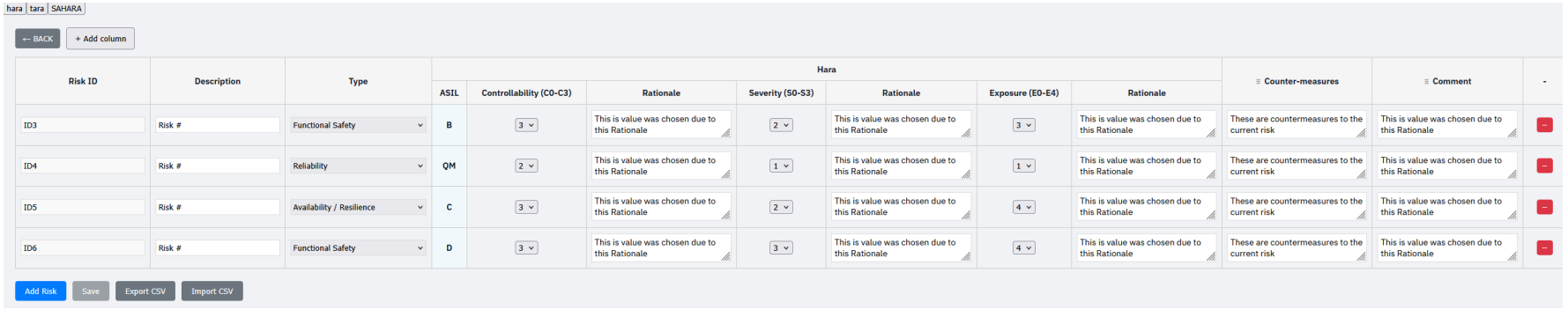
Following the steps seen in Figure A.2, A.3, A.4 we are able to implement the SAHARA framework. As the framework is built from several steps we represent it in the tool as three different Evaluation Sheets where the first sheet will be responsible for the HARA in the first phase of SAHARA, the second sheet is responsible for the TARA in the first phase of SAHARA and finally the third sheet will be responsible for the combined HARA/TARA in the second phase of SAHARA, which can be seen in Figure A.5. These pages was modeled after the Figures 1–3 in [56]. As we can see from this we have successfully replicated the framework in the tool.

Following the workflow illustrated in Figure A.2, A.3, A.4, the SAHARA framework was implemented within the tool. Due to the multi-phase structure of the framework, its implementation was decomposed into three separate evaluation sheets.

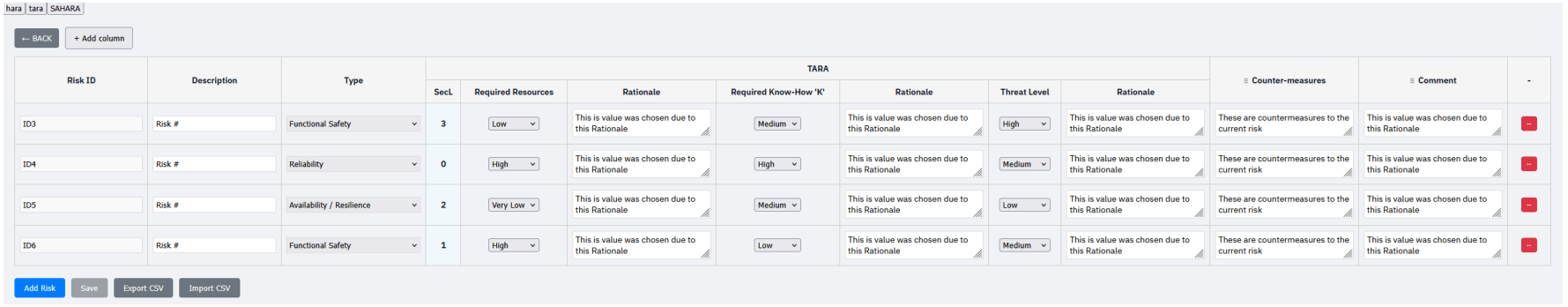
The first evaluation sheet corresponds to the HARA activities conducted in the initial phase of SAHARA. The second sheet represents the TARA activities within the same phase. Finally, the third evaluation sheet captures the combined HARA/TARA analysis performed in the second phase of the framework, as illustrated in Figure A.5.

The design of these evaluation sheets was based on the original representations of the SAHARA framework, specifically Figures 1–3 in [56]. By structuring the implementation in this manner, the tool is able to accurately replicate the procedural flow and analytical components of the SAHARA framework.

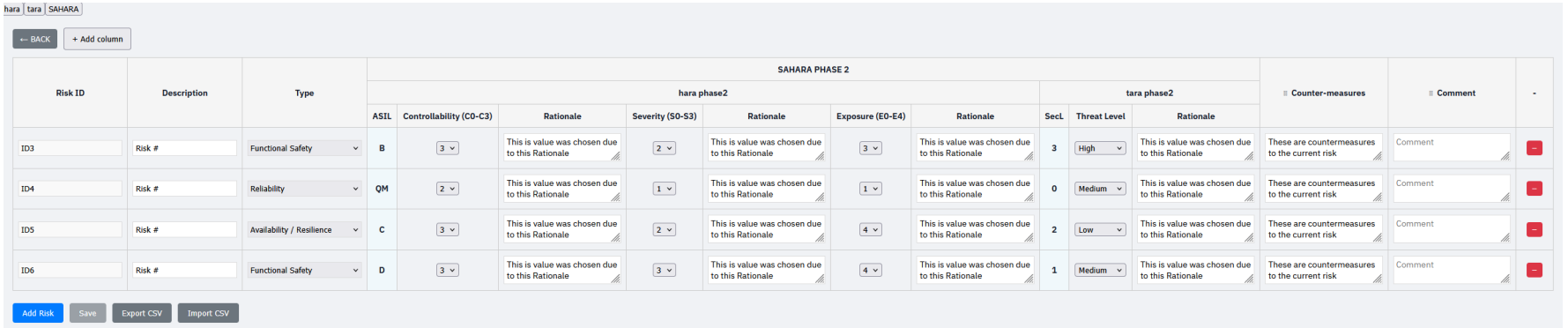
This demonstrates that the proposed method-agnostic approach is capable of supporting complex, multi-stage risk assessment frameworks without requiring framework-specific customization.



(a) HARA page of SAHARA.



(b) TARA page of SAHARA.



(c) SAHARA phase 2 page.

Figure A.5: Implementation of SAHARA.

A.3.2 Implementing HEAVENS

In contrast to SAHARA, the HEAVENS framework was implemented as a single Evaluation sheet, in which both HARA and TARA activities are performed within the same structure. This unified representation reflects the integrated nature of the HEAVENS methodology, where risk assessment and threat analyses are closely coupled.

Similar to the combined evaluation stage of SAHARA, the HEAVENS framework is structured around one root dimension and to subordinate dimensions. The subordinate dimensions are first evaluated, after which their results are mapped to a SL through a predefined lookup table associated with the root dimension.

The design of this Evaluation sheet was based on the original presentation of the HEAVENS framework, specifically Tables 3, 5, and 8 in [1]. By following this structure, the implementation preserves the logical relationships and calculation procedures defined in the original framework.

This further demonstrates that the proposed method-agnostic tool can accommodate both multi-stage frameworks, such as SAHARA, and more integrated approaches like HEAVENS, without requiring specialized adaptations.

HEAVENS

← BACK + Add column

Risk ID	Description	Type	HEAVENS																				
			SL	Threat Level									Impact Level									Counter-measures	Comment	-
				TL	Expertise	Rationale	Knowledge about TOE	Rationale	Window of Opportunity	Rationale	Equipment	Rationale	IL	Safety	Rationale	Financial	Rationale	Operational	Rationale	Privacy and Legislation	Rationale			
ID1	Risk 1	Functional Safety	low	Low (1)	Proficient	Rational e...	Sensitive	Rational e...	Small	Rational e...	Specialised	Rational e...	Medium (2)	Low	Rational e...	Low	Rational e...	Medium	Rational e...	Medium	Rational e...	Counter-measures	Comment	−
ID2	Risk 2	Privacy	medium	Medium (2)	Expert	Rational e...	Public	Rational e...	Medium	Rational e...	Standard	Rational e...	High (3)	High	Rational e...	Low	Rational e...	Low	Rational e...	None	Rational e...	Counter-measures	Comment	−

Add Risk Save Export CSV Import CSV

Figure A.6: Implementation of HEAVENS.

A.4 Risk Assessment Methods Implementation

A.4.1 FMEA

FMEA is implemented using the parameters Severity, Occurrence, and Detection. The resulting Risk Priority Number (RPN) is used to prioritize risks, as shown in Figures A.7–A.11.

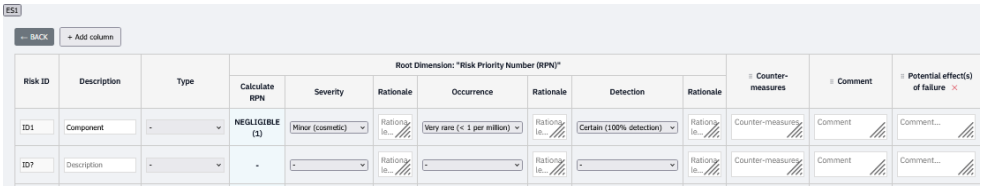


Figure A.7: FMEA 1



Figure A.8: FMEA 2

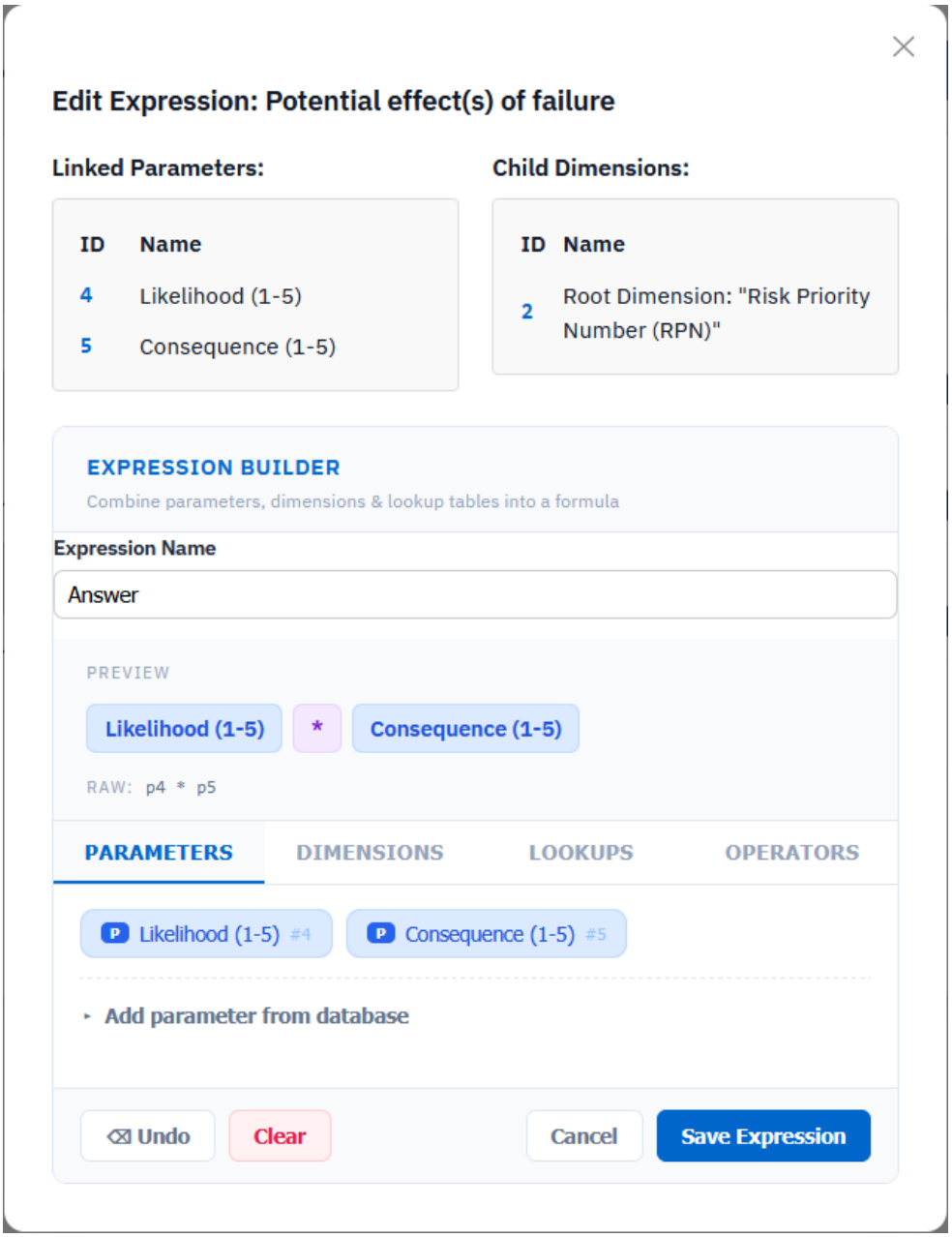


Figure A.9: FMEA 3

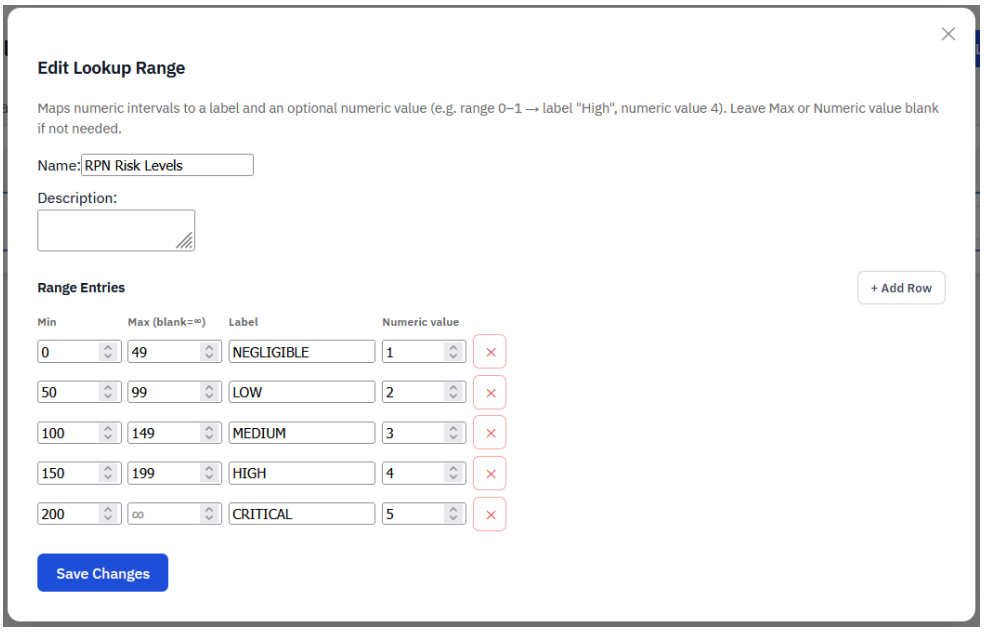


Figure A.10: FMEA 4

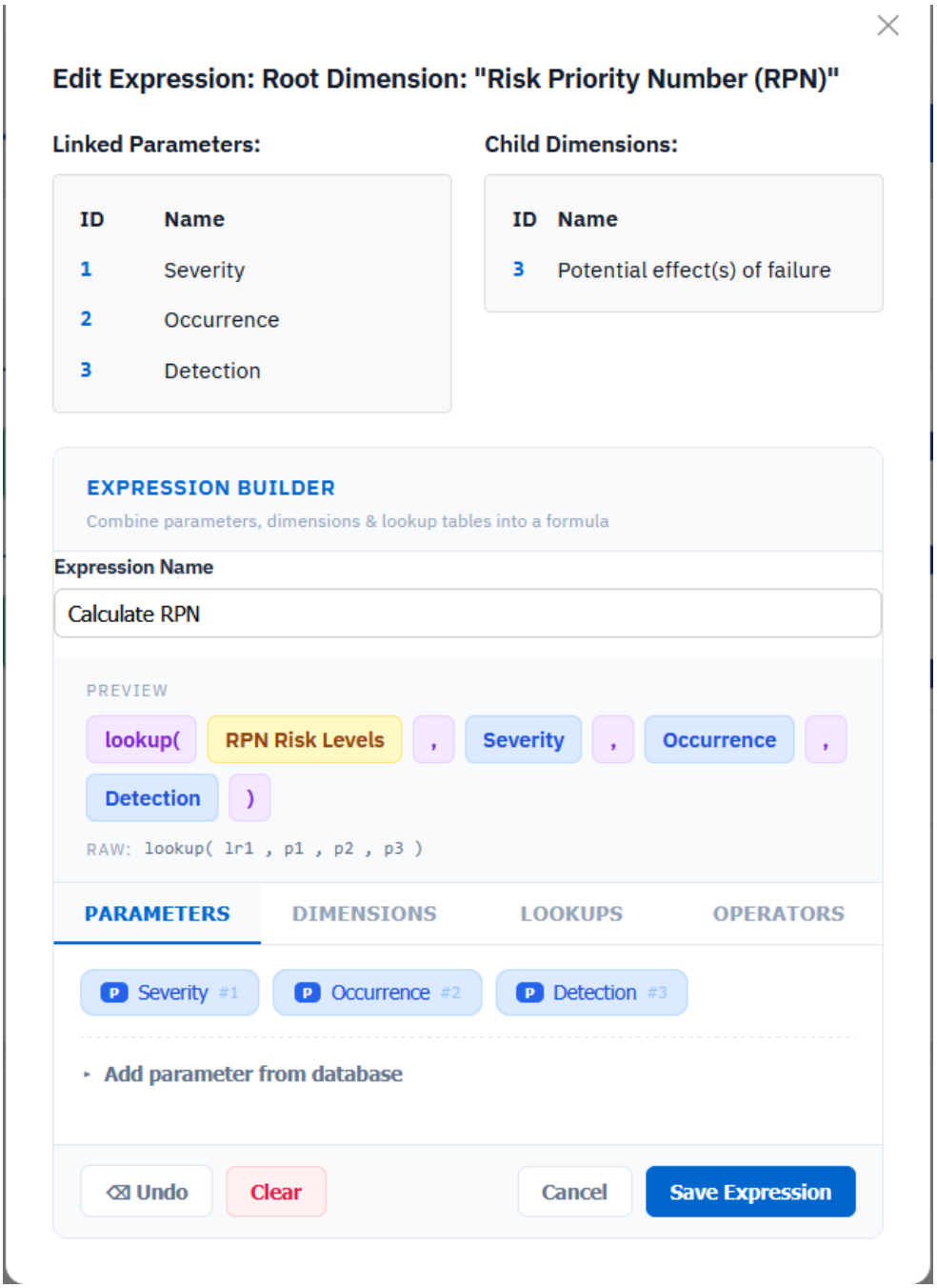


Figure A.11: FMEA 5

A.4.2 FAIR

FAIR is implemented through structured loss and scenario modeling. Figures A.12 and A.13 illustrate the representation.

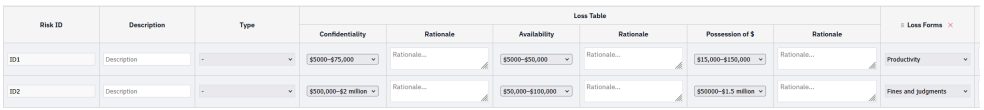


Figure A.12: FAIR Loss Table

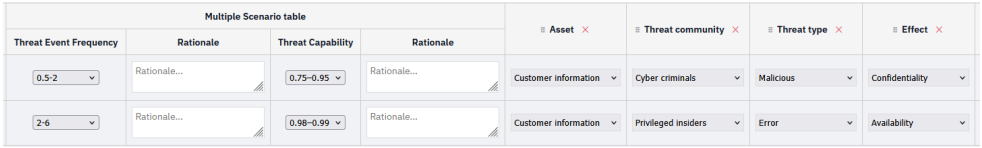


Figure A.13: FAIR Multiple Scenario Table

A.4.3 HazOp

HazOp is implemented as a qualitative method using structured deviations and causes. The illustration is shown in Figure A.14.

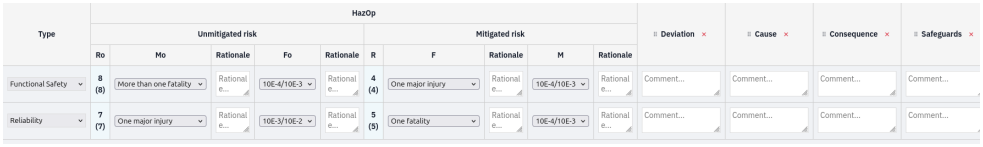


Figure A.14: Hazard and Operability Study implementation

A.4.4 HTMM

HTMM is represented through structured threat and risk evaluation. An example is shown in Figure A.15.

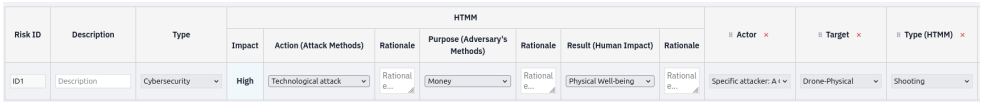


Figure A.15: HTMM implementation

A.4.5 OCTAVE Allegro

OCTAVE Allegro is implemented using multiple evaluation sheets for stepwise risk analysis. This is illustrated in Figures A.16–A.18.

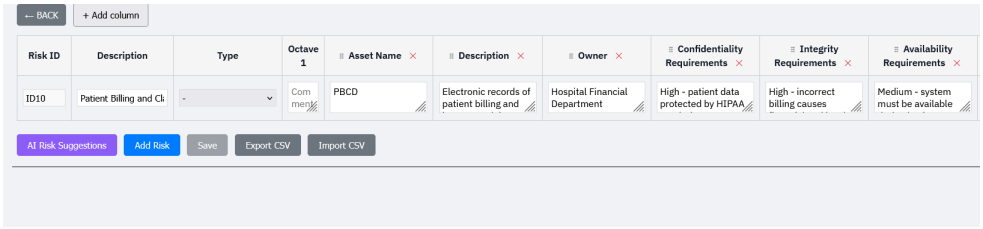


Figure A.16: OCTAVE Allegro Evaluation Sheet 1

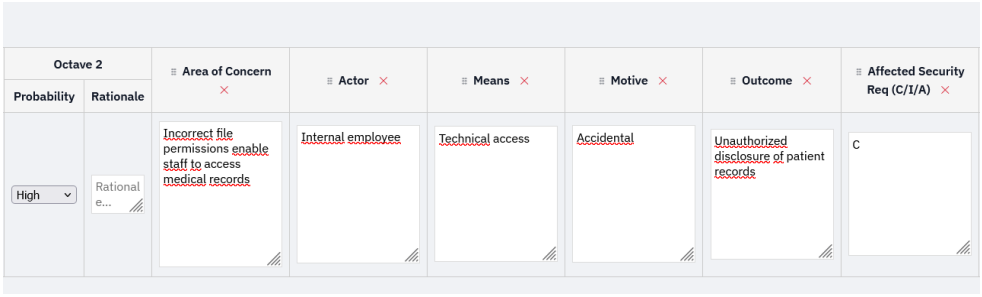


Figure A.17: OCTAVE Allegro Evaluation Sheet 2

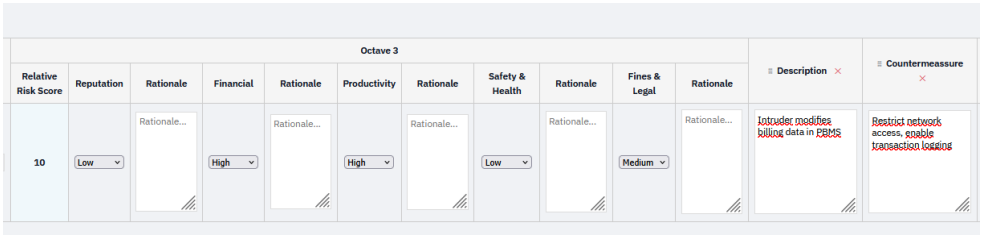


Figure A.18: OCTAVE Allegro Evaluation Sheet 3

A.4.6 PHA of Canada

The PHA method is implemented using parameter-based risk classification. The result is shown in Figure A.19.

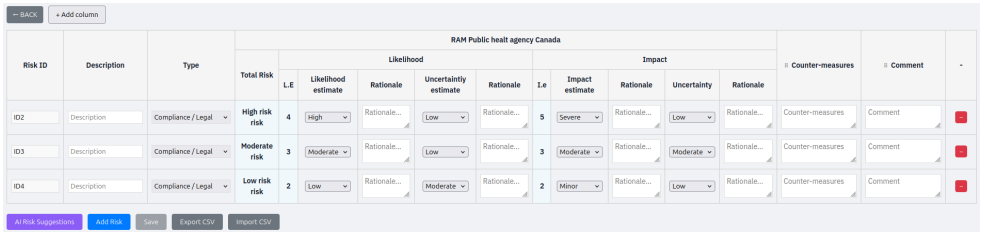


Figure A.19: PHA of Canada Risk Assessment Method

A.4.7 What-If Analysis

What-If Analysis is implemented as a qualitative method based on structured scenarios. An example is shown in Figure A.20.

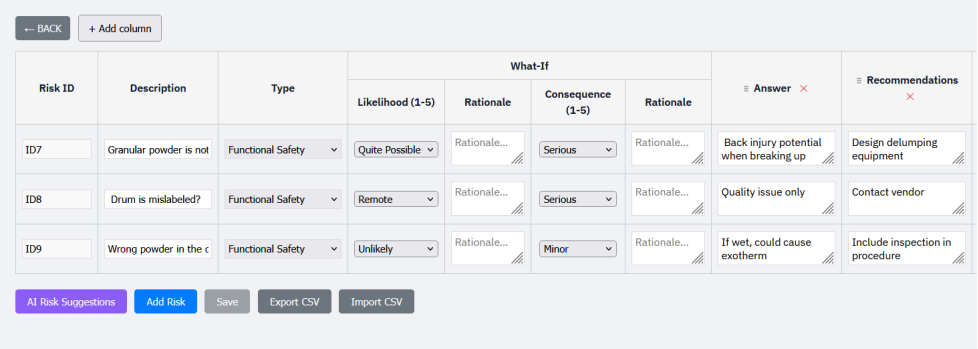


Figure A.20: What-If Analysis method for qualitative risk assessment through structured questioning

A.5 Multiple User Test

This section presents the results of the multi-user evaluation. The purpose of the evaluation was to verify that the developed artifact supports collaborative risk assessment by allowing multiple users to work simultaneously within the same project. The following figures illustrate the workflow for inviting users, concurrent editing, and synchronization of assessment data.

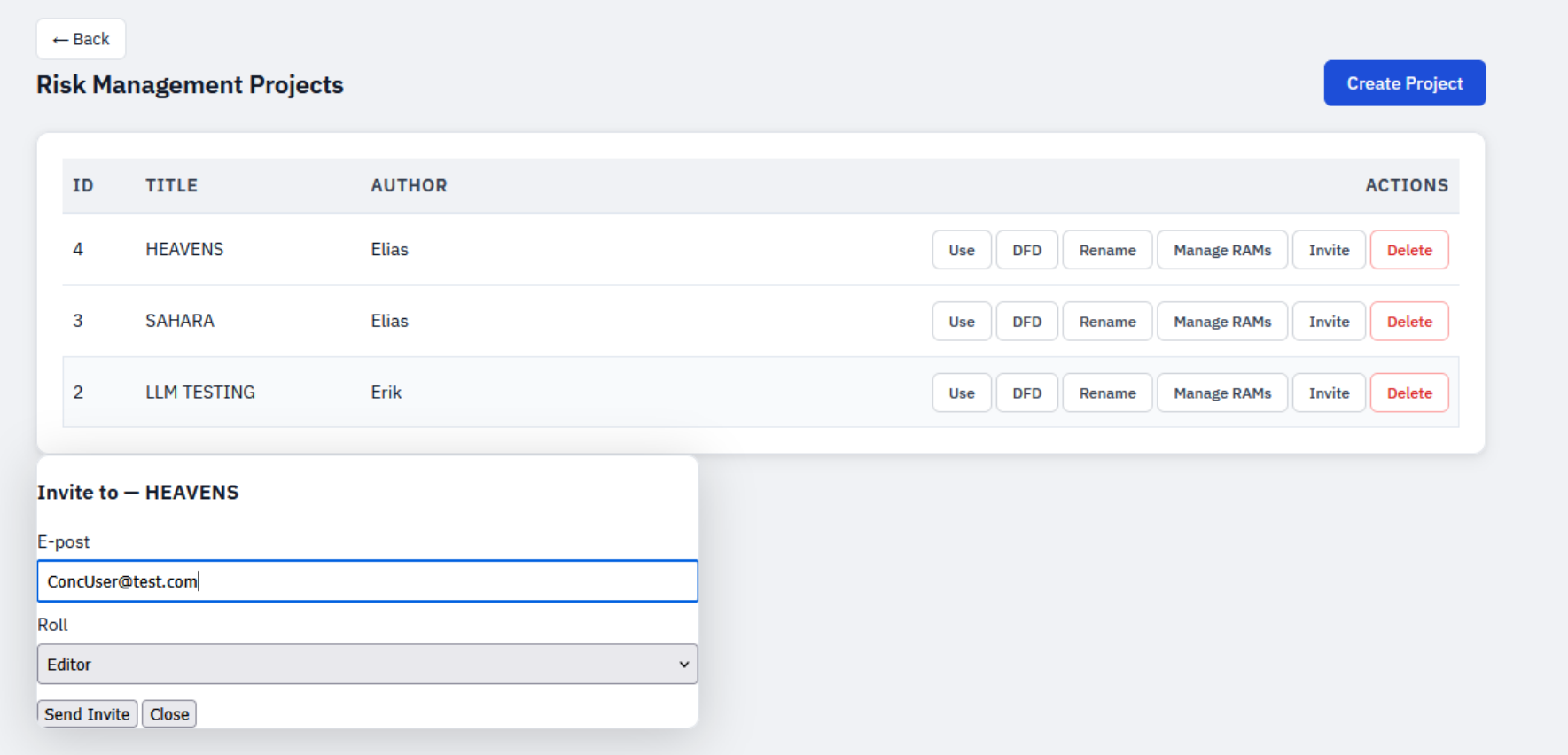


Figure A.21: User A inviting a user to a project.

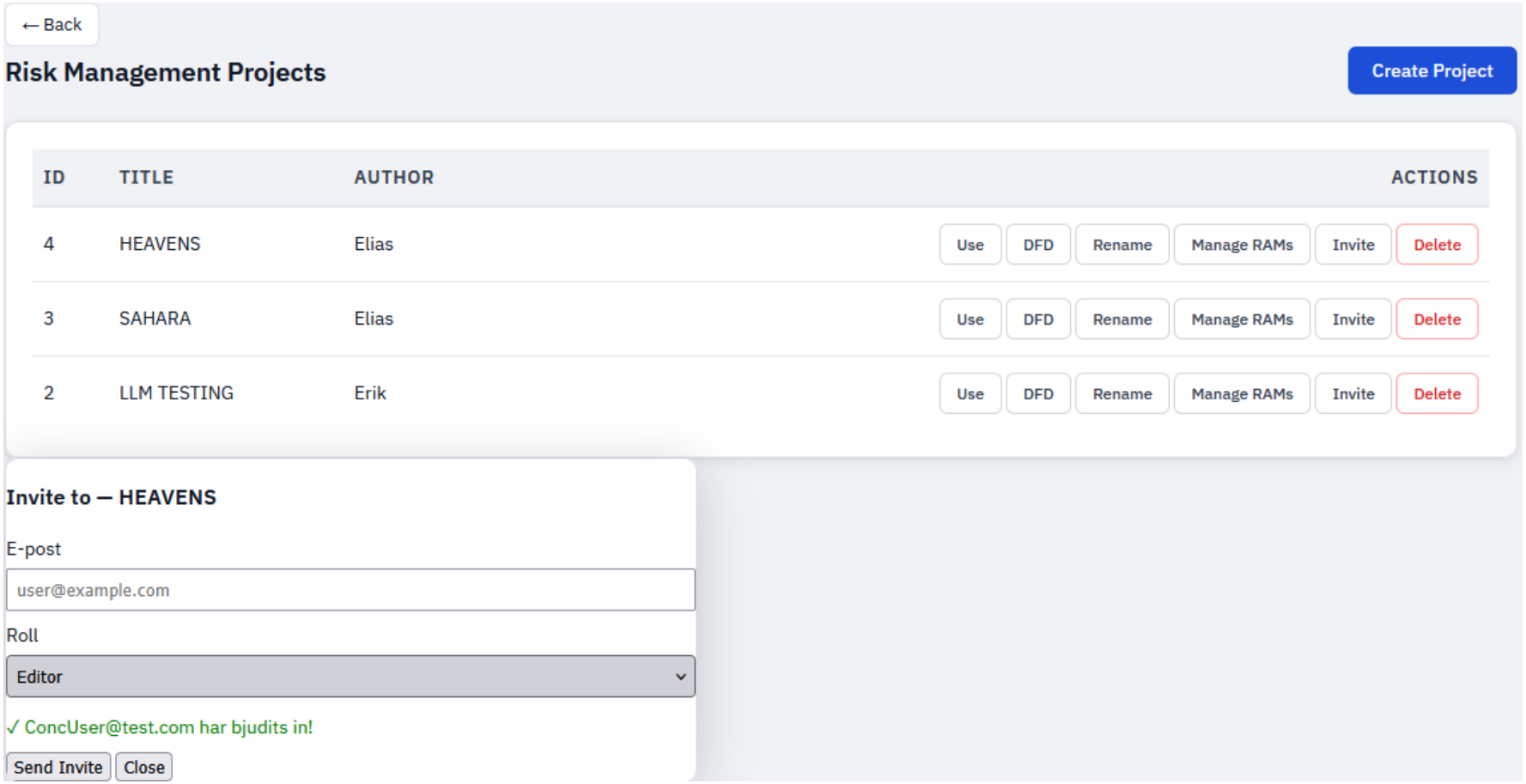


Figure A.22: User A after inviting a user to a project.

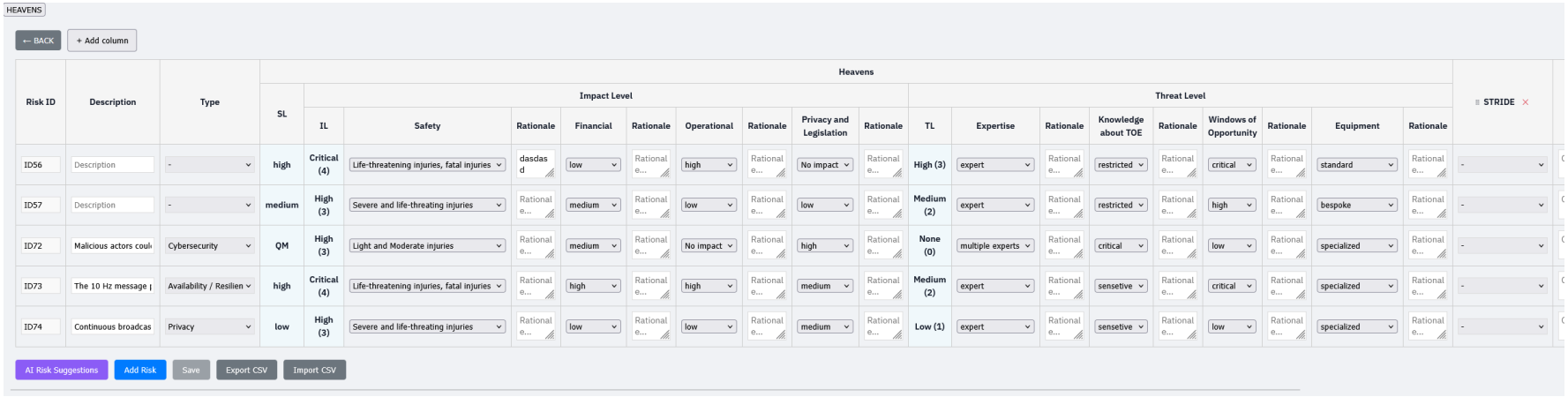


Figure A.23: User A baseline before editing.

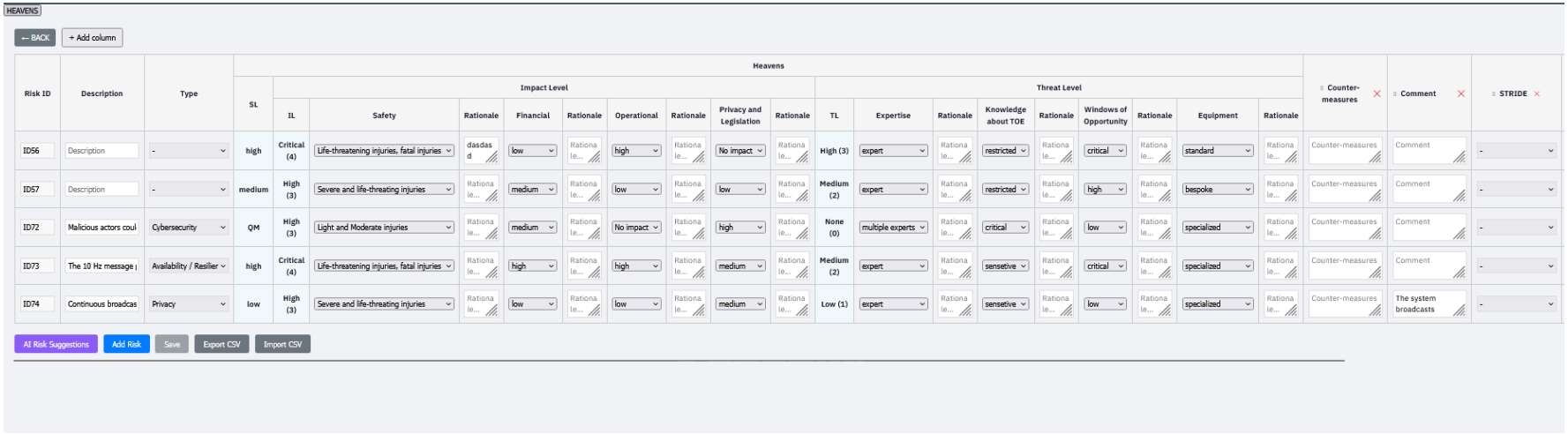


Figure A.24: User B baseline before editing.

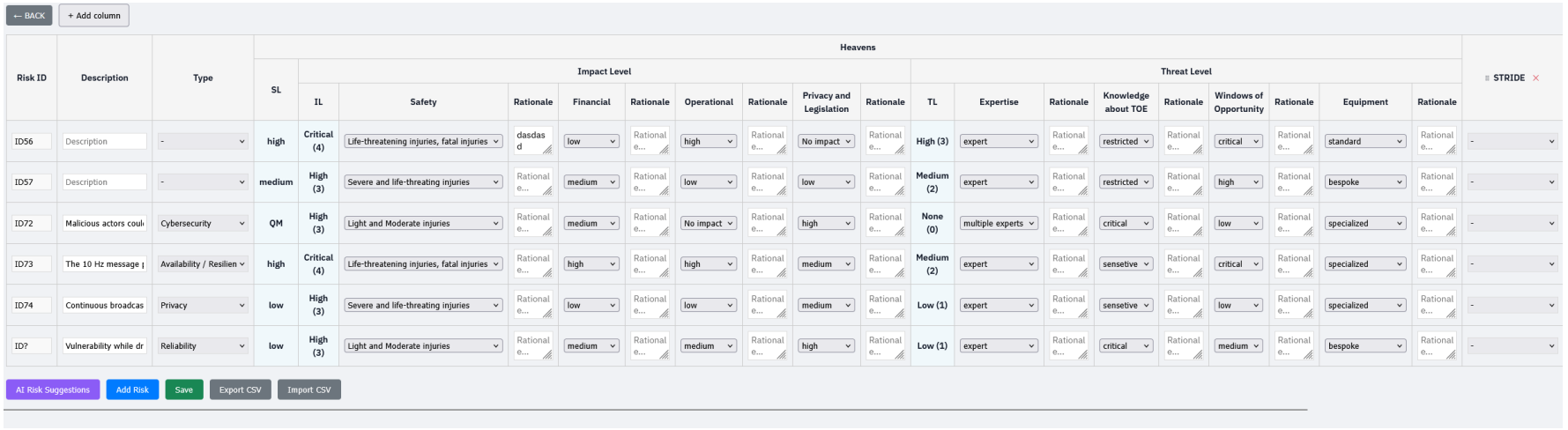


Figure A.25: User A while editing row.

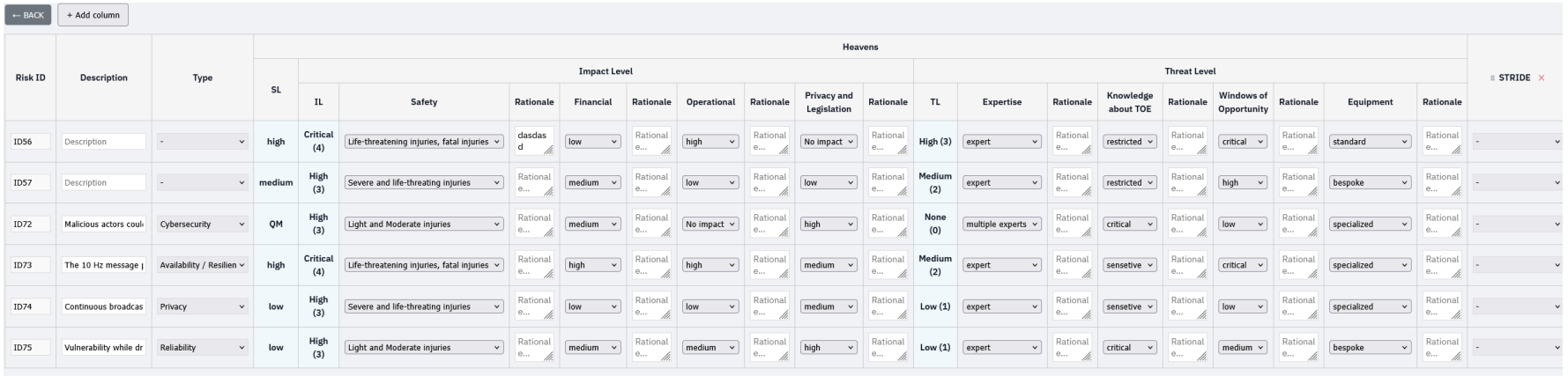


Figure A.26: User A after saving edit.

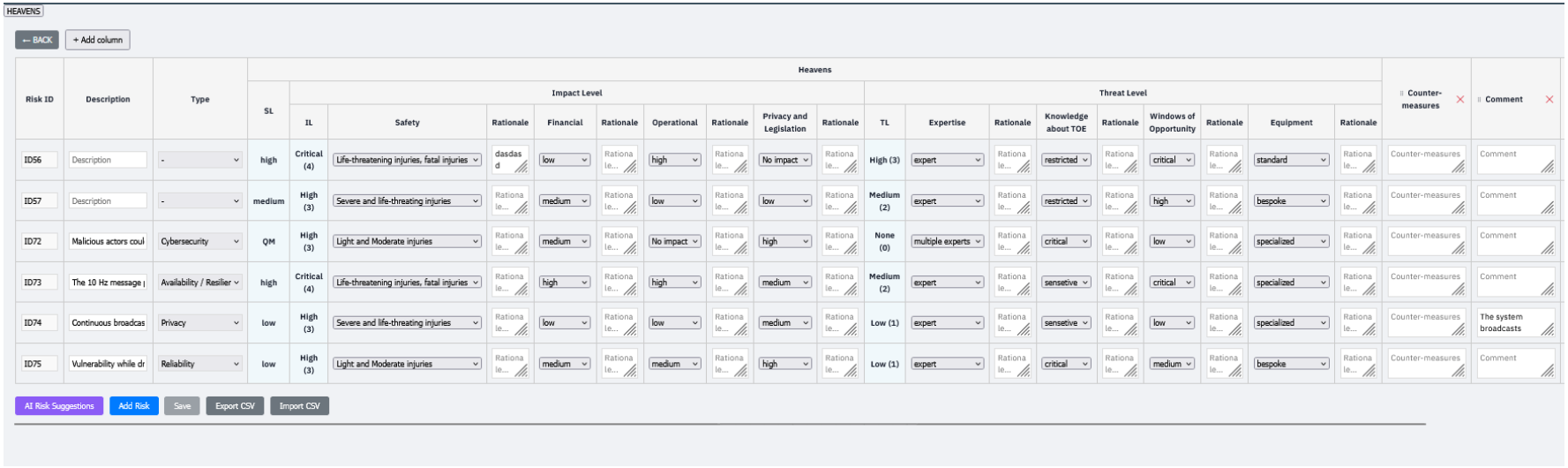


Figure A.27: User B after changes saved by user A.

A.6 Expert interview questions and answers

This appendix presents the complete interview questions and responses from the expert evaluation. The material provides the basis for the qualitative analysis presented in Chapter 7 and is included to improve transparency and reproducibility.

Questions:	Expert A	Expert B
Role, experience with HARA/TARA?	My expertise lies in security and functional safety. Until recently (2024, 2025) I have worked with developing a HARA/TARA method in the train industry.	I have recently finished my phd and before this i worked at Volvo, cybersecurity department. I mostly performed TARA-analysis but HARA-analysis at the end of my tenure. You could call me an expert in cybersecurity testing. We perform threat modeling in energy communites.
Industries worked in (automotive, cybersecurity, etc.	Train industry	Automotive and cybersecurity.
Familiar methods (ISO 26262, HEAVENS, etc.)	IEC62443, TS50701	ISO26262, ISO/SAE21434 and Heavens.
How do you currently perform HARA/TARA?	In the train project we chose out a specific system in a train and tested the system with a threat analysis. As a part of the project all stakeholders were invited to conduct common exercises. An important part of the method is to investigate essential functions, such as "communication"; What happens if communication is broken regarding; confidentiality, integrity and availability? An impact study was also investigated.	We start with understanding the system and how it works. There are different definitions of systems on a network, and we need to understand the demands, flows in a network and then threat modeling. You also have to decide at what level to start the HARA/TARA analysis. We often finish by compiling a report of the entire work.
What tools or frameworks do you use?	Excel mainly has tested Threatget but threatget costs.	Excel and Microsoft TMT, but this is however a bit limited.
What are the main pain points?	In previous projects the excel file grew quite large, the amount of tabs. It was also quite hard to navigate through the big project file. The extraction of certain information from the excel file was also quite hard in some cases.	I believe that if i for example, have a DFD-diagram the translation to threat evaluation and threat modeling. An automatic translation would be good.
First impressions	This tool could be useful, and you become curious to go through our own excel file and see if this could be translated to the tool. There is a great interest for these kinds of tools, "Threatget" is not open source so something like this tool could be great. This tool would be better than excel.	I believe that the tool is pretty good and pretty detailed. There is also a connection to an LLM which generates risks. However, there is a lot of manual work as well. Another positive is how depth is represented, but in next steps of the development more AI-functionality would be good.
Clarity of concept & Perceived purpose	It's very clear with the agnosticity within the tool. The building blocks seem to be there. The method I use has a lot of steps you need to keep in mind. What I would think the tool needs is maybe an AI summarizer of risk sheets. Secondly the possibility of giving access to different parts of a sheet to different people. Also the possibility to easily export some parts of the sheet and not the whole.	I think the clarity of concept is pretty good as follow along to your (Interviewer) instructions. Should I be working alone in the tool it would be more difficult. Perhaps a demo would be appropriate. It would take time to learn on your own. It is not clear that DFD risks is DFD risks so a filtration of DFD-risks/components would be good. I see a lot of possibilities in the tool.

Table A.1: Questions and answers part 1.

Questions:	Expert A	Expert B
How do you perceive the idea of separating the tool from a specific risk method?	This is what makes me most curious about this tool. What I thought about is that in our method, an initial risk assessment is conducted, where only impact is considered and if the risk is big enough you do a more detailed analysis of the risk. In the more detailed analysis things like probability and impact are considered. This could differ between methods. I wonder if there could be more support aside from just listing risks and the ability to change columns. Another thing to consider is, for example, evaluation of the threat landscape. This method identifies different actors, such as hackers, government and competitors, and for each actor evaluate what capability, intention and so on. The goal of this is to get a probability assessment per actor. This would probably be possible to do in this tool. In a method I have used, there is a step in which you use a list of attacks, provided by Mitre, to analyse risk. With this in mind it would be nice if this list of attacks could be shown and if there was a possibility to generate a report which declares what attacks could be applied to the system. Finally, a valuable thing that the tool could provide is the possibility to list: The essential functions the ToE has, for example headway between trains. It might be possible with this tool.	I think that the agnostic approach is the best part of the tool. It is more general, if you can apply several types of components and systems to the tool.
What advantages or risks do you see in this approach?	Already answered in earlier question	Advantages: It is not tied to a specific system and creates more possibilities. If I want to do HARA/TARA on a vehicle, and want to change something in my model, I can. If I build a specific model for a system I can continue building upon it. Risks/disadvantages: There are some specific systems within domains such as nuclear power where there is a need for specific tools that this tools would not be able to handle. Should there be any tools like this i would have to build the framework from scratch beforehand.

Table A.2: Questions and answers part 2.

Questions:	Expert A	Expert B
Would this conflict with the standards you use?	No	No, tools makes it easier. You could connect more with AI here to faster be able to get a first draft going, so the analyst can go more detailed right away. Perhaps automatic inline recommendation would be another benefit.
How well does this fit into your current workflow?	It would fit well	It is hard for me to determine exactly how much this tool would be able to help me. I do not know if I could connect the tool to my existing DFD;s
What would make it easier to adopt?	The main thing would be if it is possible to change a functionality or add a new one easily through the editor, without having to change in the code base.	As mentioned before, importing an entire DFD-workflow would make it easier.
Where would users struggle?	Users may struggle with choosing the right level of detail, balancing complexity with simplicity. Should we treat the system as a black box or more detailed? Secondly, how do we synchronize the different rationales and motivations of the users.	Risks that are added are all listed in one sheet. It will be difficult to understand the source of the risk and what risks are connected. A filtering of risks would be needed to alleviate this.
How does this compare to your current tools or methods?	Compared to excel, this solution is better for sharing and collaboration and has a more user friendly and visually appealing interface. But could use some more clarity on what step of a method you're on.	There are no templates like in Microsoft TMT and it gets very complex, there is no way to generate reports of the work. There is a possibility of creating macros. What is good with Microsoft TMT and alike is templates that help the user which is the main strength of commercial tools.
In what situations would this tool be preferable?	This tool would be preferable in situations in which I need to share a specific document and when risks needs to be separated and divided.	This is of course heavily dependent on the use case, TMT have analysis mode that finds vulnerability by itself so we can add vulnerabilities automatically.
Would you use this tool? Why/why not?	Yes absolutely, there is interest for such a tool but development takes time. If one was comfortable with the tool I would think I could use it in a project.	I have not yet, but I will be using this tool.
What is the single most important improvement?	This tool would be easier to use.	There are two things. One, if you could import a DFD directly and two, more AI features.
Anything else we didn't cover?	I would be interested in seeing real tests conducted in this tool where users got to use the tool and give feedback.	Something I really would like is a good guide of how to use the tool. That and what type of open source this will be. 1. A description on Github on how to get the tool running and how to use it. 2. If there is interest, give tips and suggestions on how future developers can connect new features to the existing code base and tell developers about where they should start.

Table A.3: Questions and answers part 3.